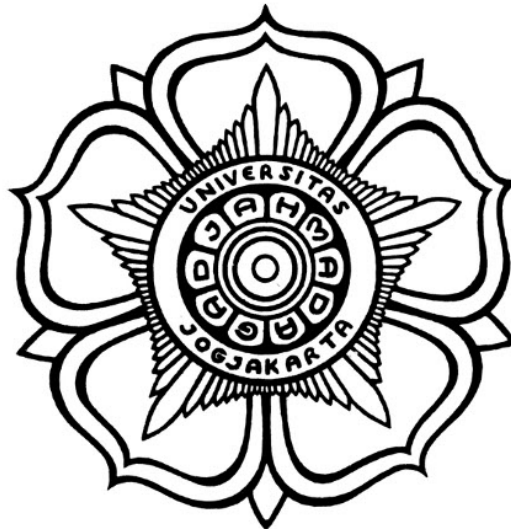


**ALGORITMA KONSENSUS *SCALABLE* DENGAN PEMBOBOTAN
REPUTASI UNTUK *BYZANTINE-ROBUST ORACLE* BERBASIS
LLM**

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Peace, Justice and Strong Institutions
Partnerships for the Goals

Disusun oleh:
Deren Tanaphan
22/503261/TK/54976

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

ALGORITMA KONSENSUS *SCALABLE* DENGAN PEMBOBOTAN REPUTASI UNTUK *BYZANTINE-ROBUST ORACLE* BERBASIS LLM

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Deren Tanaphan
22/503261/TK/54976

Telah disetujui dan disahkan

Pada tanggal 02 Juni 2026

Dosen Pembimbing I

Dosen Pembimbing II

Dr. Ir. Guntur Dharma Putra, S.T., M.Sc.
NIP 199104132024061001

Widyawan, S.T., M.Sc., Ph.D.
NIP 197312042002121001

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Deren Tanaphan
NIM : 22/503261/TK/54976
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Informasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 4 Juni 2026



Deren Tanaphan
22/503261/TK/54976

HALAMAN PERSEMBAHAN

“Scientists study the world as it is; engineers create the world that has never been.”

– Theodore von Kármán

Tugas akhir ini kupersembahkan kepada kedua orang tuaku, keluargaku, dan teman-temanku yang telah memberikan dukungan tanpa henti, serta bagi perkembangan ilmu pengetahuan dan kemajuan peradaban manusia.

KATA PENGANTAR

Puji syukur kepada Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya sehingga penulis telah menyelesaikan tugas akhir yang berjudul *Algoritma Konsensus Scalable* dengan Pembobotan Reputasi untuk *Byzantine-Robust Oracle* Berbasis LLM. Dalam menjalani masa perkuliahan dan proses penyusunan tugas akhir ini, penulis mendapatkan banyak bimbingan, dukungan, dan semangat dari berbagai pihak. Oleh karena itu, penulis ingin berterima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE. selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Dr. Ahmad Nasikun, S.T., M.Sc. selaku Ketua Program Studi Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
3. Bapak Dr. Ir. Guntur Dharma Putra, S.T., M.Sc. selaku Dosen Pembimbing Skripsi I yang selalu memberikan arahan, masukan, saran, dan motivasi serta dukungan yang luar biasa kepada penulis selama pengerjaan tugas akhir.
4. Bapak Widyawan, S.T., M.Sc., Ph.D. selaku Dosen Pembimbing Skripsi II yang senantiasa memberikan dukungan dalam pengerjaan tugas akhir ini.
5. Bapak Prof. Ir. Paulus Insap Santosa, M.Sc., Ph.D., IPU. selaku Dosen Pembimbing Akademik yang memfasilitasi kebutuhan akademik penulis selama masa perkuliahan.
6. Ayah penulis, Sunardi Tanaphan yang telah bekerja tanpa lelah dan memberikan dukungan penuh sepanjang hidup penulis.
7. Ibu penulis, Roknih yang telah memberikan kasih sayang yang tidak terhingga, mendidik, dan memberikan doa di setiap langkah kehidupan penulis serta menjadi kekuatan bagi penulis agar terus berjuang.
8. Paman penulis, Lian Coan yang telah memberikan dukungan baik secara moral, finansial, dan lain sebagainya selama hidup penulis.
9. Kakak-kakak penulis, Oktaviani Tanaphan, Sinthia Tanaphan, S.H., Elsha Tanaphan serta seluruh keluarga besar yang selalu memberikan dukungan.
10. Angelina Wijaya yang selalu mendukung dan memberikan *support* kepada penulis di saat suka dan duka.
11. Eben, Fawwaz, Yitzhak, Dani, Harun, Brandon, Ghalib, Grandiv, Yahya, Bintang, Sakti, dan teman-teman lainnya yang tidak dapat dituliskan satu per satu yang telah memotivasi penulis untuk terus mengembangkan diri dan juga menemani serta mendukung penulis dalam menjalani kehidupan perkuliahan.
12. Rekan-rekan penulis selama berdinamika dan berkegiatan di luar kegiatan akademik, tim Propulsi GAMAFORCE 2024, Pengurus Harian GAMAFORCE 2025, KKN Sapuran Berperan 2025, ArachnoVa, dan lainnya yang tidak dapat penulis sebutkan semuanya.
13. Seluruh pihak lain yang tidak dapat dituliskan semuanya yang telah membantu, baik secara langsung maupun tidak langsung dalam kehidupan perkuliahan dan penulisan

tugas akhir ini.

Penulis menyadari bahwa tugas akhir ini masih jauh dari sempurna dan memiliki banyak kekurangan. Oleh karena itu, semua jenis saran, kritik, dan masukan yang bersifat membangun sangat diharapkan. Akhir kata, penulis berharap semoga tugas akhir ini dapat memberikan manfaat bagi kita semua. *Saddhu! Saddhu! Saddhu!*

Yogyakarta, 4 Juni 2026

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	vi
DAFTAR ISI	vii
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
DAFTAR SINGKATAN.....	xii
INTISARI.....	xiii
ABSTRACT	xiv
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
1.4 Batasan Penelitian	5
1.5 Manfaat Penelitian	6
1.6 Sistematika Penulisan.....	6
BAB II Tinjauan Pustaka dan Dasar Teori	8
2.1 Tinjauan Pustaka	8
2.1.1 Arsitektur <i>Leader-Driven</i> pada <i>Multi-Agent</i> LLM	8
2.1.2 Arsitektur <i>Leaderless</i> dan Masalah Egaliter	8
2.1.3 Mekanisme Insentif dan Reputasi Dinamis	9
2.2 Dasar Teori	11
2.2.1 <i>Large Language Model</i> (LLM) dan <i>Multi-Agent Systems</i> (MAS) ..	11
2.2.2 <i>Blockchain</i> dan <i>Smart Contract</i>	12
2.2.3 Sistem Terdistribusi dan Toleransi Kesalahan <i>Byzantine</i>	14
2.2.4 Kompleksitas Algoritma dan Notasi Asimtotik.....	16
2.2.5 Agregasi Spasial dan <i>Robust Statistics</i>	17
2.2.6 <i>Trust</i> , <i>Reputation</i> , dan <i>Probabilistic Selection</i>	19
2.3 Analisis Perbandingan Metode	21
BAB III Metode Penelitian.....	24
3.1 Alat dan Bahan Tugas akhir	24
3.1.1 Alat Tugas akhir.....	24
3.1.2 Bahan Tugas akhir	25
3.2 Metode yang Digunakan.....	25
3.2.1 Desain Penelitian.....	26

3.2.2	Pemodelan Sistem dan Aktor Jaringan	26
3.2.3	Arsitektur yang Diusulkan	28
3.2.3.1	<i>Agent Layer (Off-Chain)</i>	29
3.2.3.2	<i>Consensus Layer (Middleware)</i>	30
3.2.3.3	<i>Blockchain Layer (On-Chain)</i>	31
3.2.4	Tahapan Penerapan Metode Sistem	32
3.2.4.1	Evaluasi Ruang Vektor 5-Dimensi	32
3.2.4.2	Seleksi Komite Probabilistik (<i>Softmax Sharding</i>)	33
3.2.4.3	Agregasi Spasial <i>Weighted Geometric Median</i> (WGM) .	35
3.2.4.4	Pembaruan Reputasi Dinamis berbasis <i>Exponential Moving Average</i> (EMA).....	37
3.3	Alur Penelitian	38
3.3.1	Analisis dan Pemodelan	39
3.3.1.1	Studi Literatur.....	39
3.3.1.2	Analisis <i>State-of-the-Art</i> (SOTA) dan Perumusan Masalah	39
3.3.1.3	Perancangan Arsitektur dan Model Matematis	39
3.3.2	Pengembangan <i>Prototype</i> Sistem	40
3.3.2.1	Pembuatan Entitas Agen LLM	40
3.3.2.2	Implementasi Modul <i>Sharding</i> Komite	40
3.3.2.3	Implementasi <i>Weighted Geometric Median</i> (Weiszfeld) .	41
3.3.2.4	Implementasi <i>Exponential Moving Average</i> (EMA)	43
3.3.2.5	Pengembangan <i>Smart Contract</i> (Pembaruan Reputasi dan <i>Tie-Breaker</i>)	45
3.3.2.6	Pengujian Integrasi dan Iterasi <i>Debugging</i>	47
3.3.3	Pengujian dan Laporan	48
3.3.3.1	Simulasi dan <i>Benchmark</i>	48
3.3.3.2	Ekstraksi dan Analisis Metrik.....	49
3.3.3.3	Evaluasi Sistem dan Kalibrasi Parameter.....	50
3.3.3.4	Visualisasi Data dan Penyusunan Laporan	51
3.4	Masalah dan Keterbatasan <i>Prototype</i> Penelitian.....	52
BAB IV	Hasil dan Pembahasan.....	53
4.1	Implementasi dan <i>Proof of Concept</i> Sistem	53
4.1.1	Lingkungan Eksekusi dan Orkestrasi Agen.....	53
4.1.2	Implementasi Lapisan <i>Off-Chain</i>	54
4.1.2.1	Implementasi <i>Softmax Sharding</i>	54
4.1.2.2	Implementasi <i>Weighted Geometric Median</i> (WGM).....	56
4.1.2.3	Implementasi <i>Exponential Moving Average</i> (EMA)	57
4.1.2.4	Orkestrasi Aliran Data Antar-Modul	59

4.1.3	Implementasi Lapisan <i>On-Chain</i>	60
4.1.3.1	Penyimpanan Reputasi pada <i>Ledger</i>	60
4.1.3.2	Verifikasi <i>Tie-Breaker</i> Kriptografis <i>On-Chain</i>	63
4.1.4	Konfigurasi Hiperparameter Sistem	65
4.1.4.1	Ukuran Komite ($K = 10$)	65
4.1.4.2	Reputasi Awal ($T_0 = 0,5$)	66
4.1.4.3	Temperatur <i>Softmax</i> ($\tau = 0,3$)	66
4.1.4.4	Faktor <i>Smoothing</i> EMA ($\lambda = 0,3$)	66
4.1.4.5	Toleransi Konvergensi WGM ($\epsilon = 10^{-6}$, $I_{\max} = 100$)...	66
4.1.5	<i>Dataset</i> Evaluasi dan Skenario Ancaman <i>Byzantine</i>	67
4.1.5.1	<i>Dataset</i> Evaluasi	67
4.1.5.2	Mekanisme Evaluasi dan Penggabungan <i>Dataset</i>	68
4.1.5.3	Skenario Injeksi Ancaman <i>Byzantine</i>	68
4.1.6	Metodologi Komparasi Baseline dan <i>Passive Shadow Tracking</i>	72
4.2	Analisis Skalabilitas dan Efisiensi Komunikasi	73
4.3	Evaluasi Ketahanan <i>Byzantine</i> dan <i>Meritocratic Safety Margin</i>	75
4.3.1	Keunggulan Meritokrasi pada Lingkungan Normal	75
4.3.2	Pemulihan Dinamis pada Batas Ekstrem.....	76
4.3.3	Ilusi Ketahanan Pasif.....	78
4.4	Dinamika Pelacakan Reputasi dan Penalti <i>Stateful</i>	79
4.5	Limitasi dan <i>Trade-Off</i> Sistem	81
BAB V	Kesimpulan dan Saran.....	83
5.1	Kesimpulan.....	83
5.2	Saran.....	84
DAFTAR PUSTAKA		85
LAMPIRAN		L-1
L.1	Kode Sumber dan Repositori	L-1
L.2	Bukti Ketiadaan Solusi Aljabar Tertutup (<i>Closed-Form Solution</i>) pada <i>Weighted Geometric Median</i> (WGM) dan Derivasi Algoritma Weiszfeld ..	L-1
L.3	Penurunan Matematis <i>Inverse Min-Max Normalization</i> pada Metrik Kinerja Evaluator	L-2
L.4	Daftar Model LLM yang Digunakan dalam Eksperimen	L-3

DAFTAR TABEL

Tabel 2.1	Ringkasan tinjauan pustaka	10
Tabel 2.2	Perbandingan arsitektur multi-agen terdahulu	23
Tabel 4.1	Konfigurasi hiperparameter sistem dan peran fungsionalnya	65
Tabel 4.2	Skenario injeksi ancaman <i>Byzantine</i> dan karakteristik operasionalnya	69
Tabel 4.3	Perbandingan metrik latensi antara <i>baseline</i> DecentLLMs dan sistem yang diusulkan	74
Tabel L.1	Daftar 15 model LLM <i>worker</i>	L-4
Tabel L.2	Daftar 15 model LLM <i>evaluator</i>	L-4

DAFTAR GAMBAR

Gambar 2.1	Visualisasi <i>autoregressive</i> pada LLM.	11
Gambar 2.2	Arsitektur penautan blok pada jaringan <i>blockchain</i>	13
Gambar 2.3	Organisasi lapisan <i>middleware</i> pada sistem terdistribusi.	14
Gambar 2.4	Kegagalan konsensus pada permasalahan tiga jenderal <i>Byzantine</i>	15
Gambar 2.5	Visualisasi <i>Geometric Median</i>	19
Gambar 3.6	Arsitektur tiga lapisan <i>Reputation-Weighted Consensus</i>	29
Gambar 3.7	Diagram blok alur komputasi <i>Reputation-Weighted Consensus</i> ...	32
Gambar 3.8	Alur penelitian tiga fase	38
Gambar 4.9	Perbandingan waktu eksekusi rata-rata per ronde	73
Gambar 4.10	Timeline latensi per ronde	74
Gambar 4.11	Perbandingan akurasi konsensus tanpa serangan	76
Gambar 4.12	Evolusi <i>Meritocratic Safety Margin</i> pada <i>Amplification Collusion</i>	77
Gambar 4.13	<i>Dual-panel Meritocratic Safety Margin</i> vs akurasi pada <i>Random Attack</i>	78
Gambar 4.14	Persentase partisipasi agen tidak jujur dalam komite	79
Gambar 4.15	Evolusi reputasi agen yang melakukan <i>Inverse Attack</i>	80
Gambar 4.16	Evolusi reputasi agen yang melakukan <i>Oscillation Attack</i>	80
Gambar L.1	<i>QR Code Repository</i> GitHub	L-1

DAFTAR SINGKATAN

W	=	Agen Pekerja
E	=	Agen Evaluator
N	=	Jumlah total agen dalam jaringan
K	=	Ukuran komite (jumlah evaluator terpilih)
T / T_i	=	Skor Reputasi evaluator i
S_t	=	Skor performa pada ronde t
w_i	=	Bobot reputasi ternormalisasi evaluator i
Λ	=	<i>Honest Weight Fraction</i>
SOTA	=	<i>State of The Art</i>
NLP	=	<i>Natural Language Processing</i>
LLM	=	<i>Large Language Model</i>
API	=	<i>Application Programming Interface</i>
P2P	=	<i>Peer-to-Peer</i>
MAD	=	<i>Multi-Agent Debate</i>
EMA	=	<i>Exponential Moving Average</i>
WGM	=	<i>Weighted Geometric Median</i>
GM	=	<i>Geometric Median</i>
BFT	=	<i>Byzantine Fault Tolerance</i>
PoT	=	<i>Proof-of-Thought</i>
WBFT	=	<i>Weighted Byzantine Fault Tolerance</i>
HTTP	=	<i>Hypertext Transfer Protocol</i>

INTISARI

Keterbatasan *smart contract* dalam mengakses data dunia nyata selama ini dibatasi oleh *oracle* konvensional yang terbatas pada data terstruktur, padahal aplikasi modern semakin menuntut ekstraksi keputusan dari data tidak terstruktur. Untuk menjembatani keterbatasan tersebut, *Large Language Model* (LLM) diintegrasikan sebagai *blockchain oracle* karena kemampuannya dalam memproses data tidak terstruktur, sementara risiko halusinasi dimitigasi melalui implementasi sistem multi-agen. Dalam sistem multi-agen, beberapa agen independen menghasilkan keluaran yang bervariasi sehingga keputusan akhir perlu ditentukan melalui algoritma konsensus sebelum diserahkan ke *smart contract*. Untuk mengatasi *Byzantine Fault*, penelitian terdahulu mengandalkan protokol berbasis pemimpin (*leader-driven*) yang menciptakan titik kegagalan tunggal (*single point of failure*) apabila agen pemimpin bertindak manipulatif. Arsitektur konsensus tanpa pemimpin (*leaderless*) mengatasi kerentanan tersebut, tetapi menimbulkan tiga kelemahan: agregasi egaliter (semua agen memiliki pengaruh yang sama) yang memudahkan agen jujur maupun jahat mendegradasi akurasi konsensus, ketiadaan memori historis (*stateless*) sehingga manipulasi berulang tidak dikenai penalti, serta komunikasi antar-agen yang melonjak secara kuadratik atau $O(N^2)$. Untuk mengatasi permasalahan tersebut, diusulkan algoritma meritokratis yang menentukan pengaruh agen berdasarkan rekam jejak performanya. Algoritma ini mengintegrasikan dua mekanisme utama, yaitu seleksi komite melalui *softmax sharding* untuk memilih subset agen berukuran tetap secara probabilistik sehingga kompleksitas komunikasi menjadi konstan, serta pembobotan suara berdasarkan skor reputasi historis yang diperbarui secara dinamis menggunakan *Exponential Moving Average*. Skor reputasi tersebut kemudian digunakan dalam algoritma agregasi *Weighted Geometric Median*. Evaluasi melalui simulasi menunjukkan algoritma yang diusulkan menurunkan rata-rata latensi eksekusi sebesar 21,0% dibandingkan *baseline* pada kondisi normal. Mekanisme reputasi secara proaktif menekan bobot efektif agen jahat menjadi 26,6% pada kondisi 40% infiltrasi jaringan, serta mempertahankan akurasi pada 76% di bawah 50% serangan *Byzantine* terkoordinasi ketika *baseline* turun ke 70%. Pada akhirnya, pendekatan ini menyediakan *data feed* yang aman dan berlatensi rendah, serta meningkatkan kapabilitas *smart contract* untuk mengotomatisasi keputusan dari data dunia nyata yang tidak terstruktur.

Kata kunci : *decentralized oracle*, LLM, *multi-agent system*, *scalable*, *byzantine-robust*

ABSTRACT

The inability of smart contracts to directly access real-world data has traditionally been addressed by conventional oracles, which are primarily limited to structured data. However, modern smart contract applications increasingly require decision extraction from unstructured data. To address this limitation, Large Language Model (LLM) are integrated as blockchain oracles due to their capability to process unstructured information, while hallucination risks are mitigated through a multi-agent system. Within such a system, multiple independent agents may produce divergent outputs, necessitating a consensus algorithm to determine a final value before submission to the smart contract. To tolerate Byzantine faults, prior studies have relied on leader-driven protocols, which introduce a single point of failure when the leader behaves maliciously. Although leaderless consensus architectures eliminates this vulnerability, they still suffer from three major limitations: egalitarian aggregation, where all agents exert equal influence regardless of reliability; the absence of historical memory, which prevents repeated malicious behaviour from being penalized; and quadratic communication complexity, $O(N^2)$, arising from extensive inter-agent communication. To address these challenges, this study proposes a meritocratic consensus algorithm that determines agent influence based on historical performance. The proposed approach integrates two key mechanisms: committee selection through softmax sharding, which probabilistically selects a fixed-size subset of agents to achieve constant communication complexity, and vote weighting based on historical reputation scores dynamically updated using Exponential Moving Average. These reputation scores are subsequently incorporated into a Weighted Geometric Median aggregation algorithm. Simulation results demonstrate that the proposed algorithm reduces average execution latency by 21.0% compared to the baseline under normal conditions. Furthermore, the reputation mechanism proactively suppresses the effective influence of malicious agents to 26.6% under a network infiltration rate of 40%, while maintaining 76% consensus accuracy under a coordinated Byzantine attack involving 50% malicious agents, where the baseline accuracy drops to 70%. Overall, the proposed approach provides a secure and low-latency data feed while enhancing the capability of smart contracts to automate decisions derived from unstructured real-world data.

Keywords : decentralized oracle, LLM, leaderless multi-agent system, reputation-weighted consensus, byzantine robustness

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan *Large Language Models* (LLM) telah mengalami kemajuan yang sangat pesat, yang ditandai dengan meningkatnya popularitas ChatGPT [1] sejak awal kemunculannya pada akhir tahun 2022. Pada fase awal ini, fungsionalitas LLM umumnya masih terbatas pada model interaksi percakapan, yaitu pengguna bertanya dan model akan menjawab pertanyaan tersebut seperti melakukan *chatting*. Namun, akhir-akhir ini, arah perkembangan LLM telah mengalami pergeseran paradigma ke arah konsep *AI Agent*. Berbeda dengan LLM konvensional yang hanya sekedar menjawab pertanyaan, *AI agent* memungkinkan LLM untuk melakukan penalaran, perencanaan, dan interaksi langsung dengan *environment* eksternal. Salah satu contohnya adalah *AI agent* sekarang memiliki kemampuan untuk membaca dan menulis file, menggunakan *tools*, hingga melakukan pemanggilan API. Evolusi ini telah membuka jalan bagi *agent* untuk bertindak sebagai *personal assistant* bagi kita. Hal ini terbukti dari fenomena kehadiran *agent* interaktif seperti Claude Code [2] yang dirilis oleh Anthropic pada awal tahun 2025, yang mampu bertindak sebagai *autonomous coding assistant*. Memasuki tahun 2026, tren *personal assistant AI agent* menjadi hal yang sering dibicarakan. Antusiasme yang sangat tinggi ini dibuktikan oleh dukungan komunitas di GitHub, yaitu proyek *open-source personal agent* seperti OpenClaw [3] dan Hermes Agent [4] meraih lebih dari 100 ribu GitHub stars.

Akan tetapi, terlepas dari kemampuan yang sangat canggih tersebut, penerapan LLM *agent* secara tunggal masih dihadapkan pada satu permasalahan utama yang sangat krusial, yaitu halusinasi [5–7]. Fenomena halusinasi ini terjadi ketika LLM menghasilkan konten yang terlihat benar dan logis, namun pada kenyataannya memuat informasi yang sepenuhnya direayasa, salah secara faktual, atau tidak didukung oleh bukti. Tingkat keparahan masalah ini sangat nyata, data statistik dari pengujian *benchmark* tahun 2026 terhadap 37 model menunjukkan bahwa tingkat halusinasi rata-rata LLM masih berada di kisaran 15% hingga 52% [7]. Pada domain kritis yang membutuhkan tingkat akurasi absolut, risiko ini jauh menjadi lebih fatal, sebab tingkat halusinasi dapat mencapai 58% hingga 88% pada kueri hukum, dan pada ringkasan kasus medis, tingkat kesalahan dapat menyentuh angka 64,1% jika tidak ada mekanisme *safeguards* [7]. Jika dibiarkan beroperasi sendirian dalam lingkungan yang terbuka, variansi akurasi dan kerentanan ini membuat agen LLM bertindak sebagai *single point of failure*. Fakta bahwa bahkan model-model LLM terbaik saat ini masih menghasilkan halusinasi di atas 15% dalam tugas penalaran akhirnya memotivasi perlunya *multi-agent systems*.

Secara arsitektural, *smart contract* beroperasi di dalam lingkungan *blockchain* yang bersifat deterministik dan terisolasi dari dunia luar, atau sering digambarkan sebagai "*walled gardens*" [8]. *Smart contracts* tidak bisa secara langsung mengakses data eksternal karena sifat deterministik dari *blockchains*. Keterbatasan ini membuat *smart contract* tidak memiliki kemampuan untuk mengetahui kondisi/data diluar lingkungan *onchain*. Oleh karena itu, jaringan *blockchain* membutuhkan suatu perantara yang disebut *oracle* untuk menjembatani antara komputasi *onchain* dan realitas *offchain* [8].

Secara historis, *oracle* konvensional telah berhasil memfasilitasi integrasi data tersebut, namun fungsinya masih sangat terbatas pada penyampaian data numerik atau terstruktur yang sederhana tanpa adanya kemampuan interpretasi lebih lanjut [9]. Seiring dengan berkembangnya ekosistem desentralisasi, muncul kesenjangan yang signifikan antara batasan *oracle* tradisional dengan kebutuhan *smart contract* modern yang semakin kompleks. Saat ini, banyak keputusan otomatisasi dalam *smart contract* tingkat lanjut yang bergantung pada ekstraksi data tidak terstruktur sehingga *oracle* numerik biasa tidak lagi memadai [9].

Untuk mengatasi kesenjangan kemampuan tersebut, integrasi LLM sebagai *oracle* menjadi semakin dipertimbangkan. Dengan memanfaatkan teknologi NLP dan kemampuan penalaran tingkat tinggi, LLM mampu menginterpretasikan, membedah, dan memverifikasi fakta dari sumber data tidak terstruktur seperti artikel berita global, laporan keuangan perusahaan, hingga dokumen regulasi hukum [9]. Sebagai contoh kasus penggunaan spesifik, LLM *oracle* sangat esensial dalam memfasilitasi penyelesaian pasar prediksi dengan menyimpulkan hasil langsung dari riset halaman web, memfasilitasi asuransi parametrik berbasis teks dan berita, hingga melakukan moderasi konten terdesentralisasi.

Menyadari besarnya potensi ini, arah industri *blockchain* secara aktif mulai bertransisi menuju pengembangan LLM *Oracle*. Berbagai pelopor industri telah menginisiasi pergerakan ini. Sebagai contoh, Chainlink Labs telah menguji coba prototipe *oracle* yang beberapa LLM digunakan untuk membedah siaran pers perusahaan dan mengekstraksi fakta ke dalam format terstruktur [10]. Di sisi lain, proyek seperti *Oraichain* telah meluncurkan jaringan khusus yang difungsikan sebagai *AI-centric oracle* untuk melayani kebutuhan analisis tingkat lanjut bagi *smart contract* [9]. Hal ini mengonfirmasi bahwa penerapan LLM *agent* sebagai *oracle* bukan sekadar wacana teoretis, melainkan fondasi infrastruktur nyata yang sedang dibangun oleh industri sehingga menuntut arsitektur yang tangguh untuk mengamankannya.

Meskipun LLM *oracle* cukup banyak dibutuhkan oleh industri, membangun infrastruktur yang tangguh tidak mungkin dicapai jika sistem hanya mengandalkan satu entitas LLM. Apalagi dengan sifat *blockchain* yang deterministik ini bertolak belakang dengan LLM yang memiliki kemungkinan untuk halusinasi. Untuk mengatasi kelemahan

tersebut, paradigma penelitian dan pengembangan kini telah bergeser menuju arsitektur *Multi-Agent Systems* (MAS) [11]. Dalam arsitektur ini, beban komputasi dan kognitif didistribusikan ke beberapa LLM *agent*, memungkinkan adanya spesialisasi dan eksekusi paralel untuk menyelesaikan tugas-tugas yang kompleks [12]. Berbeda dengan model agen tunggal, MAS memanfaatkan dinamika interaktif yang berbagai instansiasi LLM dapat saling berdiskusi dan bernalar secara interaktif untuk merumuskan solusi secara kolektif [13]. Melalui implementasi mekanisme koordinasi tingkat lanjut, agen-agen ini dapat secara proaktif mengusulkan jawaban, mengkritik dan menyempurnakan respons satu sama lain [11]. Pendekatan ini terbukti secara signifikan mampu memperbaiki konsistensi logis dan menekan kecenderungan halusinasi. Sebagai bukti empiris, evaluasi terhadap MAS menunjukkan bahwa mekanisme ini mampu mereduksi tingkat halusinasi secara drastis hingga 85,5% [14]. Keberhasilan dalam meminimalkan halusinasi ini secara efektif menghilangkan risiko *single point of failure* yang pada akhirnya menghasilkan kualitas jawaban yang lebih baik dibandingkan dengan sistem LLM tunggal.

Meskipun arsitektur MAS menawarkan kemampuan kolaborasi dan koreksi yang superior, penerapannya sebagai *oracle* di platform *blockchain* yang bersifat terbuka (*open networks*) memperkenalkan dimensi risiko yang baru. Jaringan terdesentralisasi ini beroperasi dalam lingkungan P2P (*peer-to-peer*) yang di dalamnya entitas yang berniat buruk, atau yang dikenal sebagai agen jahat, dapat berpartisipasi dan mengeksploitasi sistem [11]. Dalam konteks ini, agen jahat dapat dengan sengaja menyabotase proses kolaborasi dengan memberikan jawaban yang tidak akurat, mengganggu protokol, hingga melakukan serangan untuk menyesatkan pengguna atau *smart contract*. Selain serangan yang disengaja, agen yang ikut ke dalam MAS juga tetap berisiko untuk berhalusinasi yang dapat memicu perilaku yang tidak diinginkan tanpa disadari. Seiring dengan skala jaringan yang semakin membesar, ancaman ini menjadi semakin kritis karena agen-agent jahat tersebut berpotensi melakukan kolusi secara terkoordinasi untuk membajak hasil akhir [13]. Oleh karena itu, agar MAS dapat berfungsi secara aman sebagai *oracle* dalam *permissionless blockchain*, sistem tersebut membutuhkan mekanisme *Byzantine fault tolerance* yang tangguh untuk memastikan bahwa integritas data tetap terjaga meskipun jaringan disusupi oleh agen-agen yang tidak terpercaya.

Ketika banyak agen LLM beroperasi dan mengevaluasi masalah secara bersamaan, sistem membutuhkan sebuah mekanisme untuk menengahi perbedaan pendapat di antara mereka untuk menengahi perbedaan pendapat di antara mereka dan menyepakati satu jawaban akhir yang paling valid, yang dikenal sebagai mekanisme konsensus. Untuk memitigasi ancaman *Byzantine* di lingkungan terdesentralisasi, berbagai arsitektur konsensus untuk MAS telah diusulkan. Pendekatan awal umumnya mengandalkan arsitektur *leader-driven*, seperti penerapan mekanisme *Proof-of-Thought* atau variasi *Byzantine Fault Tolerance* (BFT) tradisional, yang satu agen ditunjuk secara khusus untuk mengoordini-

nasikan proses evaluasi [13]. Sayangnya, pendekatan ini memiliki kerentanan terhadap serangan pada agen pemimpin. Jika agen pemimpin bertindak secara *Byzantine*, sistem akan terpaksa melakukan siklus pengulangan protokol secara terus menerus yang akan menambah latensi [13]. Sebagai respons terhadap kelemahan *single point of failure* pada sistem *leader-driven*, arsitektur SOTA saat ini mulai beralih pada pendekatan tanpa pemimpin (*leaderless*).

1.2 Rumusan Masalah

Arsitektur tanpa pemimpin (*leaderless*) pada sistem multi-agen LLM memang telah berhasil mengeliminasi kerentanan titik kegagalan tunggal (*single point of failure*). Akan tetapi, ketiadaan koordinator terpusat ini justru memunculkan kerentanan baru pada mekanisme pencapaian konsensusnya. Oleh karena itu, rumusan masalah yang diangkat pada penelitian ini adalah sebagai berikut:

1. Algoritma agregasi yang bersifat egaliter memberikan bobot pengaruh sama rata kepada seluruh agen, sehingga agen jujur yang berhalusinasi maupun agen jahat memperoleh pengaruh yang sama dengan agen yang andal. Hal ini menyebabkan konsensus akhir rentan terdegradasi oleh anomali informasi.
2. Arsitektur yang beroperasi secara *stateless* membuat sistem tidak memiliki memori historis terkait rekam jejak perilaku agen antar siklus konsensus. Ketidadaan memori lintas-siklus ini mengakibatkan sistem tidak mampu mendeteksi dan mendegradasi agen yang secara berulang menunjukkan performa buruk, sehingga agen dengan performa buruk dapat beroperasi tanpa penalti dan terus merusak integritas konsensus dari waktu ke waktu.
3. Ketidadaan koordinator terpusat memaksa konsensus beroperasi dengan komunikasi menyeluruh antar agen. Hal ini menyebabkan kompleksitas komunikasi melonjak secara kuadratik $O(N^2)$. Beban komunikasi meningkat secara kuadratik seiring bertambahnya populasi agen, memicu lonjakan latensi yang menghambat skalabilitas jaringan.

1.3 Tujuan Penelitian

Penelitian ini difokuskan pada upaya penyempurnaan algoritma konsensus pada arsitektur *leaderless* multi-agen LLM *oracle* yang sudah ada, agar memiliki ketahanan terhadap serangan *Byzantine* dan skalabilitas jaringan yang tinggi. Secara spesifik, tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang dan mengimplementasikan mekanisme konsensus yang mampu mendiferensiasi bobot suara setiap agen secara dinamis berdasarkan rekam jejak keandalannya, guna memitigasi degradasi hasil akibat variabilitas *scoring* agen yang jujur

dan perilaku agen jahat.

2. Merancang dan mengimplementasikan mekanisme pelacakan performa agen yang mampu memonitor rekam jejak performa setiap agen secara kumulatif serta memberikan penalti secara adaptif terhadap agen yang menunjukkan performa yang tidak konsisten atau berkualitas buruk.
3. Menerapkan mekanisme seleksi komite (subset agen) berbasis probabilitas untuk menekan beban kompleksitas komunikasi antar agen dengan tetap menjamin peluang representasi bagi agen baru dan mempertahankan keamanan jaringan.

1.4 Batasan Penelitian

Untuk menjaga agar penelitian tetap terfokus, terarah, serta dapat diselesaikan secara efektif dan akurat, ruang lingkup dan batasan pada penelitian ini didefinisikan sebagai berikut:

1. **Lingkungan Jaringan dan Simulasi:** Pengujian sistem multi-agen tidak diimplementasikan pada jaringan P2P terdesentralisasi secara fisik. Sebagai gantinya, interaksi multi-agen disimulasikan secara lokal pada lingkungan VPS dan menggunakan API komersial terpusat (*Open Router*). Oleh karena itu, metrik latensi eksekusi yang diukur akan mencakup latensi tambahan akibat batasan *rate limit* API (dibatasi oleh mekanisme *semaphore* berkapasitas 30 permintaan diwaktu yang bersamaan).
2. **Infrastruktur Blockchain:** Lapisan *blockchain* yang digunakan untuk menyimpan rekam jejak reputasi agen dan menjalankan *smart contract* pembaruan reputasi tidak di-deploy di *mainnet* publik, melainkan disimulasikan secara lokal menggunakan *testnet* yang kompatibel dengan Ethereum (Anvil) berbasis Solidity.
3. **Populasi Agen dan Komite:** Populasi agen LLM dibatasi pada 30 model LLM heterogen, yang dipartisi secara tetap menjadi 15 agen *worker* dan 15 agen *evaluator*. Mekanisme *Softmax Sharding Engine* untuk mereduksi kompleksitas komunikasi dari $O(N^2)$ menjadi $O(K^2)$ akan dibatasi pada pemilihan komite evaluator berukuran tetap, yaitu $K = 10$ agen per *round*.
4. **Dataset dan Kriteria Evaluasi:** Pengujian konsensus dilakukan selama 50 *rounds* (putaran) menggunakan *dataset* gabungan yang terdiri dari TruthfulQA [15] dan ARC-Easy [16]. Evaluasi kualitas jawaban oleh agen evaluator dibatas pada 5 dimensi penilaian (skala 0 - 20), yaitu: *Factual Contradiction*, *Factual Fabrication*, *Instruction Inconsistency*, *Context Inconsistency*, dan *Logical Inconsistency* [11].
5. **Model Ancaman Byzantine:** Evaluasi ketahanan sistem terhadap agen jahat dibatasi pada empat skenario serangan: serangan *Random*, serangan *Inverse*, serangan *Oscillation*, dan serangan *Amplification Collusion*. Pengujian batas toleransi kega-

galan (*fault tolerance threshold*) diuji hingga tingkat infiltrasi agen jahat maksimal sebesar 50% di dalam jaringan.

1.5 Manfaat Penelitian

Berdasarkan rumusan masalah dan tujuan yang telah ditetapkan, penelitian ini diharapkan dapat memberikan kontribusi dan manfaat yang signifikan, baik dari segi akademis maupun praktis, yang dijabarkan sebagai berikut:

1. Manfaat Akademis

- Berkontribusi pada bidang keilmuan sistem terdistribusi dan *Multi-Agent System* dengan mengusulkan model konsensus meritokratis pada arsitektur *leaderless multi-agent LLM oracle* yang membedakan kontribusi agen evaluator berbasis rekam jejak keandalan, sebagai alternatif dari model egaliter konvensional.
- Menyediakan analisis kompleksitas dan validasi empiris terhadap mekanisme seleksi komite untuk mereduksi beban komunikasi dari $O(N^2)$ menjadi $O(K^2)$ pada setiap putaran konsensus, yang dapat menjadi rujukan bagi perancangan sistem *multi-agent* terdesentralisasi.

2. Manfaat Praktis

- Menyediakan desain infrastruktur *oracle* LLM terdesentralisasi yang mengoptimalkan *trade-off* antara latensi eksekusi dan ketahanan terhadap agen jahat sehingga *smart contract* pada aplikasi terdesentralisasi dapat mengakses data dunia nyata kompleks.
- Menyediakan mekanisme pertahanan proaktif berbasis pembobotan reputasi dinamis bagi pengembang aplikasi terdesentralisasi untuk memitigasi risiko kegagalan konsensus dan kerugian finansial akibat manipulasi oleh agen jahat pada jaringan *oracle* terbuka.

1.6 Sistematika Penulisan

Untuk memberikan gambaran secara menyeluruh mengenai alur pembahasan dalam penelitian ini, penulisan disusun secara sistematis ke dalam 5 bab yang saling berkaitan dengan topik bahasan sebagai berikut:

1. Bab I membahas pendahuluan yang menguraikan konteks permasalahan, alasan mendasar dilakukannya penelitian, serta arah dan batasan ruang lingkup penelitian.
2. Bab II menyajikan tinjauan pustaka yang menelaah penelitian terdahulu dan mengidentifikasi celah yang belum terjawab, serta dasar teori yang membangun landasan

konseptual dan matematis bagi perancangan solusi.

3. Bab III menguraikan metode penelitian yang menjelaskan bagaimana solusi dirancang, dimodelkan, dan diuji, termasuk tahapan penerapan algoritma serta parameter evaluasi yang digunakan.
4. Bab IV menyajikan hasil dan pembahasan yang membuktikan kinerja solusi melalui eksperimen, meliputi analisis skalabilitas, ketahanan terhadap serangan, dinamika pelacakan reputasi, serta keterbatasan sistem.
5. Bab V berisi kesimpulan yang merangkum pencapaian penelitian dan saran untuk pengembangan penelitian selanjutnya.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Penelitian mengenai integrasi *Large Language Models* (LLM) ke dalam sistem multi-agen telah berkembang pesat untuk mengatasi kelemahan model tunggal, seperti halusinasi dan bias informasi. Namun, ketika agen LLM diimplementasikan pada jaringan terdesentralisasi terbuka seperti *blockchain*, sistem terekspos pada ancaman agen jahat (agen jahat yang mencoba memanipulasi hasil). Tinjauan pustaka ini membedah evolusi protokol koordinasi multi-agen yang tahan terhadap ancaman *Byzantine*, serta mekanisme pelacakan reputasi yang ada saat ini.

2.1.1 Arsitektur *Leader-Driven* pada *Multi-Agent* LLM

Generasi awal dari sistem multi-agen yang tahan terhadap serangan *Byzantine* umumnya sangat bergantung pada arsitektur berbasis pemimpin (*leader-driven*). Salah satu kerangka kerja utama di bidang ini adalah *BlockAgents* yang diusulkan oleh Chen et al. [13]. Kerangka kerja tersebut memperkenalkan mekanisme konsensus *Proof-of-Thought* (PoT). Dalam sistem ini, agen dibagi peran pekerja (*workers*) dan evaluator (*miners*) dengan satu agen pemimpin bertugas mengajukan proposal jawaban yang kemudian melalui proses pemungutan suara bergaya debat multi-putaran [13]. Pendekatan serupa juga diadopsi oleh Luo et al. [17] melalui *Trusted MultiLLMN* yang menggunakan konsensus *Weighted Byzantine Fault Tolerance* (WBFT). Sistem ini memberikan bobot pemungutan suara berdasarkan keandalan agen dan menggunakan pemimpin yang bergilir (*rotating leader*) untuk mencapai konsensus.

Meskipun kedua sistem ini berhasil memperkenalkan struktur keamanan, keduanya memiliki kelemahan arsitektural yang kritis. Kelemahan tersebut adalah ketergantungan pada agen pemimpin. Jika agen yang terpilih sebagai pemimpin secara berturut-turut adalah agen jahat, sistem akan berulang kali gagal mencapai konsensus dan harus mengulang protokol pemilihan pemimpin [11]. Kegagalan berulang ini memicu lonjakan latensi eksekusi yang tinggi dan menciptakan *single point of failure*.

2.1.2 Arsitektur *Leaderless* dan Masalah Egaliter

Untuk mengatasi masalah pada agen pemimpin, Jo et al. [11] mengusulkan *DecentLLMs*, sebuah arsitektur konsensus desentralisasi tanpa pemimpin (*leaderless*). Dalam protokol ini, agen pekerja menghasilkan jawaban secara paralel, sementara agen evaluator secara independen memberikan skor pada jawaban-jawaban tersebut tanpa perlu menunggu instruksi dari pemimpin [11]. *DecentLLMs* menggunakan algoritma agre-

gasi spasial *Geometric Median* (GM) untuk memilih jawaban akhir secara *Byzantine-robust* [11].

Meskipun arsitektur *leaderless* ini berhasil memecahkan masalah latensi akibat agen pemimpin yang jahat, protokol ini memunculkan kerentanan baru yang dikenal sebagai masalah keandalan evaluator (*evaluator reliability problem*). Dalam desain sistem terdistribusi, pendekatan egaliter merujuk pada mekanisme konsensus yang secara kaku memberikan nilai kepercayaan yang seragam dan bobot suara yang sama rata kepada setiap agen, tanpa mempertimbangkan kualitas, keandalan, atau rekam jejak historis dari agen tersebut. Karena algoritma GM pada *DecentLLMs* [11] beroperasi dengan prinsip egaliter ini, skor evaluasi dari agen yang jujur namun sedang berhalusinasi tidak dapat dibedakan dari skor agen yang *reliable*. Hal ini mendegradasi kualitas konsensus ketika variansi evaluasi dari agen jujur sangat tinggi. Selain itu, ketiadaan batasan komite pada arsitektur *leaderless* mewajibkan semua evaluator untuk saling mengirimkan skor evaluasi ke seluruh agen di jaringan, menciptakan hambatan skalabilitas akibat kompleksitas komunikasi menyeluruh yang mencapai skala $O(N^2)$.

2.1.3 Mekanisme Insentif dan Reputasi Dinamis

Untuk mengatasi ketidakkonsistenan performa agen, beberapa literature terbaru mulai mengeksplorasi penggunaan mekanisme reputasi. Chong et al. [18] mengusulkan *LLM-Net*, sebuah jaringan pakar berbasis *blockchain* dengan penyedia LLM yang dispesialisasikan berdasarkan domain pengetahuan. Sistem ini menggunakan mekanisme reputasi untuk memilih responden bekinerja tinggi melalui evaluasi sejawat. Di sisi lain, Qi et al. [12] mengusulkan kerangka kerja yang lebih dinamis dengan mengadopsi aturan pembaruan *Exponential Moving Average* (EMA) untuk melacak performa historis agen, serta menggunakan distribusi *Softmax* untuk menentukan probabilitas penugasan tugas berdasarkan skor keandalan agen tersebut.

Meskipun mekanisme pelacakan reputasi (seperti EMA) dan seleksi probabilistik (seperti *softmax*) telah terbukti efektif, literatur saat ini hanya mengimplementasikannya secara terbatas pada *task marketplace*. Dalam sistem yang dirancang oleh Qi et al. [12], *task marketplace* beroperasi layaknya sebuah bursa yang di dalamnya *smart contract* mengelola publikasi tugas, pemetaan kapabilitas, dan proses penawaran antar agen. Dalam sistem tersebut, metrik reputasi dan distribusi *Softmax* hanya dimanfaatkan sebagai mekanisme alokasi awal untuk menyeleksi dan mendistribusikan tugas kepada agen tertentu sebelum proses eksekusi dimulai. Belum ada penelitian yang mengintegrasikan metrik reputasi dinamis ini secara langsung ke dalam logika agregasi spasial pada konsensus *leaderless* untuk menimbang bobot suara evaluator secara proporsional. Inilah yang menjadi dasar pengembangan arsitektur konsensus berbasis pembobotan reputasi dalam penelitian ini.

Tabel 2.1. Ringkasan tinjauan pustaka

Referensi	Kontribusi Utama	Kelemahan / Limitasi (<i>Research Gap</i>)
Chen et al. [13]	<i>BlockAgents</i> : Koordinasi agen LLM dengan peran <i>worker</i> dan <i>miner</i> menggunakan konsensus <i>Proof-of-Thought</i> (PoT).	Rentan terhadap agen pemimpin (<i>leader</i>). Jika pemimpin bersifat Byzantine, latensi sistem akan melonjak drastis akibat kegagalan konsensus.
Luo et al. [17]	<i>Trusted MultiLLMN</i> : Jaringan multi-agen berbasis <i>Weighted Byzantine Fault Tolerance</i> (WBFT) dengan pemimpin bergilir.	Ketergantungan pada pemimpin bergilir tetap tidak sepenuhnya menghilangkan <i>single point of failure</i> dan isu latensi.
Jo et al. [11]	<i>DecentLLMs</i> : Arsitektur <i>leaderless</i> yang memisahkan generasi jawaban dan evaluasi, menggunakan agregasi <i>Geometric Median</i> (GM).	Rentan terhadap variasi evaluator jujur (<i>evaluator reliability problem</i>) akibat bobot suara egaliter, serta beban komunikasi yang tidak <i>scalable</i> ($O(N^2)$).
Chong et al. [18]	LLM-Net: Desentralisasi LLM-as-a-Service dengan jaringan pakar menggunakan validasi <i>blockchain</i> dan evaluasi reputasi.	Reputasi digunakan sebagai referensi historis, belum disuntikkan sebagai parameter bobot suara konsensus agregasi spasial yang aktif.
Qi et al. [12]	Mengusulkan pembaruan reputasi <i>Exponential Moving Average</i> (EMA) dan distribusi seleksi <i>Softmax</i> untuk agen LLM.	Mekanisme EMA dan Softmax hanya diterapkan untuk alokasi tugas awal (<i>task allocation marketplace</i>), belum diintegrasikan ke dalam perhitungan algoritma konsensus.

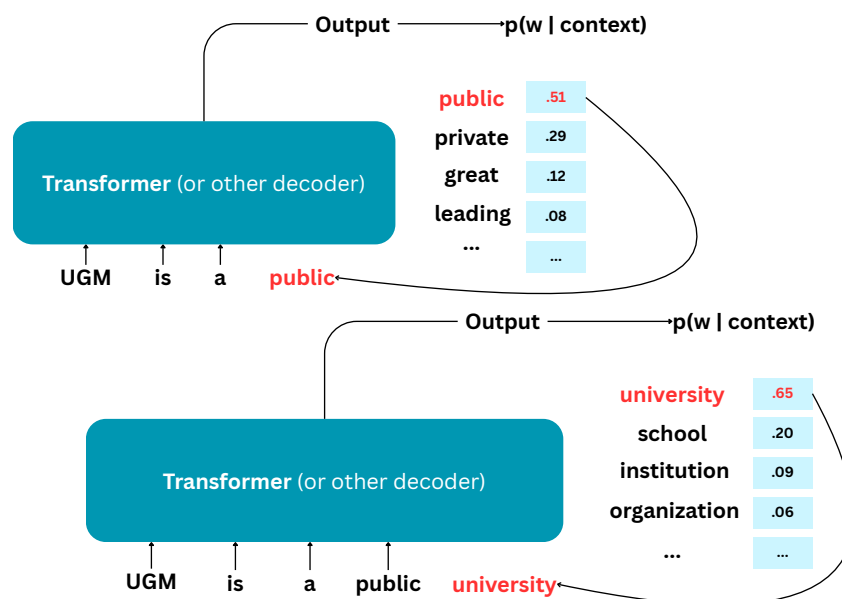
Berdasarkan ringkasan pada Tabel 2.1 dapat ditarik kesimpulan bahwa evolusi arsitektur multi-agen LLM saat ini masih menyisakan celah penelitian. Pendekatan *leader-driven* terbukti rentan terhadap lonjakan latensi dan *single point of failure* akibat agen pemimpin yang bersifat *Byzantine*. Di sisi lain, meskipun arsitektur *leaderless* berhasil mengeliminasi ketergantungan pada agen pemimpin, penggunaan agregasi yang egaliter justru memicu *evaluator reliability problem* serta hambatan skalabilitas komunikasi akibat keharusan *broadcast* secara menyeluruh ke semua agen. Sementara itu, literatur yang membahas mekanisme reputasi dinamis sejauh ini hanya menyerapkannya pada fase pra-pemrosesan alokasi agen dan belum menyentuh inti dari perhitungan konsensus. Oleh karena itu, penelitian ini memposisikan diri untuk menjembatani celah-celah tersebut. Dengan mengintegrasikan metrik keandalan historis ke dalam algoritma agregasi spasial serta memanfaatkan probabilitas seleksi untuk membatasi ukuran komite, penelitian ini bertujuan untuk menghadirkan solusi *oracle* yang tidak hanya tahan terhadap agen jahat,

namun juga *scalable* dan memiliki latensi eksekusi yang rendah.

2.2 Dasar Teori

2.2.1 *Large Language Model (LLM)* dan *Multi-Agent Systems (MAS)*

LLM adalah agen komputasional yang dilatih menggunakan korpus teks dalam skala masif untuk memprediksi token (atau kata) selanjutnya secara berurutan berdasarkan konteks sebelumnya [19]. Model *state-of-the-art* saat ini pada umumnya dibangun diatas arsitektur *Transformer*, sebuah arsitektur jaringan saraf yang secara eksklusif mengandalkan *attention mechanism* dan meninggalkan pemrosesan sekuensial tradisional seperti pada *Recurrent Neural Network (RNN)* [20]. Khususnya, LLM generatif beroperasi menggunakan arsitektur *Transformer* tipe *decoder*. Kunci utama dari arsitektur ini adalah mekanisme *self-attention* yang memungkinkan model untuk menghubungkan dan menghitung representasi dari berbagai posisi token di dalam sebuah sekuens masukan tanpa terikat oleh jarak posisinya [20]. Dalam proses pembuatannya, LLM dilatih secara *autoregressive*, yaitu model menghasilkan keluaran dari kiri ke kanan dengan mengonsumsi token yang dihasilkan sebelumnya sebagai tambahan konteks untuk memprediksi probabilitas matematis dari token berikutnya [19, 20]. Gambar 2.1 adalah visualisasi dari cara kerja LLM.



Gambar 2.1. Visualisasi *autoregressive* pada LLM. Gambar merupakan hasil modifikasi dari Jurafsky dan Martin [19]

Meskipun mekanisme ini memungkinkan LLM menghasilkan teks yang sangat fasih dan koheren, model yang beroperasi secara tunggal dan terisolasi memiliki kere-

tanam yang fatal, yaitu halusinasi. Halusinasi pada LLM adalah fenomena berupa model yang menghasilkan informasi yang secara sintaksis dan linguistik terlihat meyakinkan, namun pada kenyataannya salah secara faktual atau tidak didukung oleh realitas eksternal [19]. Oleh karena itu, mengandalkan satu agen LLM yang terisolasi sebagai pembuat keputusan deterministik memiliki risiko *single point of failure* yang sangat tinggi.

Untuk mengatasi keterbatasan dan kerentanan halusinasi pada model tunggal, paradigma kecerdasan buatan kini bertransisi dari pendekatan model terisolasi menuju pendekatan yang berpusat pada kolaborasi, yang diwujudkan melalui *Multi-Agent System* (MAS) [21]. Sistem multi-agen berbasis LLM memungkinkan sekelompok agen cerdas untuk saling berkoordinasi, mempersepsikan lingkungan, belajar, menalar, dan bertindak secara kolaboratif guna memecahkan tugas-tugas kompleks dalam skala besar [21].

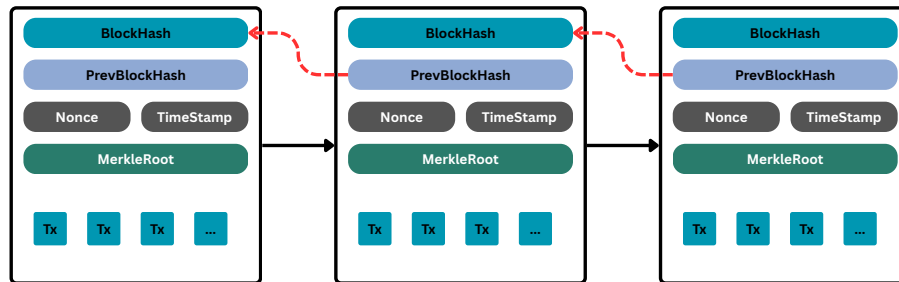
Dalam arsitektur MAS, beban kognitif tidak lagi ditanggung oleh satu model. Karakteristik utama dari sistem ini terletak pada mekanisme kolaborasinya, yang dapat dirancang berdasarkan berbagai dimensi strategi, seperti interaksi (P2P atau terdistribusi) dan protokol koordinasi spesifik [21]. Melalui interaksi antarkelompok agen ini, sistem dapat melakukan proses verifikasi silang terhadap informasi yang dikumpulkan. Secara komputasional, distribusi proses evaluasi dan agregasi kepada banyak agen ini sangat efektif dalam meredam variansi keluaran (*output variance*) dan memitigasi anomali halusinasi probablistik yang sering terjadi pada tingkat agen individu. Hal inilah yang menjadikannya fondasi arsitektural ideal untuk membangun sebuah sistem *oracle* terdesentralisasi yang tahan terhadap ancaman agen jahat (*Byzantine*).

2.2.2 *Blockchain dan Smart Contract*

Blockchain merupakan *distributed ledger technology* (DLT) yang transaksi dan perubahan statusnya direplikasi serta disimpan secara permanen di banyak *node* dalam jaringan [22]. Secara fundamental, teknologi ini memecahkan masalah kepercayaan (*trust*) dalam sebuah sistem terdistribusi, yaitu bagaimana menciptakan penyimpanan dokumen *timestamped* tanpa ada satu pihak pun yang dapat memanipulasi data tanpa terdeteksi [23]. Berbeda dengan DLT tradisional yang mengharuskan pengguna untuk mempercayai satu pihak, *blockchain* menyediakan mekanisme kepercayaan terdistribusi yang memungkinkan setiap partisipan memverifikasi bahwa urutan dan waktu transaksi tidak dirusak [23].

Secara arsitektural, *blockchain* mengelompokkan set data transaksi ke dalam blok-blok yang saling dirantai menggunakan fungsi *hash* kriptografi [22, 24]. Fungsi *hash* bertindak secara matematis untuk mengambil sembarang *string* masukan dan menghasilkan nilai keluaran dengan panjang karakter tetap [23]. Setiap blok baru secara inheren menyimpan nilai *hash* dari blok sebelumnya, membentuk sebuah rantai yang tidak terputus dan saling terkait. Jika ada entitas yang mencoba memanipulasi data transaksi suatu

blok, nilai *hash* dari blok tersebut akan otomatis berubah karena hampir mustahil secara komputasi untuk menemukan data yang mampu menghasilkan keluaran *hash* yang sama seperti data yang ingin dimanipulasi. Perubahan *hash* ini akan membuat seluruh blok setelahnya menjadi tidak valid sehingga jaringan dapat seketika menolak upaya manipulasi tersebut. Detail mengenai struktur penautan *hash* dari satu blok ke blok yang lainnya dapat dilihat pada Gambar 2.2.



Gambar 2.2. Arsitektur penautan blok pada jaringan *blockchain*. Gambar merupakan hasil modifikasi dari Mohanta et al. [24]

Lebih lanjut, blok-blok baru ini divalidasi dan ditambahkan ke *ledger* melalui algoritma konsensus, seperti *Proof of Work* (PoW), *Proof of Stake* (PoS), atau *Practical Byzantine Fault Tolerane* (PBFT) [22, 24]. Mekanisme konsensus ini memastikan integritas jaringan tetap terjaga secara transparan dan *immutable*.

Di atas fondasi infrastruktur yang *immutable* inilah, konsep *smart contract* diimplementasikan untuk menambah lapisan programabilitas pada *blockchain*. Konsep *smart contract* pertama kali diusulkan oleh Nick Szabo pada tahun 1994 yang mendefinisikannya sebagai protokol transaksi terkomputerisasi yang mengeksekusi persyaratan dari sebuah kontrak [24]. Dalam ekosistem *blockchain*, kehadiran *smart contract* menambah lapisan programabilitas yang secara fundamental mengubah jaringan dari sekadar buku besar pencatat transaksi menjadi sebuah *platform* komputasi terdesentralisasi.

Secara arsitektural, *smart contract* adalah sekumpulan program komputer yang memiliki sifat dapat memverifikasi diri sendiri (*self-executing*) dan *tamper resistant* [24]. Program ini beroperasi secara *event-driven*, yang berarti *smart contract* menerima transaksi masuk sebagai *input*, mengeksekusi kode yang sesuai, dan kemudian memicu keluaran (*output*) secara deterministik [24]. Sebagai contoh, sebuah janji atau kondisi dapat disimpan di dalam *blockchain* dalam bentuk kode *smart contract* yang secara otomatis mengeksekusi transaksi tersebut apabila kondisi yang disepakati telah terpenuhi [23].

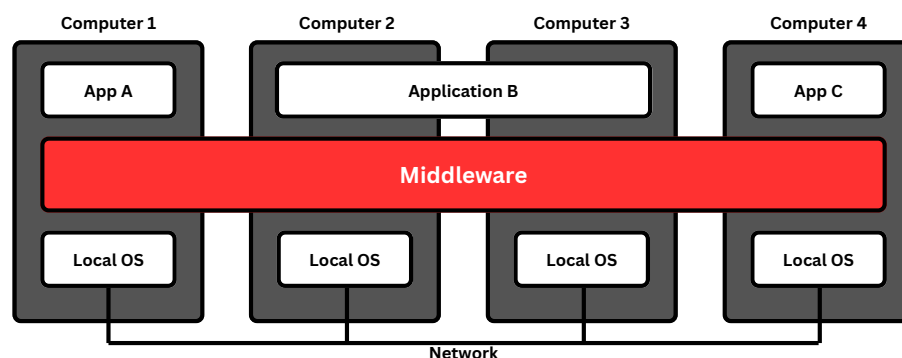
Meskipun *smart contract* menawarkan keamanan komputasi, teknologi ini memiliki keterbatasan yang sangat kritis. Lingkungan *smart contract* dan *blockchain* dirancang menyerupai sebuah sistem tertutup (*enclosed system*) yang tidak memiliki kapabilitas atau akses terhadap informasi di dunia luar (*off-chain*) secara langsung [22]. Kebutuhan *smart contract* akan kemampuan untuk menghubungkan data dari dunia nyata

ke dalam *blockchain* ini berbenturan dengan arsitektur yang terisolasi tersebut, memunculkan sebuah celah operasional yang dikenal luas sebagai masalah *oracle* (*the oracle problem*) [22]. Untuk memitigasi keterbatasan ini, *blockchain oracle* (atau *data feeds*) diperkenalkan sebagai entitas terpercaya yang berfungsi menjembatani dan memverifikasi informasi eksternal agar dapat masuk ke dalam *blockchain* [22]. Oleh karena itu, kebutuhan akan sebuah layanan *oracle* yang aman, desentralisasi, dan bebas dari agen jahat menjadi justifikasi logis dalam penelitian ini.

2.2.3 Sistem Terdistribusi dan Toleransi Kesalahan *Byzantine*

Dalam konteks arsitektur multi-agen *Large Language Model* (LLM) yang dirancang pada penelitian ini, sangat penting untuk memahami lingkungan komputasi dasar tempat agen-agen tersebut beroperasi dan berkoordinasi. Sebelum membedah metrik efisiensi komunikasi maupun toleransi terhadap agen yang bertindak menyimpang, fondasi teoretis mengenai sistem terdistribusi harus didefinisikan terlebih dahulu.

Secara formal, sistem terdistribusi didefinisikan sebagai kumpulan elemen komputasi otonom yang berkolaborasi sedemikian rupa sehingga tampak oleh penggunanya sebagai satu kesatuan sistem yang koheren [25]. Definisi ini menyoroti dua karakteristik fundamental. Pertama, sistem ini berisikan sekumpulan *node* yang sepenuhnya independen, dengan setiap *node* dapat beroperasi secara otonom dan memiliki pemahaman waktunya sendiri akibat ketiadaan jam rujukan global (*global clock*) [25]. Kedua, terlepas dari pemisahan otonomi fisik tersebut, sistem terdistribusi dirancang untuk menyajikan transparansi distribusi *distribution transparency*. Transparansi ini bertujuan untuk menyembunyikan kompleksitas dan penyebaran proses maupun sumber daya sehingga sistem beroperasi secara terpadu tanpa mengharuskan pengguna mengetahui lokasi sebenarnya dari eksekusi sebuah komputasi [25]. Untuk menjembatani kesenjangan antara perangkat keras yang beraneka ragam dengan aplikasi, sistem terdistribusi umumnya mengimplementasikan lapisan perangkat lunak tambahan yang berada di atas sistem operasi lokal, yang dikenal sebagai *middleware* [25].

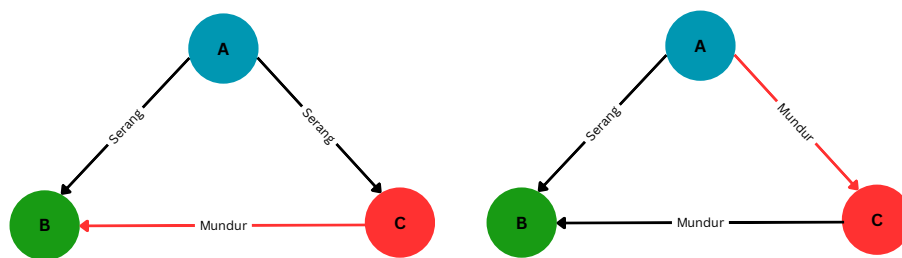


Gambar 2.3. Organisasi lapisan *middleware* pada sistem terdistribusi. Gambar merupakan hasil modifikasi dari [25].

Karakteristik dari sistem terdistribusi inilah yang menjadi landasan operasional utama bagi arsitektur *oracle* yang diusulkan. Dalam penelitian ini, agen-agen LLM direpresentasikan sebagai *node-node* otonom tersebut. Pemodelan jaringan multi-agen sebagai sebuah sistem terdistribusi menegaskan bahwa meskipun setiap agen beroperasi secara mandiri dan terisolasi saat mengevaluasi data *off-chain*, mereka dituntut untuk berkoordinasi secara koheren melalui lapisan *middleware* guna menghasilkan satu luaran nilai yang disepakati bersama. Ilustrasi konseptual mengenai bagaimana lapisan *middleware* ini diposisikan dapat dilihat pada Gambar 2.3. Gambar 2.3 merupakan modifikasi dari [25].

Untuk mencapai kesatuan yang koheren tersebut, sistem terdistribusi mengandalkan algoritma konsensus. Tujuan utama dari algoritma konsensus adalah memungkinkan *node-node* di dalam sistem terdistribusi untuk mencapai sebuah kesepakatan (*agreement*) [26]. Secara formal, algoritma konsensus terdistribusi bertujuan untuk memastikan semua proses yang sehat mencapai kesepakatan dalam jumlah langkah yang terbatas dengan memenuhi tiga syarat utama: seluruh proses harus menghasilkan nilai keluaran yang sama, nilai keluaran tersebut mutlak harus valid, dan setiap proses pada akhirnya harus memberikan luaran [25].

Meskipun demikian, upaya untuk mencapai konsensus antar *node* otonom memunculkan tantangan fundamental terkait penanganan kegagalan (*failure handling*). Ancaman yang paling parah dan menjadi fokus krusial dalam perancangan mesin konsensus adalah kegagalan arbitrer, yang secara teoritis pertama kali diperkenalkan oleh Leslie Lamport [27, 28] pada tahun 1982 melalui *Byzantine Generals Problem*. Masalah ini menganalogikan sekelompok jenderal yang harus mengoordinasikan rencana serangan, namun komunikasi mereka rentan disabotase oleh pengkhianat di antara mereka yang dapat memberikan informasi palsu secara sengaja [26].



Gambar 2.4. Kegagalan konsensus pada permasalahan tiga jenderal *Byzantine*. (Kiri) Skenario ketika Wakil C adalah pengkhianat; (Kanan) Skenario ketika Panglima A adalah pengkhianat. Gambar merupakan hasil modifikasi dari [27].

Sebagai ilustrasi, pertimbangkan skenario tiga jenderal *Byzantine*, Panglima A beserta dua Wakilnya, B dan C, yang harus mencapai konsensus mengenai keputusan menyerang atau mundur, seperti yang ditunjukkan pada Gambar 2.4. Dalam protokol klasik, komunikasi dibagi menjadi dua tahap [27]: (1) Panglima mengirimkan perintah

kepada seluruh Wakil; (2) setiap Wakil meneruskan perintah yang diterima kepada Wakil lainnya guna verifikasi silang atas kejujuran Panglima.

Pada skenario pertama (Gambar 2.4, kiri), Wakil C adalah pengkhianat. Panglima A mengirim “Serang” ke B dan C. Wakil B jujur meneruskan “Serang” ke C, namun C melakukan *equivocation*, melaporkan ke B bahwa A memerintahkan “Mundur”. Wakil B kini menerima pesan bertentangan (“Serang” dari A, “Mundur” dari C) dengan rasio 1 : 1. Karena B tidak dapat menentukan mana di antara A dan C yang berkhianat, konsensus gagal [26].

Pada skenario kedua (Gambar 2.4, kanan), Panglima A adalah pengkhianat. A melakukan *equivocation*: mengirim “Serang” ke B namun “Mundur” ke C. Kedua Wakil yang jujur meneruskan informasi apa adanya, sehingga B menerima laporan C yang mengklaim “Mundur” sementara C menerima laporan B yang mengklaim “Serang”. Tanpa cara mengidentifikasi siapa pengkhianatnya, kedua Wakil mengalami kebuntuan dan konsensus kembali gagal [26].

Kedua skenario mendemonstrasikan bahwa dengan satu pengkhianat di antara tiga partisipan, konsensus tidak tercapai, menjadi landasan teoretis bahwa kegagalan *Byzantine* terpicu ketika komponen yang rusak atau jahat memberikan informasi bertentangan ke berbagai bagian sistem.

Dalam konteks jaringan *oracle* multi-agen berbasis LLM, kegagalan *Byzantine* memiliki makna yang lebih luas. Agen tidak selalu harus dikendalikan secara aktif oleh penyerang (*malicious actors*) untuk dapat memicu anomali ini. Mengingat sifat probabilitas yang melekat pada LLM, agen yang secara tidak sengaja memberikan evaluasi yang salah atau merespons secara tidak konsisten ini secara komputasional bertindak layaknya jenderal pengkhianat, karena mereka menyumbangkan data yang korup dan berpotensi merusak kesepakatan jaringan.

Untuk melindungi integritas sistem dari anomali semacam ini, jaringan membutuhkan mekanisme *Byzantine Fault Tolerance* (BFT), ketahanan jaringan sangat bergantung pada sebuah batasan matematis yang biasa disebut *Byzantine threshold*. Sebuah jaringan secara teoretis membutuhkan setidaknya $3f + 1$ *node* untuk dapat menoleransi f anggota yang mengalami kegagalan arbitrer [25, 26].

2.2.4 Kompleksitas Algoritma dan Notasi Asimtotik

Dalam merancang dan mengevaluasi sistem multi-agen terdistribusi, memprediksi sumber daya yang dibutuhkan oleh protokol jaringan merupakan tahapan analitis yang esensial. Menganalisis sebuah algoritma berarti memprediksi sumber daya komputasional yang dibutuhkannya, meskipun sering kali yang diukur adalah waktu komputasi, sumber daya lain seperti memori, konsumsi energi, maupun *bandwidth* komunikasi ja-

ringan juga sama pentingnya untuk dianalisis dan dibatasi [29]. Untuk mengkarakterisasi efisiensi algoritma dan membandingkannya dengan pendekatan alternatif seiring dengan bertambahnya ukuran jaringan, kita mempelajari efisiensi asimtotiknya [29].

Pendekatan asimtotik berfokus pada bagaimana kebutuhan sumber daya dari suatu algoritma meningkat pada batasnya, yakni saat ukuran input bertambah tanpa batas [29]. Dalam melakukan analisis ini, kita menggunakan abstraksi penyederhanaan di mana laju pertumbuhan (*rate of growth*) adalah metrik yang paling difokuskan [29]. Dalam evaluasi matematisnya, kita membuang suku-suku berorde lebih rendah dan mengabaikan koefisien konstanta pada suku utama, karena untuk ukuran input yang cukup besar, faktor konstanta dan suku berorde rendah menjadi kalah signifikan dibandingkan dengan efek pertumbuhan dari ukuran input itu sendiri [29].

Salah satu notasi asimtotik yang paling umum digunakan untuk menyatakan kompleksitas ini adalah *Big O-notation*. *O-notation* mengkarakterisasi batas atas asimtotik (*asymptotic upper bound*) dari perilaku suatu fungsi [29]. Secara formal, notasi ini menyatakan bahwa suatu fungsi tumbuh tidak lebih cepat dari laju tertentu yang didasarkan secara eksklusif pada suku dengan orde tertingginya [29]. Selain *O-notation*, literatur juga mendefinisikan Notasi Θ (*Theta-notation*) yang memberikan batas ketat, yaitu fungsi tumbuh secara persis pada laju tertentu hingga selisih faktor konstanta dari atas dan bawah [29]. Namun, dalam mengevaluasi batas terburuk skalabilitas jaringan, *O-notation* lebih sering dijadikan metrik acuan.

Dalam konteks arsitektur multi-agen, notasi asimtotik digunakan secara khusus untuk memodelkan kompleksitas komunikasi. Pada protokol BFT tradisional yang bersifat egaliter, proses pencapaian konsensus sering kali mengharuskan setiap agen (atau *node*) untuk menyiarkan pesannya secara berulang ke seluruh node lainnya (*all-to-all broadcast*). Jika jaringan terdiri dari N agen, maka jumlah pesan yang dipertukarkan tumbuh secara kuadratik seiring pertambahan partisipan yang direpresentasikan dalam notasi asimtotik sebagai $O(N^2)$. Laju pertumbuhan kuadratik ini sangat membatasi dan menciptakan hambatan skalabilitas yang masif pada jaringan terdesentralisasi.

2.2.5 Agregasi Spasial dan *Robust Statistics*

Dalam menganalisis sekumpulan data, ilmu statistika menggunakan ukuran tendensi sentral untuk memformulasikan pusat dari lokasi data tersebut. Salah satu ukuran fundamental yang paling penting adalah sampel *median* (nilai tengah). Secara matematis, apabila terdapat sekumpulan observasi yang diurutkan berdasarkan besaran, *median* adalah nilai yang berada tepat di tengah [30]. Berbeda dengan perhitungan rata-rata (*sample mean*) konvensional yang nilainya sangat mudah bergeser akibat kehadiran observasi ekstrem, *median* secara teoretis dirancang untuk mencerminkan pusat data yang sesungguhnya sedemikian rupa sehingga nilainya kebal dari pengaruh perusak nilai-nilai ekstrem

atau *outliers* [30]. Sifat *median* yang mampu memitigasi anomali data ini menjadikannya fondasi utama dalam disiplin statistik yang tangguh (*robust statistics*).

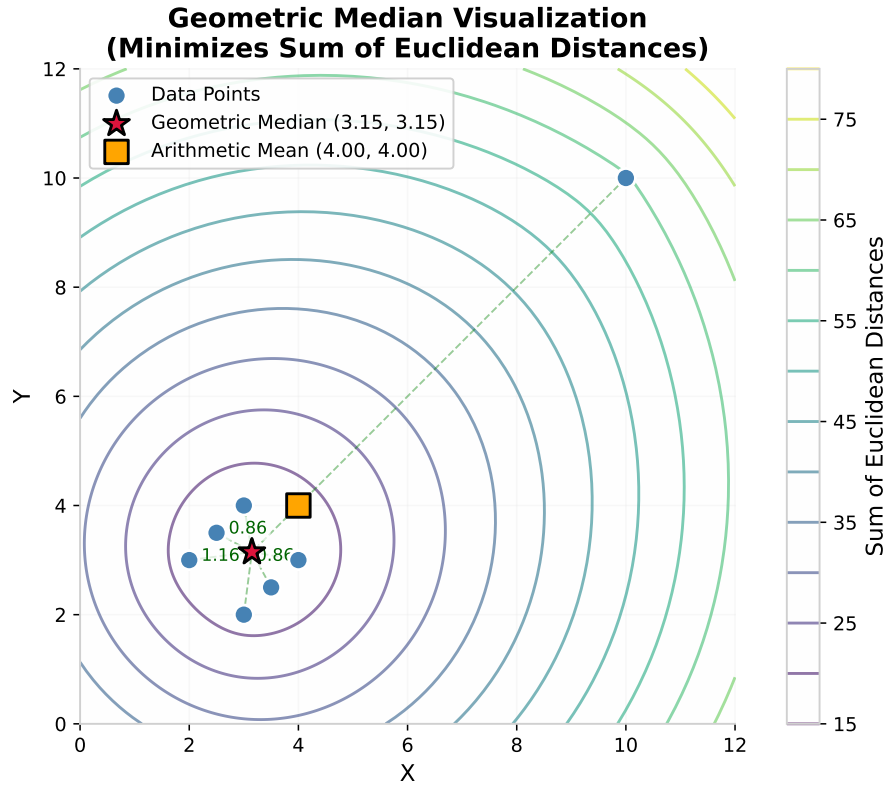
Konsep ketangguhan matematis dari *median* inilah yang diadopsi ke dalam arsitektur multi-agen. Setelah proses evaluasi dilakukan oleh kelompok agen secara paralel, sistem membutuhkan sebuah mekanisme untuk menyatukan berbagai hasil tersebut menjadi satu keputusan akhir yang disepakati. Namun, agen berbasis LLM umumnya tidak sekadar menghasilkan pilihan diskret, melainkan memberikan evaluasi dalam format skor atau parameter vektor multidimensional. Mengingat evaluasi berada dalam ruang vektor spasial, metrik *median* satu dimensi biasa tidak lagi mencukupi. Oleh karena itu, diperlukan perluasan teknik agregasi spasial untuk menemukan titik konsensus yang tangguh dan kebal terhadap manuver nilai ekstrem dari agen jahat.

Dalam literatur *Machine Learning*, beberapa agregasi spasial telah dirancang untuk menoleransi anomali ini. Salah satu aturan agregasi *Byzantine-resilient* yang dikenal luas adalah fungsi *Krum* yang bekerja dengan mengevaluasi setiap vektor dan memilih vektor yang meminimalkan jumlah kuadrat jarak (*sum of squared distances*) ke $n - f - 2$ tetangga terdekatnya [31]. Meskipun secara komputasi efisien, arsitektur toleransi tinggi pada sistem sering kali lebih mengandalkan *Geometric Median* (GM). *Geometric Median* adalah sebuah metode estimasi lokasi yang mencari satu titik spasial target dengan cara meminimalkan jumlah jarak Euclidean ke seluruh titik vektor masukan [32]. Visualisasi GM bisa dilihat pada Gambar 2.5. Bisa dilihat, jika dibandingkan dengan *mean*, GM lebih robust karena ia tidak tertarik terlalu jauh ke arah *outlier*.

Secara geometris, metrik jarak *Euclidean* mengukur besaran atau jarak garis lurus terpendek antara dua titik dalam ruang multidimensional. Jarak ini dihitung menggunakan *Euclidean norm* (*L2 norm*) yang secara matematis disimbolkan dengan notasi $\|\cdot\|_2$ [33]. Secara spesifik, *L2 norm* didefinisikan sebagai akar kuadrat dari jumlah kuadrat koordinat sebuah vektor di dalam ruang koordinat. Notasi ini mengukur besaran metrik dari sebuah vektor sehingga jarak *Euclidean* antara dua titik vektor x_i dan x_j secara formal diekspresikan sebagai $\|x_i - x_j\|_2$ [33]. Oleh karena itu, jika terdapat himpunan vektor $A_1, A_2, \dots, A_N$, maka titik target *Geometric Median* y didefinisikan secara matematis sebagai [32]:

$$y = \arg \min_z \sum_{i=1}^N \|z - A_i\|_2 \quad (2-1)$$

Dibandingkan dengan algoritma agregasi *Byzantine-robust* lainnya seperti *Krum* yang hanya memiliki batas toleransi $f \leq \lfloor \frac{N-2}{2} \rfloor$ atau *Bulyan* sebesar $f \leq \lfloor \frac{N-3}{4} \rfloor$, GM menawarkan tingkat ketahanan yang lebih tinggi [11]. Secara teoretis, GM tetap dapat secara andal menyeleksi jawaban terbaik selama jumlah agen jahat tidak melampaui batas



Gambar 2.5. Visualisasi *Geometric Median*

toleransi absolute $f \leq \left\lfloor \frac{N-1}{2} \right\rfloor$ [11].

2.2.6 *Trust, Reputation, dan Probabilistic Selection*

Dalam lingkungan sistem multi-agen yang terdesentralisasi dan terbuka, agen-agen yang saling berinteraksi sering kali tidak memiliki kendali otoritas pusat sehingga kemampuan untuk menilai keandalan agen lain menjadi tantangan yang sangat esensial. Kepercayaan (*trust*) secara fundamental didefinisikan sebagai ekspektasi atau keyakinan subjektif bahwa suatu pihak akan bertindak secara kooperatif dan tidak membahayakan pihak yang memercayainya [34]. Dalam literatur sistem terdistribusi, konsep *trust* secara umum dibangun melalui dua sudut pandang utama, yaitu sudut pandang individual dan sudut pandang komunal. Sudut pandang individual dibentuk murni berdasarkan pengalaman langsung agen tersebut di masa lalu, sementara sudut pandang komunal secara luas dikenal dengan istilah reputasi (*reputation*) [34]. Secara lebih spesifik, reputasi adalah keyakinan terhadap kemampuan, kejujuran, dan keandalan sebuah komponen yang dibangun berdasarkan agregasi observasi atau rekomendasi dari komunitas secara keseluruhan [35]. Melalui pemanfaatan metrik reputasi, sebuah jaringan tanpa pemimpin (*leaderless*) dapat merekam jejak historis secara kontinu yang berfungsi sebagai landasan rasional untuk membedakan antara entitas yang andal dan agen yang berperilaku menyimpang.

Dalam disiplin ilmu statistika, melacak fluktuasi data pada lingkungan yang sangat dinamis membutuhkan mekanisme pembaruan yang mampu menyeimbangkan antara data historis dan observasi terkini. Salah satu metode *smoothing* yang paling diandalkan untuk tujuan ini adalah *Exponentially Weighted Moving Average* (EWMA). Dalam bidang ilmu komputer, machine learning, dan blockchain, teori tersebut biasa disebut dengan *Exponential Moving Average* (EMA). Kata "weighted" dihilangkan karena dalam komputasi setiap pergerakan yang eksponensial sudah pasti melibatkan pembobotan. Secara teoretis, EMA adalah model agregasi rata-rata bergerak yang membiarkan bobot observasi masa lalu menurun atau memudar secara eksponensial dalam bentuk deret geometri [36]. Secara formal, persamaan statistik umum untuk EMA didefinisikan sebagai:

$$Z_i = \lambda Y_i + (1 - \lambda)Z_{i-1} \quad (2-2)$$

Pada Persamaan 2-2, Z_i adalah nilai rata-rata bergerak pada saat ini, Y_i merepresentasikan nilai observasi aktual terbaru, dan parameter $\lambda \in [0, 1)$ bertindak sebagai koefisien *smoothing* atau pembobotan [36]. Persamaan matematis ini secara alami mengimplementasikan *forgetting mechanism*, sehingga semakin lama umur sebuah observasi, semakin kecil pengaruhnya terhadap nilai agregasi masa depan.

Sifat *smoothing* historis dari EMA inilah yang kemudian diadopsi secara konseptual ke dalam sistem multi-agen terdesentralisasi untuk memodelkan pembaruan metrik *trust/reputation* [12]. Mengingat reputasi tidak dapat direpresentasikan sebagai nilai yang statis, mekanisme EMA digunakan untuk memastikan bahwa metrik tersebut senantiasa merefleksikan keandalan agen saat ini berdasarkan interaksi terbaru [12]. Dengan mengadaptasi pendekatan ini, sebuah sistem memberikan kesempatan bagi agen untuk secara perlahan pulih dari kesalahan masa lalu, namun pada saat yang bersamaan tetap menjatuhkan penalti terhadap agen yang secara konsisten menunjukkan perilaku buruk [12]. Dalam konteks jaringan *oracle* berbasis LLM, adopsi konsep ini menjamin bahwa fluktuasi eror sesekali akibat halusinasi probabilistik dapat dimanfaatkan secara bertahap, sementara anomali dari aktor *Byzantine* akan mengakibatkan penurunan reputasi yang terukur secara sistematis.

Setelah metrik reputasi dari suatu komponen berhasil dievaluasi dan terus diperbarui, sistem terdistribusi membutuhkan sebuah mekanisme untuk memanfaatkan nilai tersebut dalam proses alokasi tugas atau pemilihan. Jika sistem hanya menggunakan aturan deterministik murni yang selalu memilih agen dengan nilai reputasi tertinggi, sistem tersebut akan kehilangan kemampuannya untuk mengeksplorasi agen-agen baru di dalam jaringan. Untuk menjembatani kebutuhan tersebut, literatur pemodelan probabilistik secara luas menggunakan fungsi *Softmax*. Secara matematis, fungsi *softmax* adalah sebuah pemetaan yang mengambil suatu vektor input berupa skor numerik yang tidak di-

batasi (*non-normalized scores*), lalu mentransformasikannya menjadi sebuah vektor probabilitas yang total seluruh probabilitasnya selalu berjumlah satu [37]. Persamaan fungsi *softmax* secara formal didefinisikan sebagai:

$$p_i = \frac{\exp(\alpha s_i)}{\sum_j \exp(\alpha s_j)} \quad (2-3)$$

Dengan p_i merepresentasikan probabilitas keluaran untuk opsi ke- i , s_i adalah skor input awal dari opsi tersebut, dan parameter α merupakan parameter optimalitas yang memodulasi bentuk dari distribusi probabilitas keluaran dari tingkat acak hingga optimalisasi maksimum [37]. Dalam aplikasinya sebagai model pilihan stokastik, formulasi *softmax* memastikan bahwa agen dengan skor metrik yang lebih tinggi akan mendapatkan probabilitas untuk terpilih secara proporsional lebih besar, namun pada saat yang sama tetap memberikan peluang bagi komponen berperingkat rendah untuk terpilih.

2.3 Analisis Perbandingan Metode

Dalam merancang sistem multi-agen berbasis *Large Language Model* (LLM) yang tangguh terhadap serangan *Byzantine*, berbagai arsitektur konsensus telah diajukan dalam literatur terkini. Untuk menjustifikasi urgensi dan keunggulan dari arsitektur yang diusulkan pada penelitian ini, diperlukan analisis komparatif baik secara kualitatif maupun kuantitatif terhadap metode-metode *State-of-the-Art* (SOTA) pendahulu.

Secara kualitatif, evaluasi keandalan sistem dapat ditinjau dari tingkat ketergantungan terhadap pemimpin (*leader-dependency*) dan paradigma pembobotan suara konsensus. Metode generasi awal seperti [13] dan [17] secara fundamental masih sangat bergantung pada arsitektur berbasis pemimpin (*leader-based*). Analisis menunjukkan bahwa apabila agen yang secara deterministik ditunjuk sebagai pemimpin adalah *Byzantine*, seluruh siklus konsensus akan menemui jalan buntu sehingga memaksa sistem untuk melakukan pemilihan pemimpin ulang yang memicu lonjakan latensi [11].

Untuk menghindari ketergantungan pada *single node*, arsitektur DecentLLMs memperkenalkan jaringan tanpa pemimpin (*leaderless*) yang menggunakan agregasi spasial *Geometric Median* (GM) [11]. Meskipun pendekatan ini secara efektif melenyapkan ancaman manipulasi pada satu titik, *DecentLLMs* beroperasi di atas paradigma pengumutan suara yang sepenuhnya egaliter, yang mengasumsikan semua agen evaluator memiliki derajat keandalan yang sama rata [11]. Ketiadaan pembedaan kualitas suara ini memunculkan kerentanan kualitatif yang dikenal sebagai *evaluator reliability problem*. Maksud dari hal tersebut adalah variansi penilaian yang tinggi dari agen evaluator jujur yang sedang berhalusinasi dapat dengan mudah menarik titik tengah konsensus menjauh dari jawaban yang akurat [11].

Dari sudut pandang kuantitatif komputasional, batasan terbesar dari penerapan ar-

sitektur *leaderless* egaliter seperti *DecentLLMs* adalah tingginya kompleksitas komunikasi jaringan. Karena meniadakan peran koordinator pusat, jaringan desentralisasi mengharuskan seluruh agen evaluator, dengan ukuran himpunan N , untuk menyiarkan skor evaluasi mereka secara menyeluruh ke seluruh evaluator lainnya (*all-to-all broadcast*) [11]. Proses ini secara matematis menghasilkan kompleksitas komunikasi asimtotik jaringan yang cukup berat, yakni sebesar $O(N^2)$. Seiring dengan bertambahnya *node* agen di dalam jaringan, biaya pengiriman pesan akan meningkat eksponensial yang menyebabkan sistem melambat drastis.

Sebagai justifikasi perbaikan komputasional, kerangka kerja yang diusulkan membatasi proses pengiriman matriks agregasi hanya pada partisi komite evaluator berukuran konstan K . Dengan demikian, metode yang diusulkan secara kuantitatif berhasil memangkas kompleksitas pertukaran data dari $O(N^2)$ menjadi $O(K^2)$.

Berdasarkan pada sintesis keterbatasan arsitektur pendahulu, penelitian ini mengusulkan kerangka kerja yang berkontribusi untuk memperbaiki keterbatasan tersebut. Sistem yang diusulkan memitigasi agen jahat dan variansi evaluator melalui agregasi *Weighted Geometric Median* (WGM) yang memperhitungkan rekam jejak reputasi agen dalam proses konsensus. Rekam jejak reputasi tersebut diatur melalui *Exponential Moving Average* (EMA). Selain itu, sistem mencoba untuk memperbaiki limitasi komunikasi jaringan melalui komponen *Sharded Selection* berbasis distribusi probabilitas *Softmax*.

Seluruh hasil analisis komparatif dan sistensis keunggulan dari arsitektur yang diusulkan dibandingkan dengan metode-metode SOTA pendahulu dirangkum pada Tabel 2.2.

Tabel 2.2. Perbandingan arsitektur multi-agen terdahulu

Dimensi	BlockAgents [13]	MultiLLM [17]	DecentLLMs [11]	Ours
Arsitektur	<i>Leader</i>	<i>Leader</i>	<i>Leaderless</i>	<i>Leaderless</i>
<i>Weighted Voting</i>	×	✓	×	✓
<i>Byzantine-Robust</i>	✓	✓	✓	✓
<i>Sharded Selection</i>	×	~	×	✓
Meritokratik	×	~	×	✓
Konsensus	PoT	WBFT	GM	WGM
Komunikasi	$O(N^2)$	$O(K)$	$O(N^2)$	$O(K^2)$
Validasi	Benchmark	Simulasi	Benchmark	Keduanya

Simbol: ✓ = Dukungan penuh; ~ = Parsial; × = Tidak didukung.

Singkatan: PoT = *Proof-of-Thought*; WBFT = *Weighted Byzantine Fault Tolerance*; GM = *Geometric Median*; WGM = *Weighted Geometric Median*

BAB III

METODE PENELITIAN

Bab ini menjelaskan tentang metodologi dan model matematis yang diimplementasikan untuk menjawab rumusan masalah serta mencapai tujuan penelitian yang telah ditetapkan pada sub-bab 1.3.

3.1 Alat dan Bahan Tugas akhir

Bagian ini menjelaskan perangkat keras, perangkat lunak, serta bahan-bahan yang digunakan dalam pelaksanaan penelitian. Pembahasan dibagi menjadi dua subbab, yaitu alat tugas akhir pada Subbab 3.1.1 dan bahan tugas akhir pada Subbab 3.1.2.

3.1.1 Alat Tugas akhir

Alat-alat yang digunakan dalam tugas akhir ini meliputi perangkat keras dan perangkat lunak sebagai sarana pendukung penelitian. Berikut rinciannya:

1. *Laptop* dengan spesifikasi sistem operasi Fedora Linux 43, *processor* Intel Core i7-12700H, memori 16 GB DDR4. Perangkat ini digunakan untuk pengembangan perangkat lunak, penulisan kode, serta analisis data.
2. *Virtual Private Server* (VPS) dengan spesifikasi sistem operasi Ubuntu 22.04, 2 *vCPU*, memori 4 GB, dan penyimpanan 60 GB. VPS ini digunakan sebagai lingkungan eksekusi untuk menjalankan eksperimen simulasi secara konsisten dan terisolasi.
3. *Python* versi ≥ 3.12 , < 3.14 sebagai bahasa pemrograman utama, dengan manajemen paket menggunakan `uv` versi 0.9.26. Beberapa *library* Python yang digunakan antara lain `numpy` untuk komputasi numerik, `langchain` sebagai *wrapper* untuk mengakses LLM melalui API, `httpx` untuk komunikasi HTTP, dan `web3` untuk interaksi dengan jaringan *blockchain*.
4. Layanan *Large Language Model* (LLM) melalui OpenRouter yang diakses menggunakan *wrapper* LangChain.
5. Anvil Foundry versi 1.5.1-stable (termasuk `cast`, `forge`, dan `anvil`) untuk pengembangan dan pengujian *smart contract* secara lokal.
6. *Text editor* Zed untuk penulisan dan pengeditan kode.
7. Zotero untuk manajemen referensi dan pengutipan.
8. Canva dan draw.io untuk pembuatan diagram arsitektur sistem.

3.1.2 Bahan Tugas akhir

Dalam penelitian ini, bahan yang digunakan berfokus pada aset digital, yaitu kumpulan data pengujian (*dataset*), serta literatur yang menjadi landasan dari simulasi sistem multi-agen. Rincian bahan tugas akhir yang digunakan adalah:

1. **Dataset Evaluasi:** Untuk menguji keandalan, tingkat halusinasi, dan akurasi konsensus dari arsitektur jaringan *oracle*, penelitian ini memanfaatkan gabungan dari dua *dataset benchmark* yang bersifat *open-source*, yaitu:
 - **TruthfulQA** [15]: Digunakan sebagai bahan pengujian utama untuk mengukur sejauh mana agen LLM rentan terhadap miskonsepsi umum dan seberapa faktual jawaban yang dihasilkan.
 - **ARC-Easy (AI2 Reasoning Challenge)** [16]: Digunakan untuk mengevaluasi kemampuan penalaran ilmiah dan logika agen dalam menjawab pertanyaan-pertanyaan yang membutuhkan analisis bertahap.

Kedua *dataset* ini dikombinasikan dan diacak menggunakan *seeded random number generator* untuk memastikan reproduktibilitas selama eksperimen simulasi berjalan.

2. **Bahan Komputasi dan Agen (*Digital Consumables*):** Penelitian ini menggunakan kredit kuota API (*API credits*) sebagai bahan habis pakai digital. Eksperimen ini mengonsumsi token API melalui platform *OpenRouter* untuk menginisialisasi *pool* agen yang heterogen. Bahan komputasi terdiri dari 30 model LLM berbea yang bersumber dari 14 *provider* berbeda.
3. **Dokumen Panduan dan Standar Referensi:** Penelitian ini juga bersandar pada beberapa dokumen yang bertindak sebagai standar referensi matematis dan arsitektural, yaitu:
 - **Protokol Dasar *Baseline*:** Model arsitektur DecentLLMs [11] digunakan sebagai dokumen standar (kontrol) untuk membandingkan performa sistem pendahulu terhadap sistem yang diusulkan.
 - **Formulasi Matematis:** Dokumen referensi terkait *Exponential Moving Average* (EMA) dan *Weighted Geometric Median* (WGM) digunakan sebagai standar dalam menyusun logika pembaruan metrik kepercayaan dan agregasi spasial [12,17].

3.2 Metode yang Digunakan

Bagian ini membahas tentang kerangka metodologi yang diaplikasikan untuk merealisasikan dan menguji arsitektur yang diusulkan. Pendekatan utama yang digunakan dalam penelitian ini berpusat pada eksperimen simulasi dan *software prototyping*. Secara berurutan, bagian ini akan mendefinisikan rancangan lingkungan eksperimen yang digunakan, menjabarkan tahapan penerapan algoritma matematis di dalam sistem, serta

menetapkan parameter komparasi data untuk membuktikan tingkat skalabilitas dan ketahanan sistem usulan terhadap arsitektur *baseline*.

3.2.1 Desain Penelitian

Penelitian ini menggunakan pendekatan eksperimental kuantitatif yang dieksekusi melalui metode **Simulasi dan Prototyping Perangkat Lunak**. Dalam disiplin ilmu rekayasa perangkat lunak, pengembangan purwarupa (*prototype*) merupakan teknik yang diakui secara luas untuk mendemonstrasikan konsep, menguji opsi desain, serta mengevaluasi kelayakan fungsional dari sebuah solusi arsitektural secara cepat dan terukur [38]. Pemilihan desain penelitian ini juga divalidasi oleh metodologi standar yang digunakan dalam literatur sistem multi-agen LLM terdesentralisasi terbaru, sebab pengujian arsitektur konsensus dan *smart contract* lazim dilakukan menggunakan *prototype* di dalam lingkungan simulasi.

Untuk membuktikan hipotesis dan mengukur metrik performa dari arsitektur yang diusulkan, sebuah purwarupa jaringan multi-agen dibangun dan dieksekusi di dalam lingkungan simulasi. Lingkungan eksperimen ini memodelkan interaksi jaringan *peer-to-peer* (P2P) secara terdesentralisasi, namun secara fisik dieksekusi di dalam sebuah laboratorium visual. Pendekatan simulasi ini dipilih karena memungkinkan isolasi variabel pengujian secara ketat sehingga dampak dari serangan dan manipulasi skor oleh agen jahat terhadap mekanisme agregasi dapat diukur secara presisi tanpa adanya interferensi latensi dari anomali jaringan fisik eksternal.

Secara teknis, laboratorium virtual yang menopang simulasi ini berada di atas sebuah *Virtual Private Server* (VPS) seperti yang sudah dijelaskan pada bagian 3.1.1. Purwarupa perangkat lunak dikembangkan menggunakan bahasa pemrograman Python [39] dengan memanfaatkan kerangka kerja LangChain [40] untuk mengorkestrasi alur kerja agen secara konkuren. Sebagai representasi agen di dalam jaringan, sebanyak 30 model LLM yang berbeda diakses secara terprogram melalui OpenRouter [41]. Untuk merealisasikan lapisan *blockchain*, sistem menggunakan *smart contract* yang dideploy pada jaringan lokal Anvil testnet melalui Foundry [42]. *Smart contract* ini secara deterministik mengunci pembaruan skor reputasi agen sebagai *ledger* yang tidak dapat diubah (*immutable*). Guna menjamin validitas simulasi terhadap kondisi jaringan yang serealistis mungkin, sebuah sistem *semaphore* diimplementasikan untuk memberikan batasan konkurensi lalu lintas data sehingga sistem dapat mereplikasi antrean komputasi selayaknya pada jaringan desentralisasi nyata.

3.2.2 Pemodelan Sistem dan Aktor Jaringan

Arsitektur yang diusulkan beroperasi di dalam lingkungan jaringan *peer-to-peer* (P2P) terdesentralisasi yang menerapkan topologi tanpa pemimpin (*leaderless*). Sistem

ini dimodelkan ke dalam dua lapisan utama: lapisan agen *off-chain* yang bertugas memproses komputasi LLM dan lapisan *on-chain* yang bertindak sebagai *immutable ledger* untuk menyimpan skor reputasi agen. Dalam lingkungan ini, populasi jaringan didistribusikan ke dalam beberapa peran fungsional dan batasan ancaman sebagai berikut:

1. **Agen Pekerja (*Worker Agents, W*)**: Himpunan agen yang bertugas menerima *prompt* dari pengguna dan merumuskan kandidat jawaban, dinotasikan sebagai $W = \{w_1, w_2, \dots, w_{N_w}\}$. Jawaban yang dihasilkan oleh agen pekerja ke- j dinotasikan sebagai a_j . Agen-agen pada lapisan ini memproses teks secara paralel dan independen. Hal ini memungkinkan sistem memproses kandidat jawaban tanpa adanya *bottleneck* antrean agen-agen.
2. **Agen Evaluator (*Evaluator Agents, E*)**: Himpunan agen yang bertindak sebagai penilai (*jury*), yang bertanggung jawab mengevaluasi dan memberikan skor pada vektor 5-dimensi terhadap kandidat jawaban a_j dari agen pekerja. Himpunan ini dinotasikan sebagai $E = \{e_1, e_2, \dots, e_{N_e}\}$. Untuk mengatasi masalah skalabilitas komunikasi dari kompleksitas $O(N^2)$ menjadi $O(K^2)$, sistem tidak menugaskan seluruh populasi N_e untuk mengevaluasi, melainkan mendelegasikannya kepada subset komite berukuran K yang diseleksi secara probabilistik.
3. **Model Ancaman (*Byzantine Threat Model*)**: Setiap agen dalam jaringan diasumsikan dapat bersifat jujur (*Honest*) atau jahat (*Byzantine*). Model sistem ini mendefinisikan dua bentuk serangan yang dapat dilakukan oleh agen jahat:
 - ***Byzantine Worker***: Agen yang secara sengaja menghasilkan jawaban yang berhalusinasi, menyisipkan data tidak relevan, atau melancarkan *backdoor attacks* dengan memasukkan frasa pemicu tersembunyi.
 - ***Byzantine Evaluator***: Agen yang melakukan manipulasi nilai spasial untuk merusak konsensus. Hal ini mencakup *collusion attack* (memberikan skor sempurna pada jawaban *Byzantine worker* dan skor 0 pada agen jujur) serta *oscillation attacks* (sesekali berperilaku jujur untuk mendapatkan kepercayaan dan menaikkan skor reputasi sebelum meluncurkan serangan).

Dalam arsitektur *leaderless baseline* [11], batasan keamanan sistem (*Byzantine Fault Tolerance*) umumnya diukur menggunakan jumlah populasi agen atau *headcount*, sistem hanya aman jika jumlah agen jahat $f \leq \lfloor \frac{n-1}{2} \rfloor$. Namun, karena sistem ini dirancang untuk menyelesaikan masalah keandalan evaluator dengan mengubah mekanisme konsensus menjadi meritokrasi berbasis reputasi, batasan *headcount* konvensional menjadi tidak relevan. Di dalam sistem ini, ancaman tidak lagi ditentukan oleh seberapa banyak jumlah agen jahat yang ada, melainkan oleh seberapa besar akumulasi pengaruh spasial yang mereka miliki [17]. Sebuah jaringan bisa saja memiliki jumlah agen jahat yang lebih banyak secara kuantitas, namun jika bobot

reputasi mereka lemah, konsensus tetap aman.

Oleh karena itu, pemodelan ancaman pada sistem ini secara khusus bertumpu pada **Fraksi Bobot Jujur** (*Honest Weight Fraction*, Λ). Sistem diasumsikan aman dan konsensus spasial tidak akan dapat dibajak selama rasio total bobot reputasi agen jujur terhadap total bobot komite memenuhi kondisi mayoritas:

$$\Lambda = \frac{\sum w_{\text{honest}}}{\sum w_{\text{total}}} \geq 0.5 \quad (3-1)$$

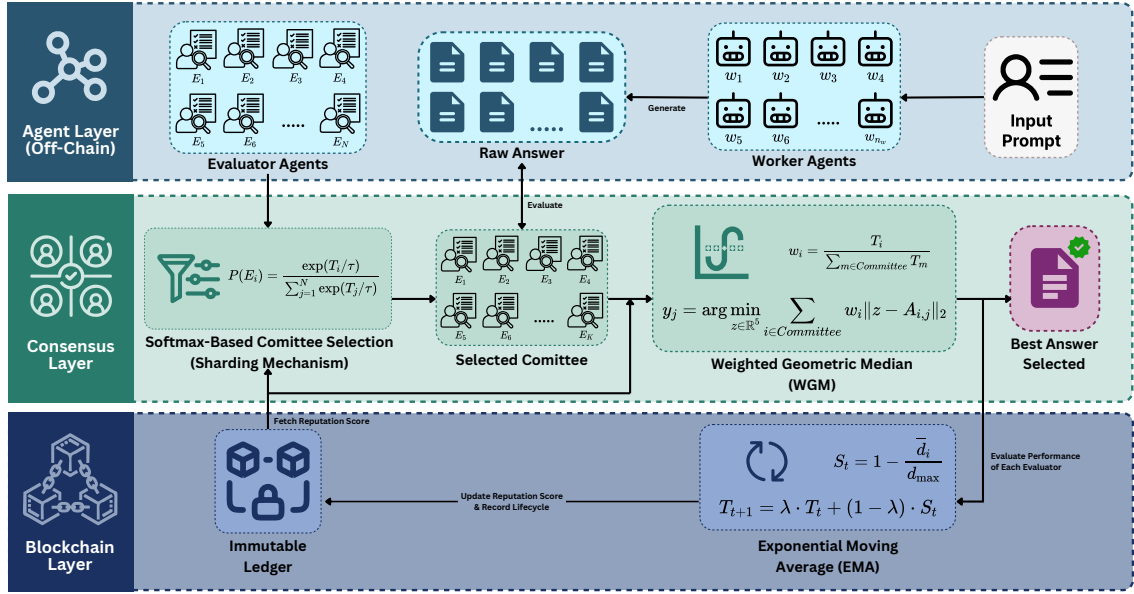
dengan $\sum w_{\text{honest}}$ adalah akumulasi bobot pengaruh dari evaluator yang jujur di dalam komite dan $\sum w_{\text{total}}$ adalah total keseluruhan bobot komite. Parameter bobot/weight (w) dalam formulasi ini merepresentasikan tingkat pengaruh spasial agen, yang mekanisme perhitungannya akan dijabarkan secara rinci pada tahapan agregasi *Weighted Geometric Median* (WGM) di Subbab 3.2.4.3. Selama arsitektur sistem mampu mempertahankan nilai Λ di atas batas 0.5, pengaruh dari manipulasi agen jahat akan terbukti secara matematis dapat dinetralisasi [17].

4. **Siklus Waktu Diskrit (Ronde Konsensus):** Model sistem ini beroperasi berdasarkan siklus waktu diskrit yang direpresentasikan dengan variabel waktu t . Satu ronde konsensus didefinisikan sebagai satu siklus utuh yang dimulai dari masuknya *prompt*, paralelisme *worker*, seleksi komite, agregasi spasial, hingga diakhirinya siklus melalui pembaruan skor reputasi (*post-round settlement*). Formulasi waktu diskrit ini bertindak sebagai formulasi matematis sistem umpan balik, sehingga performa pada ronde t akan secara langsung memengaruhi probabilitas dan bobot agen pada ronde $t + 1$.

3.2.3 Arsitektur yang Diusulkan

Berdasarkan pemodelan sistem dan aktor jaringan yang telah dijabarkan pada 3.2.2, bagian ini menyajikan rancangan arsitektur *Reputation-Weighted Consensus* secara komprehensif. Arsitektur ini dirancang sebagai jaringan *oracle* terdesentralisasi yang menyelesaikan dua permasalahan fundamental pada protokol *leaderless baseline*, yakni kerentanan terhadap manipulasi evaluator *Byzantine* dan hambatan skalabilitas komunikasi, melalui sintesis tiga lapisan operasional yang saling bergantung. Secara visual, arsitektur keseluruhan sistem diilustrasikan pada Gambar 3.6.

Seperti yang ditunjukkan pada Gambar 3.6, arsitektur dibagi menjadi tiga lapisan yang merepresentasikan pemisahan tanggung jawab (*separation of concerns*) secara ketat: lapisan agen (*off-chain*) yang menangani komputasi LLM, lapisan konsensus yang bertindak sebagai *middleware* penengah, serta lapisan *blockchain* (*on-chain*) yang berfungsi sebagai *immutable ledger*. Pemisahan ini dirancang agar tidak ada *single point of failure*. Berikut adalah penjelasan rinci dari masing-masing lapisan.



Gambar 3.6. Arsitektur tiga lapisan *Reputation-Weighted Consensus*: *Agent Layer*, *Consensus Layer*, dan *Blockchain Layer*.

3.2.3.1 *Agent Layer (Off-Chain)*

Lapisan ini merupakan lingkungan komputasi tempat entitas LLM beroperasi. Dalam arsitektur yang diusulkan, populasi agen diisolasi secara fungsional ke dalam dua peran yang secara ketat terpisah, sebagaimana telah dimodelkan pada Subbab 3.2.2. Pemisahan peran ini bersifat fundamental: tidak ada satu agen pun yang merangkap fungsi pekerja dan evaluator secara simultan. Pemisahan ini mencegah terjadinya konflik kepentingan yang memungkinkan agen memanipulasi proses evaluasi demi keuntungan jawabannya sendiri, sehingga secara arsitektural mengeliminasi kemungkinan *single point of failure* akibat kompromi peran ganda.

1. **Agen Pekerja (*Worker Agents*, W):** Himpunan agen yang bertugas menerima *prompt* dari pengguna dan menghasilkan kandidat jawaban secara paralel. Setiap agen pekerja $w_j \in W$ memproses teks secara independen tanpa adanya koordinasi atau komunikasi antar-agen pekerja. Arsitektur paralelisme ini memaksimalkan *throughput* sistem karena kandidat jawaban dihasilkan secara konkuren, menghindari *bottleneck* antrean pemrosesan sekuensial. Dengan demikian, waktu produksi kandidat jawaban ditentukan oleh latensi inferensi LLM terlama (*slowest inference*), bukan oleh akumulasi latensi seluruh agen.
2. **Agen Evaluator (*Evaluator Agents*, E):** Himpunan agen yang membentuk *pool* penilai yang siap untuk mengekstraksi dan menilai kualitas jawaban dari para pekerja. Setiap evaluator $e_i \in E$ bertanggung jawab memetakan kandidat jawaban ke dalam ruang vektor 5-dimensi sebagaimana didefinisikan pada Subbab 3.2.4.1. Penting untuk ditegaskan bahwa himpunan E ini merepresentasikan *seluruh populasi* eva-

luator yang tersedia di jaringan, bukan himpunan yang secara aktif mengevaluasi pada setiap ronde. Mekanisme seleksi agen evaluator yang benar-benar berpartisipasi didelegasikan kepada lapisan konsensus, bukan ditentukan oleh lapisan agen itu sendiri.

3.2.3.2 *Consensus Layer (Middleware)*

Lapisan ini merupakan kontribusi utama dari arsitektur yang diusulkan. *Consensus Layer* bertindak sebagai *middleware*/penengah antara lapisan agen yang menghasilkan dan menilai jawaban, dengan lapisan *blockchain* yang menyimpan *state* reputasi secara permanen. Tanpa lapisan ini, jaringan tidak memiliki mekanisme untuk menyeleksi evaluator, mengagregasi skor, maupun menghubungkan hasil evaluasi dengan pembaruan reputasi. Lapisan konsensus mengandung dua mesin komputasional utama yang bekerja secara berurutan dalam setiap ronde:

1. ***Sharding Engine (Modul 1 — Seleksi Komite Probabilistik)***: Modul ini memecahkan hambatan skalabilitas komunikasi yang merupakan kelemahan fundamental arsitektur *leaderless baseline*. Alih-alih mengharuskan seluruh populasi N_e evaluator untuk berpartisipasi dalam setiap ronde, mesin *sharding* secara dinamis menyaring populasi tersebut menjadi sebuah Komite Juri berukuran tetap K menggunakan probabilitas berbasis fungsi *Softmax*. Melalui mekanisme *weighted random sampling without replacement* yang dirumuskan pada Persamaan 3-2, agen-agen ditarik secara probabilistik berdasarkan skor reputasi historis mereka, sehingga menyeimbangkan antara eksploitasi agen bereputasi tinggi dan eksplorasi agen baru. Dengan membatasi partisipasi hanya pada komite K , mesin ini secara efektif memangkas kompleksitas komunikasi dari $O(N_e^2)$ menjadi $O(K^2)$, sebuah reduksi yang bersifat konstan terlepas dari pertumbuhan populasi jaringan, sebagaimana akan dibuktikan pada Subbab 3.2.4.2.
2. ***Consensus Engine (Modul 2 — Agregasi Spasial WGM)***: Setelah komite memberikan vektor evaluasinya, modul ini mengagregasi seluruh skor tersebut menggunakan algoritma *Weighted Geometric Median (WGM)* sebagaimana dirumuskan pada Persamaan 3-4. Agregasi ini bersifat meritokratis (suara setiap evaluator tidak diperlakukan setara, melainkan dibobotkan berdasarkan skor reputasi historis mereka sesuai Persamaan 3-3). Mekanisme pembobotan ini memastikan bahwa evaluator dengan rekam jejak evaluasi yang konsisten dan akurat memiliki tarikan gravitasi spasial yang lebih kuat terhadap titik konsensus final, sementara vektor evaluasi anomali dari agen jahat secara matematis akan tersisih karena bobot pengaruh mereka yang lemah. Dengan demikian, modul konsensus ini secara inheren menetralisasi manipulasi spasial dan menghasilkan jawaban yang paling akurat dan andal.

3.2.3.3 *Blockchain Layer (On-Chain)*

Lapisan terakhir berfungsi sebagai *immutable ledger* yang menjamin transparansi dan integritas data kepercayaan sistem. Jika lapisan agen menyediakan kapasitas komputasi dan lapisan konsensus menyediakan logika pengambilan keputusan, maka lapisan *blockchain* menyediakan *ingatan* kolektif yang tidak dapat dimanipulasi. Arsitektur ini memanfaatkan *smart contract* yang dideploy pada jaringan lokal untuk merealisasikan dua fungsionalitas utama:

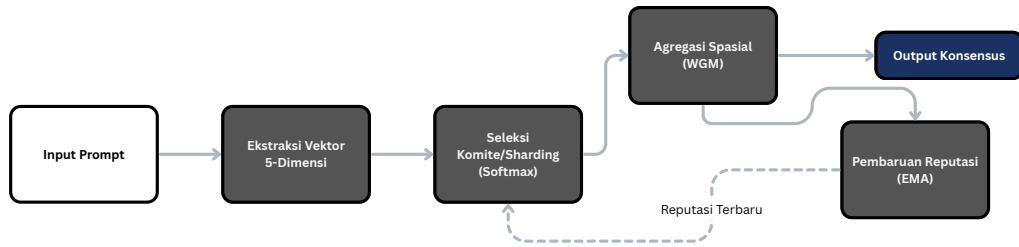
1. ***Reputation Store***: *Smart contract* menyimpan skor reputasi historis (T_i) dari setiap agen yang tercatat di jaringan secara permanen dan tidak dapat diubah (*tamper-proof*). Penyimpanan ini bersifat *source of truth* tunggal, seluruh modul *off-chain* (*sharding* dan konsensus) membaca skor reputasi secara eksklusif dari lapisan ini. Arsitektur *write-through cache* memastikan bahwa setiap penulisan pembaruan reputasi secara atomik disinkronkan ke *on-chain* sebelum *cache* lokal diperbarui, sehingga tidak terjadi inkonsistensi data antara *state* off-chain dan on-chain.
2. ***After Round Settlement***: Setelah konsensus tercapai dan titik pusat spasial final ditetapkan oleh modul WGM, performa setiap evaluator di dalam komite dinilai secara retrospektif. Lapisan ini kemudian mengeksekusi fungsi *Exponential Moving Average* (EMA) sebagaimana dirumuskan pada Persamaan 3-7 untuk memperbarui skor reputasi agen secara dinamis sebagai persiapan untuk ronde berikutnya. Pembaruan ini bersifat *feedback loop*: evaluasi yang akurat pada ronde t meningkatkan reputasi agen untuk ronde $t + 1$, yang pada gilirannya meningkatkan probabilitas terpilihnya agen tersebut ke dalam komite dan memperkuat bobot pengaruhnya dalam agregasi WGM. Sebaliknya, manipulasi evaluator *Byzantine* akan menghasilkan *spatial error* yang besar, yang kemudian menurunkan reputasi mereka secara progresif dan bertahap mengurangi kemampuan agen tersebut untuk memengaruhi konsensus di masa depan.

Ketiga lapisan di atas membentuk sebuah siklus tertutup yang bersifat adaptif dan *self-correcting*. Aliran data dimulai dari *Agent Layer* yang menghasilkan kandidat jawaban dan evaluasi, kemudian mengalir melalui *Consensus Layer* yang menyeleksi komite dan mengagregasi skor secara meritokratis, dan berakhir di *Blockchain Layer* yang merekam hasil dan memperbarui reputasi. Skor reputasi terbaru ini kemudian mengalir kembali (*feedback*) ke kedua lapisan sebelumnya, mengendalikan probabilitas seleksi komite pada modul *sharding* dan bobot pengaruh pada modul konsensus, menciptakan mekanisme umpan balik positif yang secara dinamis memperkuat kontribusi agen jujur dan mendegradasi pengaruh agen jahat sepanjang siklus hidup jaringan. Detail algoritmik dari setiap tahapan dalam arsitektur ini akan dijabarkan secara komprehensif pada Subbab 3.2.4.

3.2.4 Tahapan Penerapan Metode Sistem

Subbab ini menguraikan alur kerja algoritmik dari arsitektur yang diusulkan secara komprehensif. Arsitektur yang diusulkan selanjutnya akan disebut sebagai *Reputation-Weighted Consensus*. Untuk mencegah konsensus oleh agen jahat sekaligus memecahkan hambatan skalabilitas komunikasi, proses pengambilan keputusan dalam jaringan multi-agen ini dieksekusi melalui 4 tahapan yang saling berhubungan.

Secara visual, aliran data dan logika komputasi pada arsitektur ini diilustrasikan pada Gambar 3.7. Alur dimulai dari *input prompt* yang direspons oleh agen *worker*, lalu teks tersebut diekstraksi ke dalam bentuk matriks kuantitatif melalui tahapan 3.2.4.1. Untuk meringankan beban komputasi jaringan, sistem kemudian menyeleksi sebuah komite *evaluator* secara probabilistik menggunakan mekanisme 3.2.4.2. Vektor evaluasi yang dihasilkan oleh komite ini selanjutnya diagregasi menggunakan metode 3.2.4.3 untuk menemukan *output* konsensus. Terakhir, metrik performa setiap *evaluator* dihitung ulang dan skor reputasi jaringan diperbarui secara dinamis melalui 3.2.4.4



Gambar 3.7. Diagram blok alur komputasi arsitektur *Reputation-Weighted Consensus*

Seperti yang ditunjukkan pada *feedback loop* pada Gambar 3.7, skor reputasi terbaru yang dihasilkan pada tahap akhir tidak diabaikan, melainkan disimpan kembali sebagai parameter krusial untuk ronde berikutnya. Reputasi historis ini bertindak sebagai mekanisme adaptif yang mengendalikan probabilitas *Softmax* dan bobot pengaruh (*weight*) agen pada agregasi WGM.

3.2.4.1 Evaluasi Ruang Vektor 5-Dimensi

Tahap pertama dalam pencapaian konsensus terjadi setelah setiap agen pekerja secara paralel menghasilkan kandidat jawaban a_j . Untuk menangkap kompleksitas bahasa alami dan mengkuantifikasi tingkat halusinasi LLM, arsitektur ini tidak menggunakan mekanisme pemungutan suara biner (benar atau salah). Sebaliknya, setiap agen evaluator $e_i \in E$ melakukan ekstraksi kualitas setiap jawaban a_j ke dalam bentuk matriks kuantitatif.

Kelima dimensi evaluasi ($C = 5$) tersebut secara fundamental didasarkan pada taksonomi halusinasi generatif standar untuk LLM [43], yang secara spesifik dirancang untuk mengidentifikasi berbagai metrik catatan, meliputi:

1. ***Factual Contradiction (Kontradiksi Faktual)***: Termasuk dalam kategori halusinasi faktualitas, ini mengukur sejauh mana jawaban secara langsung bertentangan dengan fakta-fakta yang umum di dunia nyata [43].
2. ***Factual Fabrication (Fabrikasi Faktual)***: Mengukur sejauh mana agen mengarang atau merekayasa informasi yang tidak eksis secara faktual [43].
3. ***Instruction Inconsistency (Inkonsistensi Instruksi)***: Termasuk dalam kategori halusinasi *faithfulness*, ini mengukur tingkat penyimpangan jawaban dari instruksi, atau kegagalan agen dalam merespons *prompt* [43].
4. ***Context Inconsistency (Inkonsistensi Konteks)***: Mengukur tingkat deviasi jawaban dari konteks atau premis awal pertanyaan yang diberikan kepada sistem [43].
5. ***Logical Inconsistency (Inkonsistensi Logis)***: Mengukur tingkat kesalahan argumentasi atau kecatatan dalam tahapan penalaran logis, sehingga kesimpulan yang ditarik bertentangan dengan langkah-langkah sebelumnya [43].

Setiap kriteria di atas dinilai pada skala kontinu dari 0 hingga 20 [11]. Pada skala ini, skor yang mendekati 20 mengindikasikan kualitas jawaban yang sangat tinggi (bebas dari halusinasi pada dimensi tersebut), skor yang mendekati 0 menunjukkan kualitas yang sangat buruk, dan skor 10 merepresentasikan nilai netral [11].

Representasi ekstraksi kelima metrik tersebut ke dalam koordinat vektor spasial $A_{i,j} = (v^1, v^2, v^3, v^4, v^5) \in \mathbb{R}^5$ diadaptasi dari [11]. Berdasarkan model ancaman di subbab sebelumnya, agen jahat evaluator dapat meluncurkan serangan *collusion* dengan memanipulasi pemberian skor. Melalui representasi vektor 5-dimensi ini, kebohongan dan manipulasi evaluasi tersebut akan termanifestasi secara matematis sebagai anomali jarak geometris yang ekstrem, yang pada tahap selanjutnya akan dideteksi dan dinetralisasi oleh algoritma agregasi spasial.

3.2.4.2 Seleksi Komite Probabilistik (*Softmax Sharding*)

Kelemahan fundamental dari arsitektur *leaderless* adalah adanya hambatan skalabilitas komunikasi yang masif. Dalam protokol *baseline* tersebut, ketiadaan agen pemimpin mengharuskan setiap evaluator untuk menghitung titik agregasi (dalam *baseline* adalah *Geometric Median*) secara lokal. Sebagai prasyarat perhitungan lokal ini, setiap agen evaluator diwajibkan untuk menyiarkan (*broadcast*) matriks skor 5-dimensi miliknya kepada seluruh agen evaluator lainnya di dalam jaringan. Secara matematis, jika terdapat populasi sebanyak N_e agen evaluator, maka setiap agen harus mentransmisikan pesannya ke $N_e - 1$ agen lain. Secara kolektif, mekanisme *all-to-all* ini memaksa jaringan untuk mengeksekusi pertukaran pesan sebanyak $N_e \times (N_e - 1)$ atau setara dengan $N_e^2 - N_e$ pesan dalam setiap ronde. Oleh karena itu, kompleksitas komunikasi untuk arsitektur *baseline* adalah $O(N_e^2)$ atau bisa kita sederhanakan menjadi $O(N^2)$. Seiring

dengan bertambahnya partisipasi agen dalam jaringan, pertukaran pesan ini akan menjadi *bottleneck* yang akan membuat latensi sistem menjadi tinggi.

Untuk memecahkan masalah ini, arsitektur yang diusulkan mengimplementasikan mekanisme *Softmax Sharding*. Alih-alih mengharuskan seluruh populasi N_e untuk mengevaluasi, sistem mendelegasikan tugas evaluasi tersebut kepada sebuah subset komite juri berukuran tetap K . Dengan membatasi proses agregasi spasial hanya pada komite ini, sistem berhasil memangkas kompleksitas komunikasi dari $O(N_e^2)$ menjadi $O(K^2)$.

Meskipun demikian, proses penyeleksian komite ini menghadirkan *trade-off* antara keamanan jaringan dan *fairness*. Jika sistem menggunakan metode seleksi deterministik, yakni selalu memilih K agen dengan reputasi tertinggi, sistem akan menciptakan sebuah komite yang tertutup. Dalam skenario tersebut, agen-agen baru yang bergabung ke dalam jaringan dengan reputasi awal *default* (Misalnya $T = 0.5$) tidak akan pernah memiliki kesempatan untuk berpartisipasi, menghasilkan kerentanan yang dikenal sebagai *cold start problem*.

Untuk mengeliminasi celah tersebut, pemilihan komite dilakukan secara probabilistik dengan mengadaptasi distribusi konsep *Softmax* pada [12]. Namun, rumusan standar tersebut dimodifikasi pada penelitian ini dengan mengintegrasikan parameter *temperature* (τ) berdasarkan sifat fundamental manipulasi distribusi *Softmax* dalam teori keputusan [37]. Probabilitas terpilihnya agen evaluator e_i untuk masuk ke dalam komite juri dihitung menggunakan rumusan berikut:

$$P(e_i) = \frac{\exp(T_i/\tau)}{\sum_{j=1}^{N_e} \exp(T_j/\tau)} \quad (3-2)$$

Dalam persamaan di atas, T_i merepresentasikan skor reputasi historis dari evaluator ke- i , N_e adalah total populasi evaluator, dan τ (*temperature*) adalah hiperparameter skalar pengatur probabilitas.

Penting untuk ditegaskan bahwa probabilitas yang dihasilkan oleh fungsi *Softmax* ini tidak digunakan untuk secara deterministik mengurutkan dan memilih K agen dengan nilai tertinggi. Pendekatan semacam itu hanya akan mereplikasi kelemahan seleksi *Top-K* konvensional. Sebaliknya, nilai probabilitas $P(e_i)$ ini diimplementasikan sebagai bobot distribusi untuk melakukan mekanisme pengambilan sampel acak berbobot tanpa pengembalian (*weighted random sampling without replacement*). Melalui mekanisme undian probabilistik ini, sistem akan secara acak menarik K agen dari populasi N_e .

Penggunaan hiperparameter τ berfungsi sebagai titik penyeimbang matematis antara eksploitasi dan eksplorasi:

1. Nilai τ yang rendah akan memperkuat perbedaan antar skor reputasi sehingga sistem secara agresif mengkesplorasi agen-agen yang memiliki reputasi tinggi untuk

memaksimalkan ketahanan *Byzantine*, namun membatasi keragaman komite.

2. Nilai τ yang tinggi akan memperhalus distribusi probabilitas, memberikan ruang eksplorasi jaringan yang lebih merata.

Melalui fungsi eksponensial ini, agen dengan reputasi rendah tetap mempertahankan nilai probabilitas matematis yang lebih besar dari nol. Mekanisme ini menjamin terciptanya lingkungan meritokratis yang adaptif, yang memungkinkan agen baru atau agen yang sedang memulihkan reputasinya tetap memiliki ruang probabilistik untuk ditarik ke dalam komite dan kembali membangun skor reputasi mereka.

3.2.4.3 Agregasi Spasial *Weighted Geometric Median* (WGM)

Setelah komite berukuran K terbentuk dan setiap evaluator $e_i \in \text{Committee}$ menghasilkan vektor evaluasi 5-dimensi $A_{i,j}$ terhadap kandidat jawaban pekerja a_j , sistem harus mengagregasi vektor-vektor tersebut untuk menemukan satu nilai konsensus final.

Protokol *leaderless baseline* menggunakan algoritma *Geometric Median* (GM) biasa yang memperlakukan semua evaluator setara [11]. Namun, pendekatan ini rentan terhadap agen jujur yang memiliki variansi tinggi (*evaluator reliability problem*) maupun serangan kolusi *Byzantine*. Untuk menetralisasi kerentanan tersebut, arsitektur ini memodifikasi proses agregasi menjadi *Weighted Geometric Median* (WGM).

Sebelum agregasi spasial dilakukan, pengaruh dari masing-masing evaluator di dalam komite dikalibrasi menggunakan bobot relatif (w_i). Bobot ini dihitung dengan menormalisasi skor reputasi historis T_i milik agen tersebut terhadap total reputasi seluruh anggota komite:

$$w_i = \frac{T_i}{\sum_{m \in \text{Committee}} T_m} \quad (3-3)$$

Normalisasi ini menjamin bahwa $\sum w_i = 1$. Untuk setiap jawaban pekerja a_j , titik konsensus spasial y_j dicari dengan menemukan koordinat dalam ruang $\mathbb{R}^5$ yang meminimalkan total jarak *Euclidean* berbobot terhadap seluruh vektor evaluasi komite:

$$y_j = \arg \min_{z \in \mathbb{R}^5} \sum_{i \in \text{Committee}} w_i \|z - A_{i,j}\|_2 \quad (3-4)$$

Mengacu pada sifat *breakdown point* toleransi *Byzantine* dalam sistem terdistribusi [44] serta sifat estimasi statistik dari nilai tengah geometris [32], berbeda dengan rata-rata berbobot (*weighted mean*) yang sangat mudah ditarik oleh nilai ekstrem, objektif fungsi WGM terbukti secara matematis mampu mengabaikan anomali asalkan akumulasi tarikan spasial dari *Honest Weight Fraction* (Λ) mempertahankan kondisi mayoritas

$\Lambda \geq 0.5$.

Karena persamaan WGM di atas tidak memiliki solusi aljabar tertutup (*closed-form solution*), komputasi konsensus dieksekusi melalui aproksimasi numerik menggunakan adaptasi algoritma Weiszfeld (*Iteratively Reweighted Least Squares*) [45]. Mengacu pada kajian modern metode Weiszfeld serta pembatasan galat numerik pada estimasi *geometric median* [32, 45], algoritma ini dirancang dengan penanganan singularitas dan aturan penghentian iterasi (*stopping rule*) yang menjamin konvergensi. Misalkan $y_{j,k}$ adalah estimasi titik konsensus untuk jawaban a_j pada iterasi ke- k . Algoritma aproksimasi numerik ini berjalan melalui tahapan berikut:

1. **Inisialisasi:** Untuk mempercepat konvergensi, estimasi awal $y_{j,0}$ ditetapkan menggunakan nilai median standar dari matriks vektor evaluasi komite.
2. **Pembaruan Iteratif:** Posisi estimasi diperbarui secara iteratif menggunakan rumusan *fixed-point* Weiszfeld yang telah disesuaikan dengan bobot reputasi [45]:

$$y_{j,k+1} = \frac{\sum_{i=1}^K \frac{w_i \cdot A_i}{\|A_{i,j} - y_{j,k}\|_2}}{\sum_{i=1}^K \frac{w_i}{\|A_{i,j} - y_{j,k}\|_2}} \quad (3-5)$$

3. **Penanganan Singularitas *Singularity Handling*:** Algoritma Weiszfeld memiliki kelemahan matematis bawaan sehingga fungsi menjadi tidak terdefinisi (pembagian dengan nol) jika estimasi $y_{j,k}$ jatuh tepat di atas salah satu titik data jangkar atau *anchor point* ($A_{i,j}$) [45]. Untuk mencegah hal ini, dilakukan pengecekan kondisi $\|A_{i,j} - y_{j,k}\|_2 < \epsilon$. Jika terpenuhi, titik estimasi langsung dikunci (*snapped*) pada koordinat $A_{i,j}$ dan iterasi dihentikan.
4. **Konvergensi:** Mengikuti prinsip optimasi nilai tengah yang stabil [32], iterasi komputasi terus berjalan hingga jarak perpindahan spasial antar-iterasi jauh di bawah ambang batas toleransi ($\epsilon = 10^{-5}$) atau hingga mencapai batas keamanan maksimum 1000 iterasi.

Setelah titik konsensus y_j berhasil ditemukan, sistem menjumlahkan kelima dimensi dari vektor tersebut untuk menghasilkan sebuah skor keandalan skalar/*robust score* (berkisar antara 0 hingga 100) bagi jawaban a_j . Proses WGM ini diulangi untuk setiap kandidat jawaban pekerja dan sistem secara deterministik akan memilih jawaban dengan *robust score* tertinggi sebagai *output* final yang dikembalikan kepada pengguna. Jika terjadi hasil seri (*tie*), sistem akan mengeksekusi fungsi *tie-breaker* kriptografis dengan melakukan *hashing* terhadap teks jawaban beserta *blockhash* terbaru.

3.2.4.4 Pembaruan Reputasi Dinamis berbasis *Exponential Moving Average* (EMA)

Siklus akhir dalam setiap ronde konsensus adalah mengevaluasi kinerja agen komite itu sendiri. Dalam arsitektur *leaderless baseline*, setiap ronde berdiri sendiri sehingga agen jahat tidak pernah menerima implikasi atas manipulasi yang mereka lakukan (*stateless*). Untuk mengatasi kelemahan ini, arsitektur yang diusulkan mengimplementasikan mekanisme pembaruan reputasi dinamis menggunakan fungsi *Exponential Moving Average* (EMA) [12].

Setelah algoritma WGM menetapkan titik pusat konsensus final y_j , sistem secara retrospektif menghitung tingkat *spatial error* dari setiap evaluator e_i di dalam komite. *Spatial error* ini diukur melalui rata-rata jarak Euclidean ($\bar{d}_i$) antara vektor evaluasi asli mereka ($A_{i,j}$) terhadap titik konsensus final (y_j) untuk seluruh kandidat jawaban pekerja yang dievaluasi pada ronde tersebut.

Kinerja ronde saat ini, yang dinotasikan sebagai S_t , kemudian diekstraksi dari *spatial error* tersebut menggunakan fungsi *inverse min-max normalization* [46]. Rumusan ekstraksi kinerjanya didefinisikan sebagai:

$$S_t = 1 - \frac{\bar{d}_i}{d_{max}} \quad (3-6)$$

Dalam persamaan 3-6, d_{max} merepresentasikan jarak maksimum absolut yang mungkin terjadi dalam ruang metrik evaluasi. Karena setiap dimensi kriteria ($C = 5$) beroperasi pada skala 0 hingga 20, jarak Euclidean terburuk adalah $d_{max} = \sqrt{5 \times 20^2} \approx 44.72$. Melalui normalisasi ini (penurunan matematis dapat dilihat pada Lampiran L.3), agen jujur yang menghasilkan vektor dengan deviasi minimal ($\bar{d}_i \rightarrow 0$) akan menerima skor kinerja S_t yang mendekati 1, sementara agen jahat yang memberikan skor ekstrem akan menerima skor S_t yang mendekati 0.

Skor kinerja ini kemudian diintegrasikan ke dalam skor reputasi historis permanen agen (T_t) melalui aturan:

$$T_{t+1} = \lambda \cdot T_t + (1 - \lambda) \cdot S_t \quad (3-7)$$

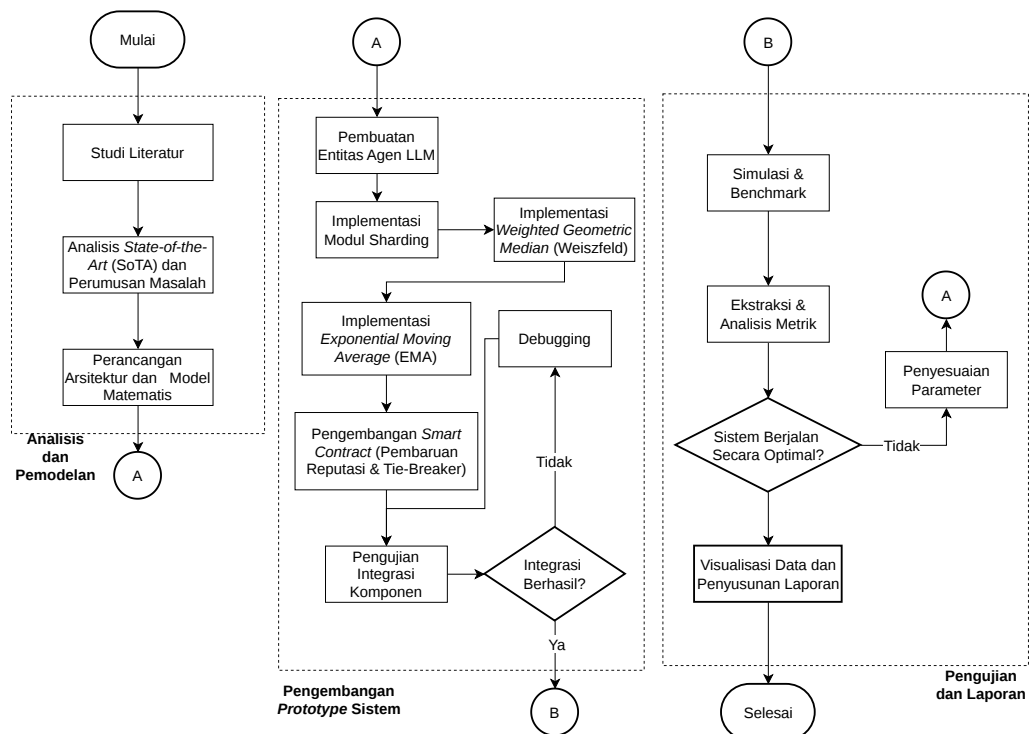
Dalam persamaan 3-7, parameter $\lambda \in [0, 1)$ bertindak sebagai koefisien yang menentukan seberapa kuat sistem mengingat rekam jejak masa lalu, sementara $(1 - \lambda)$ merupakan tingkat responsivitas sistem terhadap kesalahan baru. Secara arsitektural, penggunaan EMA dirancang sebagai mekanisme pertahanan terhadap ancaman temporal, yang memungkinkan agen jahat mencoba mengeksploitasi sistem dengan beresilasi antara perilaku jujur dan manipulatif. Parameter λ ini berfungsi sebagai titik kalibrasi un-

tuk menentukan seberapa reaktif sistem harus mendegradasi probabilitas agen saat terjadi penurunan performa tiba-tiba.

Sebagai langkah final *post-round settlement*, skor reputasi baru T_{t+1} akan disimpan di *on-chain*. Mekanisme ini menjamin bahwa perhitungan kepercayaan beroperasi sepenuhnya tanpa asumsi sentral, melainkan didasarkan pada jejak audit yang transparan dan terus diupdate melalui persamaan 3-7.

3.3 Alur Penelitian

Untuk menjamin bahwa penelitian ini berjalan secara sistematis, terukur, dan dapat direplikasi, metodologi penelitian dieksekusi melalui pendekatan rekayasa sistem (*systems engineering*) yang terbagi ke dalam tiga fase utama, yaitu analisis dan pemodelan, pengembangan *prototype* sistem, serta pengujian dan laporan. Mengadopsi prinsip pengembangan perangkat lunak yang bersifat iteratif, proses implementasi dan evaluasi dalam arsitektur ini tidak berjalan secara linear satu arah, melainkan melibatkan mekanisme umpan balik (*feedback loop*) melalui tahapan *debugging* dan kalibrasi paramter untuk mencapai sistem yang optimal. Secara komprehensif, alur tahapan penelitian ini diilustrasikan pada Gambar 3.8.



Gambar 3.8. Alur penelitian yang terdiri dari 3 fase, yaitu analisis dan pemodelan, pengembangan *prototype* sistem, serta pengujian dan laporan.

3.3.1 Analisis dan Pemodelan

Fase pertama ini merupakan fondasi konseptual dari keseluruhan penelitian yang bertujuan untuk mendefinisikan ruang lingkup masalah dan merancang solusi sebelum tahapan implementasi dilakukan. Dalam prinsip rekayasa perangkat lunak, proses rekayasa kebutuhan (*requirements engineering*) dan analisis sistem adalah tahap yang kritis, karena kelalaian pada tahap ini akan secara tak terhindarkan menyebabkan kegagalan pada desain dan implementasi sistem di tahap akhir. Fase analisis dan pemodelan ini terdiri dari 3 proses berikut:

3.3.1.1 Studi Literatur

Proses ini berfokus pada pengumpulan dan pemahaman landasan teori fundamental yang menopang penelitian. Kajian literatur difokuskan secara spesifik pada tiga domain utama, yaitu mekanisme LLM beserta kerentanannya terhadap fenomena halusinasi [5–7, 43], pemanfaatan kerangka kerja *Multi Agent System* (MAS) sebagai pendekatan untuk memitigasi halusinasi [14, 21], serta fondasi algoritma *Byzantine Fault Tolerance* (BFT) dan metode komputasi agregasi spasial seperti *Geometric Median* [26, 32]. Eksplorasi lintas disiplin ini menjadi prasyarat utama untuk memahami batasan sistem saat ini dan merancang arsitektur yang tangguh.

3.3.1.2 Analisis *State-of-the-Art* (SOTA) dan Perumusan Masalah

Setelah landasan teori terbangun, proses dilanjutkan dengan evaluasi dan analisis terhadap protokol-protokol SOTA. Analisis ini membedah kelemahan pada arsitektur SOT, seperti protokol Trusted MultiLLMN [17], BlockAgents [13], dan Decent-LLMs [11]. Dari analisis komparatif tersebut, diidentifikasi dua celah kerentanan utama yang belum terselesaikan, yaitu masalah keandalan evaluator (*evaluator reliability problem*) akibat manipulasi agen jahat, dan hambatan skalabilitas komunikasi (*scalability bottleneck*). Celah keamanan ini kemudian diformulasikan secara matematis menjadi rumusan masalah yang menjadi objektif penelitian.

3.3.1.3 Perancangan Arsitektur dan Model Matematis

Berdasarkan rumusan masalah yang telah ditetapkan, proses ini mendesain sistem yang diusulkan melalui sintesis arsitektur lintas domain dari berbagai literatur. Alih-alih merancang sistem dari nol, metodologi yang digunakan adalah mengadopsi dan mengintegrasikan metode-metode matematis dari berbagai literatur. Perancangan arsitektur ini dibagi ke dalam tiga lapisan operasional utama: *Agent Layer*, *Consensus Layer*, dan *Blockchain Layer*.

Selain itu, tahap ini mencakup perumusan dan hibridisasi persamaan matematis yang bersumber dari sintesis dua literatur SOTA:

1. **Modul Sharding Komite:** Desain distribusi probabilitas fungsi *Softmax* diadopsi dari mekanisme alokasi tugas probabilistik pada [12]. Mekanisme ini diadaptasi untuk mengeksekusi *weighted random sampling* dalam pemilihan komite evaluator secara proporsional berdasarkan reputasi guna memecahkan masalah hambatan skalabilitas komunikasi.
2. **Modul Konsensus Spasial:** Implementasi komputasi numerik algoritma Weiszfeld didasarkan pada pendekatan *Geometric Median* (GM) yang digunakan dalam [11]. Dalam penelitian ini, pendekatan tersebut diekstensi secara matematis menjadi *Weighted Geometric Median* (WGM) agar titik pusat konsensus tidak hanya tangguh terhadap anomali, tetapi juga dapat ditarik ke arah evaluator yang memiliki bobot tinggi.
3. **Modul Reputasi Dinamis:** Penerapan fungsi *Exponential Moving Average* (EMA) diadaptasi dari mekanisme insentif pembentuk perilaku (*behaviour-shaping incentive mechanism*) [12].

3.3.2 Pengembangan *Prototype* Sistem

Fase kedua dari alur penelitian adalah proses rekayasa perangkat lunak *software engineering* untuk menerjemahkan model arsitektur dan matematis ke dalam *prototype* sistem yang fungsional. Proses ini melibatkan pengembangan komponen *off-chain* dan *on-chain*, serta integrasi antarmuka untuk menghubungkan agen LLM dengan lapisan *blockchain*. Mengacu pada Gambar 3.8, fase ini dieksekusi melalui empat tahapan berikut:

3.3.2.1 Pembuatan Entitas Agen LLM

Tahap ini berfokus pada inisialisasi agen otonom menggunakan kerangka kerja orkestrasi LLM yang dihubungkan melalui API. Agen diinisialisasi menjadi dua peran yang terisolasi secara fungsional: agen pekerja dan agen evaluator. Implementasi pada tahap ini berfokus pada rekayasa *prompt*, yakni agen evaluator diprogram secara eksplisit untuk mengekstraksi dan menilai jawaban agen pekerja ke dalam format matriks evaluasi 5-dimensi.

3.3.2.2 Implementasi Modul *Sharding* Komite

Tahap ini berfokus pada pengembangan mekanisme seleksi evaluator yang efisien untuk memecahkan hambatan skalabilitas komunikasi (*scalability bottleneck*) pada arsitektur *leaderless baseline*. Dengan memilih sebagian komite (*sharding*), sistem dapat mereduksi kompleksitas komunikasi jaringan dari $O(N^2)$ menjadi $O(K^2)$. Modul ini diimplementasikan menggunakan lingkungan komputasi *off-chain* dengan logika probabilitas berbasis *Softmax* yang merujuk pada Algoritma 1.

Proses seleksi komite ini tidak dilakukan secara deterministik murni, melainkan secara probabilistik untuk menyeimbangkan antara eksploitasi agen bereputasi tinggi dan eksplorasi terhadap agen baru. Secara operasional, algoritma ini terdiri dari beberapa tahapan:

1. **Inisialisasi dan Penanganan *Cold Start*:** Sistem pertama-tama mengambil seluruh skor reputasi historis (T_i) dari lapisan *blockchain* untuk himpunan evaluator yang tersedia. Untuk memastikan lingkungan jaringan yang adil bagi agen yang baru bergabung, agen tanpa riwayat transaksi diberikan nilai *default* $T_i = 0.5$. Pendekatan ini menyelesaikan masalah *cold start*, sehingga agen baru tetap memiliki probabilitas non-nol untuk terpilih ke dalam komite dan membangun reputasi mereka.
2. **Kalibrasi Suhu (*Temperature Scaling*):** Setiap skor reputasi diskalakan menggunakan parameter suhu atau *temperature* (τ). Parameter ini bertindak sebagai pengendali eksploitasi sistem. Nilai τ yang rendah akan membuat perbedaan kecil pada reputasi menjadi sangat signifikan, sedangkan nilai τ yang tinggi akan meratakan distribusi probabilitas.
3. **Stabilisasi Numerik Eksponensial:** Sebelum menghitung fungsi eksponensial, algoritma menerapkan teknik stabilisasi numerik dengan mencari nilai reputasi berskala tertinggi m , lalu mengurangkannya dari setiap elemen s_i sebelum dimasukkan ke dalam fungsi eksponensial. Langkah ini sangat esensial pada implementasi untuk mencegah terjadinya masalah akibat limitasi presisi *floating point* saat menghitung nilai eksponensial yang sangat besar.
4. **Distribusi Probabilitas dan Pengambilan Sampel:** Nilai eksponensial yang telah distabilkan kemudian dinormalisasi terhadap total eksponensial populasi Z untuk menghasilkan probabilitas $P(e_i)$ bagi setiap agen. Berdasarkan matriks probabilitas tersebut, mesin *off-chain* mengeksekusi pengambilan sampel berbobot tanpa pengembalian (*weighted random sampling without replacement*) untuk memastikan bahwa tepat K evaluator unik ditarik menjadi komite yang sah.

3.3.2.3 Implementasi *Weighted Geometric Median* (Weiszfeld)

Tahap ini berfokus pada algoritma konsensus yang bertugas mengaggregasi matriks evaluasi 5-dimensi yang dihasilkan oleh komite evaluator. Modul ini diimplementasikan untuk menghitung titik ekuilibrium *Weighted Geometric Median* (WGM) menggunakan adaptasi numerik dari algoritma iteratif Weiszfeld. Implementasi algoritma tersebut direpresentasikan pada Algoritma 2.

Algoritma ini menerima masukan berupa matriks vektor evaluasi $A \in \mathbb{R}^{K \times 5}$ dari komite yang terpilih, beserta vektor bobot reputasi historis $w \in \mathbb{R}^K$. Secara operasional, WGM mengeksekusi empat tahapan komputasi:

Algorithm 1 Pemilihan Komite Berbasis Softmax

Require: Himpunan evaluator $E = \{e_1, e_2, \dots, e_{N_e}\}$;

Skor reputasi T_i untuk tiap e_i ;

Ukuran komite $K \in \mathbb{Z}^+$;

Suhu (temperature) $\tau > 0$

Ensure: Komite $C \subseteq E$ dengan $|C| = K$

```
1: if  $K = N_e$  then
2:   return  $E$                                 ▷ Semua evaluator menjadi komite
3: end if
4: for tiap  $e_i \in E$  do
5:    $T_i \leftarrow$  skor reputasi  $e_i$  atau 0.5    ▷ Default reputasi jika tidak tercatat
6: end for
7:  $\mathbf{s} \leftarrow \left[ \frac{T_1}{\tau}, \frac{T_2}{\tau}, \dots, \frac{T_{N_e}}{\tau} \right]$     ▷ Skalasi dengan suhu
8:  $m \leftarrow \max(\mathbf{s})$ 
9: for  $i = 1$  sampai  $N_e$  do
10:   $v_i \leftarrow \exp(s_i - m)$                 ▷ Eksponensial dengan stabilisasi numerik
11: end for
12:  $Z \leftarrow \sum_{i=1}^{N_e} v_i$ 
13: for  $i = 1$  sampai  $N_e$  do
14:   $P(e_i) \leftarrow \frac{v_i}{Z}$                     ▷ Probabilitas softmax evaluator  $e_i$ 
15: end for
16:  $C \leftarrow \text{WeightedSampleWithoutReplacement}(E, K, P)$     ▷ Pengambilan sampel
   berbobot tanpa penggantian
17: return  $C$ 
```

Algorithm 2 Iterasi Weiszfeld Berbobot dalam Ruang 5-Dimensi

```
1: function WEISZFELD-ITERATION( $A, w, \epsilon, k_{\max}$ )    ▷  $A \in \mathbb{R}^{K \times 5}, w \in \mathbb{R}^K$ 
2:    $y \leftarrow \text{median}(A, \text{sumbu} = 0)$             ▷ Inisialisasi dengan median tak berbobot
3:   for  $k = 1$  sampai  $k_{\max}$  do
4:      $d_i \leftarrow \|A_i - y\|_2$ ,    untuk  $i = 1, \dots, K$ 
5:      $\mathcal{I} \leftarrow \{i \mid d_i < \epsilon\}$ 
6:     if  $\mathcal{I} \neq \emptyset$  then
7:       return  $A_{\min(\mathcal{I})}$                 ▷ Kunci pada anchor point (singularitas)
8:     end if
9:      $\tilde{w}_i \leftarrow \frac{w_i}{d_i}$ ,    untuk  $i = 1, \dots, K$ 
10:     $y_{\text{next}} \leftarrow \frac{\sum_{i=1}^K \tilde{w}_i \cdot A_i}{\sum_{i=1}^K \tilde{w}_i}$     ▷ Pembaruan fixed-point Weiszfeld
11:    if  $\|y_{\text{next}} - y\|_2 < \epsilon$  then
12:      return  $y_{\text{next}}$                     ▷ Konvergensi tercapai
13:    end if
14:     $y \leftarrow y_{\text{next}}$ 
15:  end for
16:  return  $y$                                 ▷ Maksimum iterasi tercapai
17: end function
```

1. **Inisialisasi:** Alih-alih memulai iterasi dari *mean* atau titik acak, estimasi awal (y_0) diinisialisasi menggunakan median marginal (median tak berbobot) dari setiap sumbu dimensi. Keputusan ini sangat krusial dalam lingkungan *Byzantine*. Inisialisasi menggunakan *mean* akan langsung menarik titik awal ke arah vektor ekstrem milik penyerang, sementara inisialisasi median memberikan titik tolak yang sudah secara inheren tangguh terhadap anomali sehingga secara signifikan mempersingkat konvergensi algoritma Weiszfeld.
2. **Penanganan Singularitas:** Kelemahan matematis dari algoritma Weiszfeld standar adalah fungsi objektifnya menjadi tidak terdefinisi (terjadi pembagian dengan nol) apabila estimasi titik saat ini jatuh tepat di atas salah satu titik data jangkak (A_i). Untuk mengatasi *singularity* ini, algoritma menggunakan mekanisme penanganan eksplisit, yaitu jarak Euclidean (d_i) antara estimasi dan setiap vektor dihitung dan apabila jarak tersebut jatuh di bawah ambang batas ($\epsilon = 10^{-6}$), algoritma akan mengunci dan mengembalikan A_i tersebut sebagai titik konsensus final.
3. **Pembaruan *Fixed-Point* Berbobot:** Ini adalah inti dari mekanisme pertahanan sistem. Titik estimasi diperbarui secara rekursif (y_{next}) melalui formulasi *fixed-point*. Pada tahap ini, jarak Euclidean invers ($\frac{1}{d_i}$) dikalikan secara langsung dengan bobot reputasi agen (w_i) untuk menghasilkan bobot gabungan ($\tilde{w}_i$). Transformasi ini memastikan bahwa agen dengan rekam jejak yang jujur akan memiliki tarikan gravitasi matematis yang jauh lebih kuat terhadap titik konsensus final dibandingkan dengan agen jahat.
4. **Terminasi Ekuilibrium:** Iterasi dikonfigurasi untuk berhenti berdasarkan kriteria ganda. Terminasi optimasi dicapai apabila norma pergeseran posisi antar iterasi telah konvergen di bawah ambang batas toleransi ($|y_{k+1} - y_k|_2 < \epsilon$) dengan $\epsilon = 10^{-6}$. Namun, sebagai mekanisme pengamanan terhadap anomali isolasi, algoritma dipaksa berhenti dan mengembalikan estimasi terbaik saat itu apabila iterasi telah mencapai batas (k_{max}).

3.3.2.4 Implementasi *Exponential Moving Average* (EMA)

Setelah agregasi spasial (WGM) menghasilkan titik konsensus final untuk setiap jawaban pekerja, arsitektur memerlukan mekanisme untuk mengevaluasi kinerja masing-masing evaluator dan memperbarui bobot kepercayaan mereka. Tahap ini berfokus pada implementasi algoritma pembaruan reputasi yang bersifat dinamis dan berbobot historis. Logika komputasional pada modul ini diturunkan dari adaptasi fungsi EMA dan direpresentasikan pada Algoritma 3.

Algoritma ini bertugas menjembatani keluaran dari consensus sebelum delta perubahannya dicatat secara permanen ke dalam *on-chain*. Proses komputasi pada modul ini dieksekusi melalui empat tahapan:

Algorithm 3 Pembaruan Reputasi dengan EMA

Require: Daftar evaluasi E , himpunan titik konsensus Y , faktor smoothing $\alpha \in [0, 1]$, reputasi saat ini T

Ensure: Delta reputasi ΔT

```
1:  $\Delta T \leftarrow \{\}$ 
2: for setiap evaluasi  $e \in E$  do
3:    $evaluator \leftarrow e.evaluator\_id$ 
4:   if  $evaluator \notin T$  then
5:     lanjutkan
6:   end if
7:    $worker\_ids \leftarrow \text{daftar kunci dari } e.scores$ 
8:    $n \leftarrow |worker\_ids|$ 
9:   if  $n = 0$  then
10:    lanjutkan
11:  end if
12:   $d_{total} \leftarrow 0$ 
13:  for setiap  $worker \in worker\_ids$  do
14:     $v_{eval} \leftarrow e.scores[worker]$ 
15:     $y_j \leftarrow Y[worker]$ 
16:     $d_{total} \leftarrow d_{total} + \|v_{eval} - y_j\|_2$ 
17:  end for
18:   $\bar{d} \leftarrow d_{total} / n$ 
19:   $S_t \leftarrow 1.0 - (\bar{d} / d_{max})$ 
20:   $S_t \leftarrow \max(0.0, \min(1.0, S_t))$ 
21:   $T_{lama} \leftarrow T[evaluator]$ 
22:   $T_{baru} \leftarrow \alpha \cdot S_t + (1 - \alpha) \cdot T_{lama}$ 
23:   $T_{baru} \leftarrow \max(0.0, \min(1.0, T_{baru}))$ 
24:   $T[evaluator] \leftarrow T_{baru}$ 
25:   $\Delta T[evaluator] \leftarrow T_{baru} - T_{lama}$ 
26: end for
27: return  $\Delta T$ 
```

1. **Penyaringan Data dan Kalkulasi Galat Spasial Rata-Rata:** Untuk setiap evaluator yang berpartisipasi dalam komite, algoritma pertama-tama menghitung tingkat deviasi atau galat evaluasi mereka. Alih-alih hanya mengakumulasi total galat, algoritma menghitung galat spasial rata-rata ($\bar{d}$) menggunakan metrik jarak Euclidean. Jarak antar vektor evaluasi 5-dimensi yang diajukan oleh agen (v_{eval}) dan titik konsensus final (y_j) yang dihasilkan oleh WGM diakumulasikan, kemudian dibagi dengan total jawaban pekerja yang dievaluasi (n). Normalisasi berbasis rata-rata ini sangat krusial secara arsitektural untuk memastikan bahwa evaluator tidak dipenalti secara tidak proporsional apabila jumlah beban kerja evaluasi (n) antar-agen berbeda.
2. **Normalisasi *Inverse Min-Max* dan *Bounding*:** Karena jarak Euclidean tidak memiliki batas atas absolut, nilai $\bar{d}$ tidak dapat langsung digunakan sebagai pengali utilitas. Oleh karena itu, algoritma menerapkan fungsi normalisasi *Inverse Min-Max* untuk mentransformasi galat spasial rata-rata tersebut menjadi Skor Performa (S_t) sesuai Persamaan 3-6. Inversi ini (dikurangkan dari 1.0) memastikan bahwa jarak terdekat dengan konsensus (galat minimum) menghasilkan skor performa yang mendekati 1, sementara deviasi ekstrem menghasilkan skor mendekati 0. Sebagai langkah *defensive programming*, skor performa kemudian di-*clamp* pada rentang $[0.0, 1.0]$ untuk mencegah anomali komputasi *floating-point* yang dapat menyebabkan kegagalan pada saat penulisan ke dalam *smart contract*.
3. **Pembaruan Reputasi:** Skor performa (S_t) pada ronde berjalan tidak serta-merta menggantikan reputasi keseluruhan agen secara mutlak, melainkan disuntikkan ke dalam fungsi pembaruan EMA (T_{t+1}) sesuai Persamaan 3-7. Algoritma menghitung skor reputasi ronde berikutnya dengan menggunakan parameter *smoothing* (α). Penggunaan rumusan EMA ini memastikan bahwa sistem memiliki *forgetting mechanism* yang aman, agen jujur dapat memulihkan reputasinya dan juga agen jahat akan mendapati skor reputasi dan bobot pengaruh mereka terdegradasi.
4. **Ekstraksi Delta Reputasi:** Sebagai tahapan akhir, algoritma menghitung dan mengembalikan nilai perubahan atau delta reputasi (ΔT). Pendekatan ini dirancang untuk mengoptimalkan efisiensi integrasi *on-chain*. Dengan hanya mengirimkan ΔT ke *on-chain*, arsitektur dapat secara signifikan mereduksi beban komputasi transaksi dan *gas fee*.

3.3.2.5 Pengembangan *Smart Contract* (Pembaruan Reputasi dan *Tie-Breaker*)

Tahap ini berfokus pada perancangan lapisan arsitektural yang berperan sebagai *immutable ledger* untuk menyimpan skor reputasi agen dan merekam jejak audit setiap ronde konsensus. Lapisan *on-chain* ini dirancang sebagai *smart contract* dengan arsitektur *write-through cache*, dengan seluruh operasi pembacaan reputasi oleh modul *off-chain*

(seperti *Softmax sharding* dan WGM) dilayani dari *cache* memori lokal tanpa latency jaringan, sementara seluruh operasi penulisan secara atomik disinkronkan ke jaringan *blockchain* terlebih dahulu sebelum *cache* diperbarui.

Secara struktural, lapisan *on-chain* ini mengimplementasikan empat kelompok fungsionalitas utama:

1. **Registrasi Agen dan Inisialisasi Reputasi:** Setiap agen yang berpartisipasi dalam jaringan wajib diregistrasi sebelum dapat berinteraksi dengan sistem. Fungsi ini menerima parameter identitas agen dan peran fungsional (*Worker* atau *Evaluator*), kemudian menyimpan *record* agen ke dalam penyimpanan *on-chain*. Sebagai bagian dari mekanisme *cold start* yang telah dijabarkan pada Subbab 3.2.4.2, setiap agen yang baru diregistrasi secara otomatis diberikan skor reputasi awal sebesar $T_i = 0.5$. Nilai *default* ini dipilih agar probabilitas terpilihnya agen baru ke dalam komite tetap non-nol pada distribusi *Softmax*, sehingga tidak terjadi *deadlock* partisipasi.
2. **Pembaruan Reputasi Batch:** Setelah modul EMA pada *off-chain* menyelesaikan komputasi delta reputasi (ΔT) untuk seluruh evaluator komite, hasilnya dikirimkan secara atomik ke *on-chain*. Pendekatan *batch* (pengiriman seluruh pembaruan dalam satu transaksi tunggal) dipilih untuk mengoptimalkan efisiensi transaksi *on-chain*, mengingat pengiriman skor satu per satu melalui transaksi terpisah akan menimbulkan overhead yang secara linier bertambah seiring dengan jumlah anggota komite K . Setiap pembaruan reputasi yang berhasil dicatat memancarkan (*emit*) sebuah *event log* yang berisi identitas agen beserta skor reputasi sebelum dan sesudah pembaruan. *Event* ini berfungsi sebagai jejak audit yang transparan dan dapat diindeks untuk keperluan verifikasi dan analisis.
3. **Pencatatan Runde Konsensus:** Setiap siklus konsensus yang telah diselesaikan direkam secara permanen ke dalam *blockchain*. *Record* ronde ini menyimpan metadata lengkap berupa identitas ronde, *timestamp* blok, pertanyaan yang dievaluasi, daftar anggota komite, identitas *worker* terpilih, status kebenaran jawaban, serta delta reputasi dari setiap evaluator yang berpartisipasi. Pendekatan pencatatan terpisah antara delta reputasi dan *record* ronde ini dirancang untuk mendukung analisis retrospektif, sehingga peneliti dapat melacak secara independen bagaimana konsensus diambil pada setiap ronde dan bagaimana keputusan tersebut memengaruhi evolusi reputasi agen sepanjang eksperimen. Jika ronde melibatkan *tie-breaker*, *record* ronde secara atomik juga menyimpan *block hash*, daftar jawaban yang seri, dan indeks pemenang yang telah diverifikasi.
4. **Verifikasi Tie-Breaker Kriptografis:** Ketika algoritma WGM mendeteksi bahwa dua atau lebih jawaban pekerja memiliki *robust score* identik (*tie*), sistem mengekskusi mekanisme *tie-breaker* yang bersifat deterministik dan tidak dapat dimanipu-

lasi. *Tie-breaker* ini mengimplementasikan dua lapisan verifikasi:

Pada lapisan pertama, resolusi *tie* dilakukan secara komputasional oleh modul *off-chain*. Modul ini menghitung *hash* keccak256 [47] dari setiap jawaban yang seri yang dikonkatenasi dengan *parent block hash* terbaru dari jaringan *blockchain*:

$$H_i = \text{keccak256}(a_i \parallel \mathcal{B}) \quad (3-8)$$

dengan a_i adalah teks jawaban pekerja ke- i yang terlibat dalam *tie* dan $\mathcal{B}$ adalah *hash* blok induk (*parent block hash*). Nilai *hash* H_i yang dihasilkan kemudian diinterpretasikan sebagai bilangan bulat tak bertanda (*unsigned integer*) 256-bit. Jawaban dengan nilai H_i terbesar ditetapkan sebagai pemenang. Penggunaan *parent block hash* sebagai sumber entropi dipilih karena nilai ini bersifat publik, tidak dapat diprediksi sebelumnya, dan secara inheren berubah pada setiap blok baru, sehingga mencegah agen manapun untuk memanipulasi hasil *tie-breaker* dengan menghitung ke depan (*precomputation*).

Pada lapisan kedua, keputusan *tie-breaker* yang telah diambil oleh modul *off-chain* diverifikasi secara independen oleh *smart contract on-chain*. *Smart contract* ini menghitung ulang seluruh rangkaian *hash* $\{H_1, H_2, \dots, H_m\}$ untuk m jawaban yang seri, kemudian membandingkan indeks pemenang yang dihitung ulang dengan indeks yang dikirimkan oleh modul *off-chain*. Karena fungsi verifikasi ini tidak memodifikasi *state* (*read-only*), verifikasi dapat dieksekusi tanpa biaya tambahan. Hasil verifikasi ini kemudian dicatat secara permanen dalam *record* ronde. Arsitektur dua lapisan ini menjamin bahwa proses *tie-breaking* tidak hanya deterministik, tetapi juga terverifikasi secara on-chain sehingga integritasnya tidak dapat dipertanyakan oleh entitas manapun.

Secara keseluruhan, lapisan *on-chain* bertindak sebagai fondasi keamanan data dari arsitektur yang diusulkan. Dengan menyimpan skor reputasi dan jejak audit ronde secara *on-chain*, sistem menjamin bahwa seluruh evolusi kepercayaan agen bersifat transparan, tidak dapat diubah (*tamper-proof*), dan secara penuh dapat diaudit, sehingga memenuhi prinsip desentralisasi yang menjadi landasan arsitektur ini.

3.3.2.6 Pengujian Integrasi dan Iterasi *Debugging*

Tahap terakhir pada fase pengembangan adalah pengujian integrasi (*integration testing*) antara mesin konsensus agen LLM di lapisan *off-chain* dengan *smart contract* di lapisan *on-chain*. Pengujian ini bertujuan untuk memastikan bahwa kedua lingkungan arsitektur tersebut dapat saling bertukar data secara valid melalui antarmuka *Remote Procedure Call* (RPC).

Fase ini merepresentasikan titik keputusan pertama di dalam alur penelitian yang diilustrasikan pada Gambar 3.8. Sistem mengevaluasi apakah transaksi dari agen, seperti

pemanggilan fungsi dan pengiriman pembaruan reputasi, berhasil dieksekusi dan dicatat oleh *blockchain* tanpa penolakan (*transaction revert*). Apabila ditemukan kegagalan komunikasi atau galat pada program, alur penelitian akan berputar kembali (*loop back*) ke proses *debugging* untuk melakukan perbaikan pada tingkat kode atau konfigurasi. Siklus pengujian dan perbaikan ini dilakukan secara iteratif hingga *prototype* terbukti dapat beroperasi secara fungsional dan stabil, yang menandakan kesiapannya untuk menghadapi evaluasi simulasi serangan pada fase selanjutnya.

3.3.3 Pengujian dan Laporan

Fase terakhir dari alur penelitian merupakan tahapan evaluasi kuantitatif yang bertujuan untuk membuktikan hipotesis dan mengukur sejauh mana arsitektur *Reputation-Weighted Consensus* mampu menyelesaikan permasalahan yang telah dirumuskan. Setelah *prototype* sistem berhasil melewati pengujian integrasi dan dinyatakan fungsional pada fase sebelumnya, tahap ini berfokus pada pelaksanaan eksperimen simulasi serangan dalam berbagai skenario ancaman *Byzantine*, ekstraksi metrik performa kuantitatif, kalibrasi hiperparameter sistem, serta penyusunan laporan hasil penelitian secara komprehensif. Mengacu pada Gambar 3.8, fase ini juga menjadi titik keputusan kedua sehingga hasil evaluasi dapat memicu mekanisme umpan balik berupa kalibrasi ulang parameter apabila kinerja sistem belum memenuhi ambang batas yang ditetapkan. Secara berurutan, fase pengujian dan laporan ini dieksekusi melalui empat sub-tahapan berikut.

3.3.3.1 Simulasi dan *Benchmark*

Lingkungan simulasi dikondisikan untuk meniru jaringan terdesentralisasi publik yang di dalamnya agen-agen jahat menyusup ke dalam komite evaluator dan berupaya memanipulasi proses konsensus. Untuk mengevaluasi ketahanan sistem secara komprehensif, injeksi anomali dieksekusi melalui empat skenario *threat model*:

1. ***Random Attack***: Agen evaluator *Byzantine* menghasilkan vektor evaluasi 5-dimensi yang sepenuhnya acak tanpa mempertimbangkan konteks jawaban pekerja. Skenario ini mensimulasikan agen yang tidak berfungsi (*non-functional*) atau mengalami kerusakan sistem secara acak, sehingga berfungsi sebagai batas bawah (*lower bound*) pengujian untuk mengukur kemampuan dasar sistem dalam menoleransi noise tanpa pola tertentu.
2. ***Inverse Attack***: Agen evaluator *Byzantine* secara sistematis menginversi kriteria evaluasi, yaitu memberikan skor tinggi pada jawaban yang buruk dan skor rendah pada jawaban berkualitas. Skenario ini mensimulasikan agen yang secara sengaja bertindak bertentangan dengan kebenaran (*adversarial inversion*), menguji kemampuan sistem dalam mendeteksi dan menetralkan anomali yang memiliki pola terstruktur dan konsisten.

3. ***Oscillation Attack***: Agen evaluator *Byzantine* berperilaku jujur pada sebagian ronde untuk membangun kepercayaan dan meningkatkan skor reputasi mereka, kemudian secara tiba-tiba meluncurkan manipulasi ekstrem pada ronde-ronde tertentu. Skenario ini secara spesifik dirancang untuk mengeksploitasi kerentanan temporal pada sistem berbasis reputasi historis, menguji apakah mekanisme *Exponential Moving Average* (EMA) mampu merespons degradasi performa dengan cukup cepat sebelum dampak manipulasi terakumulasi.
4. ***Amplification Collusion***: Sekelompok agen evaluator *Byzantine* secara terkoordinasi berkolusi dengan memberikan skor sempurna (maksimal) pada jawaban pekerja *Byzantine* dan skor nol pada jawaban agen jujur. Skenario ini merepresentasikan ancaman paling destruktif dalam konteks arsitektur konsensus berbasis komite, karena upaya kolusi secara langsung berupaya menggeser titik agregasi WGM melalui akumulasi bobot spasial terkoordinasi.

Keempat skenario serangan di atas dieksekusi secara bergradasi dengan memvariasikan persentase agen jahat yang menyusup ke dalam komite, mulai dari kondisi jaringan bersih (0%) hingga kondisi infiltrasi kritis (50%). Peningkatan dilakukan secara bertahap untuk mengamati perilaku degradasi sistem secara granular dan mengidentifikasi titik kritis (*breaking point*) ketika mekanisme konsensus mulai gagal mempertahankan akurasi.

Sebagai proses *benchmarking*, skenario serangan yang identik juga dieksekusi pada arsitektur *baseline* DecentLLMs [11]. Protokol *baseline* ini menggunakan arsitektur *leaderless* dengan algoritma *Geometric Median* (GM) tanpa pembobotan reputasi dan tanpa seleksi komite (*full-participation*), sehingga seluruh populasi evaluator berpartisipasi dalam setiap ronde konsensus. Pelaksanaan skenario yang identik pada kedua arsitektur memungkinkan pengukuran komparasi kinerja secara langsung, khususnya dalam aspek akurasi konsensus, ketahanan *Byzantine*, serta latensi eksekusi ronde.

3.3.3.2 Ekstraksi dan Analisis Metrik

Selama simulasi serangan berlangsung, sistem secara pasif melakukan ekstraksi log data dari mesin konsensus *off-chain* dan rekaman transaksi (*event log*) pada lapisan *blockchain*. Proses ekstraksi ini bertujuan untuk memonitor kinerja sistem secara real-time serta menyediakan data kuantitatif yang menjadi dasar analisis pada sub-tahapan evaluasi selanjutnya. Metrik utama yang diekstraksi dan dianalisis meliputi tiga dimensi berikut:

1. **Margin Keamanan (*Meritocratic Safety Margin*)**: Keamanan sistem diukur melalui stabilitas *Honest Weight Fraction* (Λ) sebagaimana didefinisikan pada Persamaan 3-1. Metrik ini merepresentasikan rasio akumulasi bobot reputasi agen jujur terha-

dap total bobot komite pada setiap ronde. Selama Λ dipertahankan di atas ambang batas mayoritas (≥ 0.5), konsensus spasial WGM secara matematis terjamin tidak dapat dibajak oleh agen jahat [17]. Pada arsitektur yang diusulkan, margin ini bersifat dinamis karena mekanisme EMA secara aktif mendegradasi bobot agen jahat, sehingga menciptakan apa yang disebut sebagai *Meritocratic Safety Margin*, yaitu selisih antara fraksi bobot jujur aktual dan ambang batas kritis 0.5. Analisis metrik ini mengukur apakah sistem mampu secara proaktif mempertahankan margin keamanan positif meskipun tingkat infiltrasi agen jahat meningkat.

2. **Akurasi Konsensus (*Consensus Accuracy*):** Akurasi dihitung sebagai persentase ronde yang jawaban terpilihnya oleh algoritma konsensus sesuai dengan *ground truth* dari *dataset* evaluasi. Kedua *dataset* yang digunakan, yaitu TruthfulQA [15] dan ARC-Easy [16], sama-sama berformat pilihan ganda (*multiple choice*), sehingga akurasi diukur dengan membandingkan opsi yang dipilih sistem terhadap kunci jawaban referensi pada setiap ronde. Metrik ini menjadi indikator utama keandalan (*reliability*) sistem sebagai *data feed*, sebab akurasi yang tinggi mengindikasikan bahwa mekanisme agregasi WGM berhasil menetralisasi manipulasi evaluator *Byzantine* dan menghasilkan keputusan yang faktual.
3. **Latensi Eksekusi (*Execution Latency*):** Latensi diukur sebagai total waktu yang diperlukan dari masuknya *prompt* hingga sistem menghasilkan jawaban konsensus final, yang mencakup waktu pemrosesan LLM oleh agen pekerja, waktu komputasi agregasi WGM, serta waktu penulisan pembaruan reputasi ke lapisan *on-chain*. Metrik ini secara spesifik digunakan untuk mengvalidasi klaim bahwa mekanisme *Softmax Sharding* berhasil mereduksi kompleksitas komunikasi dari $O(N^2)$ menjadi $O(K^2)$, yang secara teoritis harus termanifestasi sebagai penurunan latensi komputasi agregasi. Perbandingan latensi antara arsitektur yang diusulkan (dengan komite berukuran K) terhadap arsitektur *baseline* (dengan partisipasi penuh N_e) dilakukan untuk mengukur besaran peningkatan efisiensi secara kuantitatif.

Ketiga metrik ini diekstraksi secara agregat untuk setiap kombinasi skenario serangan dan tingkat infiltrasi. Hasil ekstraksi kemudian digunakan sebagai input utama pada tahapan evaluasi sistem dan kalibrasi parameter pada sub-tahapan selanjutnya.

3.3.3.3 Evaluasi Sistem dan Kalibrasi Parameter

Tahap ini merupakan titik keputusan (*decision point*) kritis dalam alur penelitian yang mengacu pada mekanisme umpan balik pada Gambar 3.8. Data metrik yang telah diekstraksi pada sub-tahapan sebelumnya dievaluasi untuk menjawab satu kondisi absolut: apakah sistem mampu bertahan pada ekuilibrium yang optimal? Ketahanan dinilai optimal apabila metrik *Meritocratic Safety Margin* (Λ) secara konsisten berada di atas ambang batas kritis ($\Lambda \geq 0.5$) meskipun sistem berada di bawah skenario serangan

mayoritas agen jahat, tanpa terjadi penurunan signifikan pada akurasi konsensus.

Apabila hasil evaluasi menunjukkan anomali atau kegagalan pertahanan, misalnya margin keamanan yang terdegradasi di bawah ambang batas atau akurasi yang menurun drastis pada tingkat infiltrasi tertentu, proses penelitian dikembalikan (*loop back*) menuju fase pengembangan *prototype* untuk melakukan penyesuaian parameter. Kalibrasi dilakukan pada tiga variabel kunci sistem:

1. **Faktor *Smoothing* EMA (λ):** Mengatur keseimbangan antara memori historis dan responsivitas terhadap kesalahan baru, sebagaimana didefinisikan pada Persamaan 3-7. Nilai λ yang terlalu tinggi menyebabkan sistem terlalu lambat merespons serangan *oscillation*, sementara nilai yang terlalu rendah membuat sistem rentan terhadap fluktuasi noise sesaat dari agen jujur.
2. **Ukuran Komite (K):** Menentukan jumlah evaluator yang berpartisipasi dalam setiap ronde konsensus. Nilai K yang terlalu kecil meningkatkan risiko terpilihnya mayoritas agen jahat secara kebetulan, sementara nilai K yang terlalu besar mengurangi efisiensi yang dicapai oleh mekanisme *Softmax Sharding*.
3. **Parameter Suhu *Softmax* (τ):** Mengendalikan distribusi probabilitas seleksi komite, sebagaimana didefinisikan pada Persamaan 3-2. Kalibrasi τ memengaruhi *trade-off* antara eksploitasi agen bereputasi tinggi dan eksplorasi terhadap agen baru.

Siklus evaluasi dan kalibrasi ini dilakukan secara iteratif hingga sistem terbukti tangguh pada seluruh skenario serangan yang diuji. Konvergensi proses kalibrasi ditandai oleh stabilisasi ketiga metrik utama (*Meritocratic Safety Margin*, akurasi konsensus, dan latensi eksekusi) pada nilai yang memenuhi kriteria kuantitatif yang ditetapkan. Hasil dari fase iteratif ini kemudian menjadi input final untuk tahapan visualisasi data dan penyusunan laporan.

3.3.3.4 Visualisasi Data dan Penyusunan Laporan

Setelah ketahanan sistem tervalidasi dan lolos dari tahap evaluasi serta kalibrasi, penelitian memasuki fase finalisasi. Angka-angka mentah dari log ekstraksi metrik dan hasil *benchmarking* diproses menjadi representasi visual yang terstruktur. Visualisasi yang dihasilkan meliputi grafik lintasan evolusi reputasi agen sepanjang ronde konsensus untuk mengamati dinamika sistem umpan balik EMA, plot perbandingan *Meritocratic Safety Margin* antara arsitektur yang diusulkan terhadap arsitektur *baseline* pada berbagai tingkat infiltrasi *Byzantine*, serta diagram komparasi akurasi konsensus dan latensi eksekusi antar-skenario serangan. Visualisasi ini tidak hanya berfungsi sebagai alat bantu interpretasi, tetapi juga sebagai bukti empiris yang mendukung atau menolak hipotesis penelitian.

Pada tahap akhir, seluruh fondasi arsitektur, formulasi matematis, hasil simulasi, dan temuan analisis data dikonsolidasikan secara komprehensif ke dalam naskah tugas akhir. Proses konsolidasi ini mencakup penyusunan argumentasi logis yang menghubungkan antara rumusan masalah, desain arsitektur, bukti kuantitatif, dan kesimpulan penelitian sebagai bentuk pertanggungjawaban ilmiah.

3.4 Masalah dan Keterbatasan *Prototype* Penelitian

Secara arsitektural, kerangka kerja *oracle* multi-agen terdesentralisasi yang diusulkan dirancang untuk beroperasi pada jaringan fisik *peer-to-peer* (P2P) murni. Dalam skenario penerapan dunia nyata, seperti pada jaringan *mainnet blockchain*, setiap agen pekerja dan evaluator akan di-*host* pada *node* perangkat keras yang terisolasi dengan sumber daya komputasi dan akselerator masing-masing, memungkinkan paralelisasi absolut tanpa adanya antrean antar-agen.

Namun, sebagai sebuah *prototype* penelitian, lingkungan jaringan multi-agen ini disimulasikan secara lokal dengan merutekan seluruh eksekusi *Large Language Model* (LLM) melalui satu antarmuka pemrograman aplikasi (*API*) terpusat, yakni OpenRouter [41]. Karena semua permintaan inferensi dari puluhan agen dieksekusi secara konkuren melalui satu titik masuk tunggal, simulasi pertukaran pesan komite ini memicu lonjakan permintaan asinkron yang melanggar batas perlindungan server komersial, menghasilkan galat HTTP 429 *Too Many Requests*. Untuk mencegah penolakan server, *prototype* sistem mengimplementasikan mekanisme pembatasan konkurensi terkontrol menggunakan antrean `asyncio.Semaphore` dengan kapasitas maksimal 30 permintaan konkuren.

Oleh karena itu, batasan utama dari penelitian ini terletak pada metrik latensi eksekusi. Angka latensi absolut yang dihasilkan pada eksperimen simulasi mengandung *artificial queuing latency* yang tidak akan terjadi pada penerapan jaringan P2P sesungguhnya yang di dalamnya setiap agen memiliki jalur komunikasi independen. Meski demikian, karena pembatasan konkurensi ini diberlakukan secara identik baik pada pengujian arsitektur yang diusulkan maupun pada arsitektur *baseline*, validitas komparasi kinerja antar sistem, seperti efisiensi komunikasi $O(K^2)$ terhadap $O(N^2)$, tetap terjaga secara empiris maupun matematis.

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil penelitian yang bertujuan untuk menjawab tiga tujuan penelitian yang telah dirumuskan pada Subbab 1.3. Pertama, realisasi mekanisme konsensus dengan pembobotan dinamis berbasis rekam jejak keandalan agen melalui algoritma *Weighted Geometric Median* (WGM). Kedua, implementasi mekanisme pelacakan kualitas agen dengan pembaruan reputasi adaptif berbasis *Exponential Moving Average* (EMA). Ketiga, penerapan mekanisme seleksi komite berbasis probabilitas *Softmax* untuk menyeimbangkan skalabilitas dan keamanan jaringan.

4.1 Implementasi dan *Proof of Concept* Sistem

Subbab ini menyajikan realisasi arsitektur *Reputation-Weighted Consensus* yang telah dirancang pada Subbab 3.2.3 dan 3.2.4, dari lingkungan eksekusi hingga konfigurasi operasional dan arena uji eksperimen.

4.1.1 Lingkungan Eksekusi dan Orkestrasi Agen

Seluruh rangkaian eksperimen simulasi dalam penelitian ini dieksekusi secara konsisten dan terisolasi pada lingkungan *Virtual Private Server* (VPS) dengan sistem operasi Ubuntu 22.04 LTS, yang dilengkapi dengan 2 vCPU, memori 4 GB, dan penyimpanan 60 GB (spesifikasi lengkap telah diuraikan pada Subbab 3.1.1). Pemilihan VPS sebagai lingkungan eksekusi bertujuan untuk menjamin reproduktibilitas hasil eksperimen serta menghindari variabel *confounding* yang mungkin timbul dari lingkungan komputasi lokal.

Bahasa pemrograman yang digunakan adalah Python versi ≥ 3.12 dengan manajemen paket menggunakan `uv` versi 0.9.26. Eksekusi asinkron menjadi fondasi utama dalam arsitektur perangkat lunak, mengingat sistem harus mengoordinasikan puluhan panggilan LLM secara konkuren tanpa mengorbankan *throughput* dan latensi. Modul `asyncio` bawaan Python digunakan sebagai *runtime* asinkron utama, dengan setiap panggilan inferensi LLM dijalankan sebagai *coroutine* yang dapat di-*schedule* secara paralel melalui `asyncio.gather`.

Untuk mengabstraksikan kompleksitas interaksi dengan berbagai model LLM, penelitian ini memanfaatkan *framework* LangChain, khususnya modul `langchain-core` dan `langchain-openai`, sebagai lapisan pemanggilan (*invocation layer*) yang terstandarisasi. Seluruh model LLM diakses secara terpusat melalui layanan *gateway* OpenRouter API [41], yang memungkinkan akses ke 30 model LLM dari 14 *provider* berbeda menggunakan protokol API yang seragam (*OpenAI-compatible API*). Pende-

katan ini mengeliminasi kebutuhan untuk mengonfigurasi *endpoint* dan autentikasi yang berbeda-beda untuk setiap *provider*, sehingga perbedaan antar-model ditangani melalui penggantian parameter `model` pada objek `ChatOpenAI`.

Tiga puluh model LLM yang digunakan dalam eksperimen ini dibagi secara simetris menjadi dua kelompok fungsional, yaitu 15 model *worker* yang bertugas menghasilkan jawaban, dan 15 model *evaluator* yang bertugas menilai kualitas jawaban. Pemilihan model yang heterogen dari berbagai *provider* dilakukan secara deliberatif untuk merepresentasikan keragaman kemampuan, arsitektur, dan biaya inferensi yang nyata dalam ekosistem LLM kontemporer. Daftar lengkap model yang digunakan beserta perannya disajikan pada Tabel L.1 dan Tabel L.2 dalam Lampiran.

Sebagai catatan implementasi, backend OpenRouter yang dikembangkan mengadopsi mekanisme *rate limiting* adaptif berbasis *exponential backoff* untuk menangani respons HTTP 429 (*rate limit exceeded*) dari *provider*. Selain itu, sistem mendukung manajemen multi-kunci API (*multi-key round-robin pool*), dengan setiap kunci API dilengkapi dengan *semaphore* untuk membatasi konkurensi per-kunci (*capacity per key*), sehingga beban permintaan dapat didistribusikan secara merata dan kapasitas kuota API dimanfaatkan secara optimal.

4.1.2 Implementasi Lapisan *Off-Chain*

Lapisan *off-chain* merupakan inti komputasional dari arsitektur yang diusulkan, terdiri dari tiga algoritma utama, yaitu *Softmax Sharding*, *Weighted Geometric Median* (WGM), dan *Exponential Moving Average* (EMA) kemudian dikonversi dari formulasi matematis (yang telah diuraikan pada Subbab 3.2.4) menjadi logika yang dapat dieksekusi secara efisien. Seluruh operasi komputasi vektor dan matriks pada lapisan ini memanfaatkan pustaka NumPy. Pemilihan NumPy sebagai fondasi komputasi didasarkan pada dua pertimbangan utama: (1) kemampuannya untuk mengeksekusi operasi vektor secara terparalelisasi (*vectorized operations*) tanpa eksplisit perulangan Python yang berjalan lambat, dan (2) kompatibilitasnya yang luas dengan ekosistem ilmiah Python sehingga memudahkan integrasi dengan modul asinkron dan *logging* yang telah dibangun.

4.1.2.1 Implementasi *Softmax Sharding*

Formulasi probabilitas *Softmax* pada Persamaan 3-2 dan Algoritma 1 ditranslasikan ke dalam fungsi `compute_softmax_probabilities` yang menerima *dictionary* skor reputasi agen dan mengembalikan distribusi probabilitas ternormalisasi. Inti logika komputasionalnya direpresentasikan sebagai berikut:

```
1 def compute_softmax_probabilities(self, reputations):
2     evaluator_ids = list(reputations.keys())
3     rep_values = np.array(
```

```

4         [reputations[eid] for eid in evaluator_ids]
5     )
6
7     scaled = rep_values / self.config.temperature
8     exp_scaled = np.exp(scaled - np.max(scaled))
9     probs = exp_scaled / np.sum(exp_scaled)
10
11     return dict(zip(evaluator_ids, probs))

```

Kode 4.1. Komputasi Probabilitas *Softmax* Berbasis NumPy

Pada Kode 4.1, baris ke-3–5 mengonversi nilai reputasi dari struktur *dictionary* menjadi vektor NumPy (`rep_values`), yang memungkinkan seluruh operasi matematika selanjutnya dieksekusi secara vektorisasi. Baris ke-7 mengimplementasikan kalibrasi *temperature* sesuai tahapan 2 Algoritma 1, yakni setiap elemen reputasi dibagi dengan hiperparameter τ . Baris ke-8 merupakan implementasi dari teknik stabilisasi numerik yang diuraikan pada tahapan 3 Algoritma 1: pengurangan nilai maksimum ($m = \max(s)$) dari setiap elemen sebelum penerapan fungsi eksponensial mencegah *overflow* pada presisi *floating-point* ketika reputasi berskala menghasilkan nilai eksponensial yang sangat besar. Tanpa teknik ini, nilai $\exp(s_i)$ untuk s_i yang besar akan melampaui batas representasi `float64` dan menghasilkan `inf`, yang merusak seluruh normalisasi. Baris ke-9 menormalisasi vektor eksponensial menjadi distribusi probabilitas yang memenuhi $\sum P(e_i) = 1.0$.

Setelah distribusi probabilitas terbentuk, seleksi komite berukuran K dilakukan melalui mekanisme pengambilan sampel berbobot tanpa pengembalian (*weighted random sampling without replacement*), yang pada implementasinya dielegasikan ke fungsi `np.random.choice`:

```

1 selected_ids = self._rng.choice(
2     evaluator_ids,
3     size=self.config.committee_size,
4     replace=False,
5     p=probabilities,
6 )

```

Kode 4.2. Seleksi Komite Berbasis Sampel Berbobot

Fungsi bawaan NumPy ini secara internal mengimplementasikan algoritma pengambilan sampel berbobot yang menjamin bahwa tepat K agen unik terpilih, dengan probabilitas terpilihnya setiap agen sebanding dengan skor reputasinya. Penggunaan `RandomState` yang di-*seed* secara deterministik memastikan bahwa hasil seleksi bersifat *reproducible* lintas eksekusi eksperimen. Untuk agen yang belum memiliki riwayat tran-

saksi, sistem memberikan nilai reputasi *default* $T_i = 0.5$ sebelum komputasi *Softmax*, yang merupakan implementasi langsung dari mekanisme penanganan *cold start* pada tahapan 1 Algoritma 1.

4.1.2.2 Implementasi *Weighted Geometric Median* (WGM)

Agregasi spasial WGM yang dirumuskan pada Persamaan 3-4 dan dijabarkan secara algoritmik pada Algoritma 2 diimplementasikan melalui metode iteratif Weiszfeld. Inti logika iterasi *fixed-point* Weiszfeld berbobot direpresentasikan sebagai berikut:

```

1 def _weiszfeld_iteration(self, points, weights):
2     y_current = np.median(points, axis=0)
3
4     for _ in range(self.max_iter):
5         distances = np.linalg.norm(
6             points - y_current, axis=1
7         )
8
9         near_zero = np.where(distances < self.epsilon)[0]
10        if len(near_zero) > 0:
11            return points[near_zero[0]]
12
13        weighted_distances = weights / distances
14        numerator = np.sum(
15            points * weighted_distances[:, np.newaxis],
16            axis=0
17        )
18        denominator = np.sum(weighted_distances)
19        y_next = numerator / denominator
20
21        if np.linalg.norm(y_next - y_current) < self.epsilon:
22            return y_next
23
24        y_current = y_next
25
26    return y_current

```

Kode 4.3. Iterasi *Fixed-Point* Weiszfeld Berbobot pada Ruang 5-Dimensi

Pada Kode 4.3, baris ke-2 mengimplementasikan inisialisasi dengan median tak berbobot sesuai tahapan 1 Algoritma 2. Pemilihan median sebagai titik awal alih-alih *mean* memiliki justifikasi ganda: secara matematis, median merupakan estimator yang tangguh (*robust estimator*) terhadap *outlier*, sehingga titik inisialisasi tidak langsung ter-

tarik ke arah vektor ekstrem milik agen jahat. Secara komputasional, kedekatan titik awal dengan solusi optimal secara signifikan mempercepat konvergensi iterasi.

Baris ke-5–7 menghitung jarak Euclidean dari estimasi saat ini ke setiap titik evaluasi menggunakan `np.linalg.norm` dengan parameter `axis=1`, yang mengeksekusi komputasi norm per-baris pada matriks $M \times 5$ secara tervektorisasi tanpa perulangan eksplisit. Baris ke-9–11 mengimplementasikan penanganan singularitas yang diuraikan pada tahapan 2 Algoritma 2: jika jarak estimasi ke salah satu titik jangkar jatuh di bawah $\epsilon = 10^{-6}$, algoritma mengunci (*snap*) estimasi pada koordinat titik tersebut dan menghentikan iterasi. Tanpa mekanisme ini, pembagian oleh jarak yang mendekati nol pada baris ke-13 akan menghasilkan nilai $\tilde{w}_i$ yang sangat besar dan merusak stabilitas numerik pembaruan *fixed-point*.

Baris ke-13–19 merupakan implementasi langsung dari rumusan pembaruan *fixed-point* Weiszfeld pada Persamaan 3-5. Operasi `weighted_distances[:, np.newaxis]` memanfaatkan mekanisme *broadcasting* NumPy untuk memperluas vektor bobot dari dimensi M menjadi $M \times 1$, sehingga perkalian element-wise dengan matriks `points` berukuran $M \times 5$ dieksekusi dalam satu operasi vektorisasi tanpa perulangan per-dimensi. Baris ke-21–22 mengimplementasikan kriteria konvergensi ganda yang diuraikan pada tahapan 4 Algoritma 2, yakni iterasi dihentikan apabila perpindahan spasial antar-iterasi telah di bawah ambang batas ϵ .

Pada level yang lebih tinggi, metode `compute` membungkus iterasi Weiszfeld ke dalam perulangan terhadap setiap kandidat jawaban pekerja. Untuk setiap pekerja a_j , matriks vektor evaluasi $A \in \mathbb{R}^{K \times 5}$ dibentuk dengan menyusun (*stack*) vektor skor 5-dimensi dari seluruh evaluator komite, dan vektor bobot $w \in \mathbb{R}^K$ disiapkan dari skor reputasi historis yang dinormalisasi sesuai Persamaan 3-3. Keluaran iterasi Weiszfeld berupa vektor konsensus 5-dimensi, yang kemudian dijumlahkan seluruh dimensinya menggunakan `np.sum` untuk menghasilkan *robust score* skalar. Sistem secara deterministik memilih jawaban dengan *robust score* tertinggi sebagai *output* final.

4.1.2.3 Implementasi *Exponential Moving Average* (EMA)

Mekanisme pembaruan reputasi berbasis EMA yang dirumuskan pada Persamaan 3-7 dan direpresentasikan pada Algoritma 3 diimplementasikan sebagai modul yang menerima keluaran konsensus dari WGM dan menghasilkan delta reputasi untuk setiap evaluator. Inti logika komputasionalnya direpresentasikan sebagai berikut:

```

1 MAX_5D_DISTANCE = 44.721359549995794 # sqrt(5 * 20^2)
2
3 def update_reputation(self, evaluations, consensus):
4     deltas = {}
5

```

```

6     for evaluation in evaluations:
7         evaluator_id = evaluation.evaluator_id
8         if evaluator_id not in self.reputations:
9             continue
10
11        worker_ids = list(evaluation.scores.keys())
12        num_workers = len(worker_ids)
13        if num_workers == 0:
14            continue
15
16        total_distance = 0.0
17        for i, worker_id in enumerate(worker_ids):
18            evaluator_vector = np.array(
19                evaluation.scores[worker_id]
20            )
21            consensus_vector = consensus.final_score[i]
22            distance = np.linalg.norm(
23                evaluator_vector - consensus_vector
24            )
25            total_distance += distance
26
27        avg_distance = total_distance / num_workers
28        performance = 1.0 - (avg_distance / MAX_5D_DISTANCE)
29        performance = max(0.0, min(1.0, performance))
30
31        old_reputation = self.reputations[evaluator_id]
32        new_reputation = (
33            self.alpha * old_reputation
34            + (1 - self.alpha) * performance
35        )
36        new_reputation = max(0.0, min(1.0, new_reputation))
37
38        self.reputations[evaluator_id] = new_reputation
39        deltas[evaluator_id] = new_reputation - old_reputation
40
41    return deltas

```

Kode 4.4. Pembaruan Reputasi dengan EMA Berbasis Jarak Spasial

Pada Kode 4.4, baris ke-1 mendefinisikan konstanta $d_{max} = \sqrt{5 \times 20^2} \approx 44,72$ yang merepresentasikan jarak Euclidean maksimum absolut dalam ruang metrik evaluasi 5-dimensi (setiap dimensi berkisar antara 0 hingga 20). Nilai ini dihitung secara analitik

di luar *runtime* dan disimpan sebagai konstanta untuk menghindari redundansi komputasi pada setiap pemanggilan fungsi.

Baris ke-17–24 mengimplementasikan tahapan kalkulasi galat spasial rata-rata sesuai tahapan 1 Algoritma 3. Untuk setiap kandidat jawaban yang dievaluasi, vektor skor evaluator ($A_{i,j}$) dikonversi menjadi *array* NumPy, kemudian jarak Euclidean terhadap vektor konsensus final (y_j) dihitung menggunakan `np.linalg.norm`. Akumulasi jarak kemudian dinormalisasi dengan jumlah pekerja (n) untuk menghasilkan $\bar{d}_i$, memastikan bahwa evaluator tidak dipenalti secara tidak proporsional akibat perbedaan beban kerja evaluasi antar-agen.

Baris ke-27–29 mengimplementasikan normalisasi *inverse min-max* dan *bounding* sesuai tahapan 2 Algoritma 3 dan Persamaan 3-6. Fungsi `max(0.0, min(1.0, .))` merupakan operasi *clamp* yang membatasi skor performa pada rentang $[0, 1]$ sebagai langkah *defensive programming* terhadap anomali *floating-point* yang dapat menyebabkan kegagalan pada saat penulisan ke dalam *smart contract*.

Baris ke-31–36 mengimplementasikan pembaruan reputasi EMA sesuai tahapan 3 Algoritma 3 dan Persamaan 3-7. Parameter α (yang dipetakan dari `self.alpha`) merepresentasikan koefisien *smoothing* yang mengendalikan keseimbangan antara memori historis dan responsivitas terhadap sinyal kinerja baru. Hasil pembaruan kembali di-*clamp* pada rentang $[0, 1]$ sebelum disimpan. Baris ke-39 menghitung delta reputasi (ΔT), yang merupakan implementasi dari tahapan 4 Algoritma 3 yang dirancang untuk mengoptimalkan efisiensi integrasi *on-chain* yang mana hanya delta yang dikirimkan ke *smart contract*, bukan keseluruhan skor reputasi, sehingga mereduksi beban komputasi transaksi dan *gas fee*.

4.1.2.4 Orkestrasi Aliran Data Antar-Modul

Ketiga modul *off-chain* di atas tidak beroperasi secara terisolasi, melainkan terorkestrasi dalam sebuah siklus umpan balik tertutup pada setiap ronde konsensus. Aliran data antar-modul berjalan melalui lima tahapan sekuensial: (1) skor reputasi historis dibaca dari *Reputation Store*, (2) modul *Softmax Sharding* menggunakan reputasi tersebut untuk menyeleksi komite berukuran K , (3) evaluator komite menghasilkan matriks skor 5-dimensi, (4) modul WGM mengagregasi matriks evaluasi menggunakan bobot reputasi yang dinormalisasi, dan (5) modul EMA mengukur galat spasial setiap evaluator relatif terhadap keluaran WGM untuk memperbarui skor reputasi. Skor reputasi terbaru ini kemudian mengalir kembali ke modul *Softmax Sharding* dan WGM pada ronde berikutnya, menciptakan mekanisme umpan balik positif yang secara dinamis memperkuat kontribusi agen jujur dan mendegradasi pengaruh agen jahat sepanjang siklus hidup jaringan, sebagaimana telah diuraikan pada arsitektur *feedback loop* dalam Subbab 3.2.4.

Dari perspektif efisiensi komputasi, penggunaan NumPy sebagai fondasi kompu-

tasi pada ketiga modul menghasilkan beberapa keunggulan implementasi yang signifikan. Pertama, operasi vektorisasi seperti `np.linalg.norm`, `np.sum`, dan `np.exp` mengeliminasi kebutuhan perulangan Python eksplisit untuk kalkulasi per-elemen, yang secara kuantitatif mengurangi *overhead* interpreter sebanyak orde magnitudo untuk matriks berukuran $K \times 5$. Kedua, mekanisme *broadcasting* memungkinkan operasi antara vektor dan matriks dengan dimensi yang tidak sama (seperti `weighted_distances[:, np.newaxis]*points`) dieksekusi dalam satu primitif tanpa alokasi memori tambahan untuk perulangan per-dimensi. Ketiga, konstanta seperti `MAX_5D_DISTANCE` dan `epsilon` yang dihitung secara analitik di luar *runtime* menghindari redundansi komputasi pada setiap pemanggilan fungsi dalam perulangan ronde.

4.1.3 Implementasi Lapisan *On-Chain*

Lapisan *on-chain* merealisasikan *immutable ledger* yang telah dirancang pada Subbab 3.2.4 melalui sebuah *smart contract* yang dideploy pada jaringan lokal Anvil, sebuah *testnet* lokal yang disediakan oleh *toolchain* Foundry [42]. Kontrak *smart contract* dikompilasi menggunakan kompilator Solidity versi 0.8.24 dengan *pipeline* optimisasi berbasis IR (*Intermediate Representation*) yang diaktifkan untuk menghasilkan *bytecode* EVM yang efisien. Seluruh artefak kompilasi (*Application Binary Interface/ABI* dan *bytecode*) dihasilkan oleh perintah `forgebuild` dan dikonsumsi oleh klien Python melalui pustaka `web3.py`.

Integrasi antara lapisan *off-chain* dan *on-chain* dijembatani oleh modul `OnchainReputationClient` yang mengimplementasikan arsitektur *write-through cache* sesuai desain pada Subbab 3.2.4. Pada arsitektur ini, seluruh operasi pembacaan reputasi dilayani dari *cache* memori lokal tanpa latensi jaringan, sementara seluruh operasi penulisan secara atomik dikirimkan ke *blockchain* melalui antarmuka JSON-RPC terlebih dahulu sebelum *cache* diperbarui. Koneksi RPC diestablish menggunakan `Web3(HTTPProvider(rpc_url))` yang secara *default* mengarah ke `http://127.0.0.1:8545`, port standar Anvil dengan *chain ID* 31337.

4.1.3.1 Penyimpanan Reputasi pada *Ledger*

Fungsionalitas inti dari *smart contract* `ReputationOracle` adalah menyimpan skor reputasi agen secara permanen dan tidak dapat diubah (*tamper-proof*) di dalam *state blockchain*. Representasi numerik reputasi di *on-chain* menggunakan skala bilangan bulat `uint256` $\in [0, 10\,000]$ yang memetakan secara bijektif ke rentang *floating-point* $[0.0, 1.0]$ di *off-chain*. Pemilihan representasi *fixed-point* ini didasarkan pada keterbatasan EVM yang tidak mendukung operasi *floating-point* secara natif, sehingga seluruh komputasi di *on-chain* harus dieksekusi dalam aritmetika bilangan bulat. Fungsi konversi di sisi Python mengimplementasikan pemetaan $T_{\text{chain}} = \lfloor T_{\text{float}} \times 10\,000 \rfloor$ dan inversenya

$T_{\text{float}} = T_{\text{chain}}/10\,000$, dengan operasi *clamp* $[0.0, 1.0]$ sebelum penskalaan sebagai jaring pengaman terhadap anomali *floating-point*.

Inti logika pembaruan reputasi secara *batch* pada *smart contract* direpresentasikan sebagai berikut:

```
1 function batchUpdateReputations(  
2     string[] calldata ids ,  
3     uint256[] calldata newReps  
4 ) external onlyOwner {  
5     require(  
6         ids.length == newReps.length ,  
7         "Array length mismatch"  
8     );  
9  
10    for (uint256 i = 0; i < ids.length; i++) {  
11        require(  
12            agents[ids[i]].registered ,  
13            "Agent not found"  
14        );  
15        require(  
16            newReps[i] <= 10000 ,  
17            "Reputation out of range"  
18        );  
19  
20        uint256 oldRep = agents[ids[i]].reputation;  
21        agents[ids[i]].reputation = newReps[i];  
22  
23        emit ReputationUpdated(  
24            ids[i] , oldRep , newReps[i]  
25        );  
26    }  
27 }
```

Kode 4.5. Pembaruan Reputasi *Batch* pada *Smart Contract*

Pada Kode 4.5, modifier `onlyOwner` pada baris ke-4 memastikan bahwa hanya alamat yang mendeploy kontrak (*deployer*) yang berwenang mengeksekusi pembaruan reputasi, implementasi dari pola *access control* yang mencegah entitas tak sah memanipulasi *ledger*. Baris ke-5–8 merupakan *sanity check* yang memvalidasi konsistensi dimensi kedua larik masukan, menjamin tidak terjadi *out-of-bounds* akibat ketidakcocokan panjang larik. Baris ke-11–18 memverifikasi bahwa setiap agen telah terregistrasi dan skor baru berada dalam rentang valid $[0, 10\,000]$. Batas atas 10000 ini merupakan

implementasi langsung dari *constraint* $T_i \in [0, 1]$ pada Persamaan 3-7, yang dijalankan secara deterministik oleh EVM sehingga tidak mungkin dilanggar.

Baris ke-20–21 mengimplementasikan atomisitas pembaruan: skor reputasi lama disimpan sementara untuk keperluan *event logging*, kemudian *state* agen diperbarui secara *in-place* pada *storage slot* yang dipetakan oleh mapping (`string⇒AgentRecord`). Baris ke-23–25 memancarkan *event* `ReputationUpdated` yang berisi identitas agen, skor sebelum pembaruan, dan skor sesudah pembaruan. *Event log* ini bersifat *append-only* dan tersimpan secara permanen di dalam *receipt trie* blok, sehingga berfungsi sebagai jejak audit yang tidak dapat dihapus dan dapat diindeks oleh *off-chain indexer* untuk keperluan verifikasi retrospektif. Pendekatan *batch* pada fungsi ini, yang proposes seluruh pembaruan reputasi dalam satu transaksi tunggal, mengimplementasikan desain *batch update* yang telah diuraikan pada Subbab 3.2.4, sebab pengiriman skor satu per satu melalui transaksi terpisah akan menimbulkan *overhead gas* yang secara linier bertambah seiring dengan jumlah anggota komite K .

Di sisi Python, pembaruan reputasi dikirimkan ke *on-chain* melalui pola *sign-then-send* yang memastikan bahwa setiap transaksi ditandatangani secara kriptografis sebelum dikirimkan ke jaringan. Inti logika jembatan RPC direpresentasikan sebagai berikut:

```

1 def update_reputations(self, reputations):
2     ids = list(reputations.keys())
3     reps = [
4         int(max(0.0, min(1.0, v)) * 10_000)
5         for v in reputations.values()
6     ]
7
8     tx = self._contract.functions \
9         .batchUpdateReputations(
10             ids, reps
11         ).build_transaction({
12             "from": self._account.address,
13             "nonce": self._w3.eth \
14                 .get_transaction_count(
15                     self._account.address
16                 ),
17             })
18     signed = self._account.sign_transaction(
19         dict(tx)
20     )
21     tx_hash = self._w3.eth \

```

```

22         . send_raw_transaction (
23             signed . raw_transaction
24         )
25     receipt = self . _w3 . eth \
26         . wait_for_transaction_receipt ( tx_hash )
27
28     self . _cache . update ( reputations )

```

Kode 4.6. Jembatan RPC: Pembaruan Reputasi *Off-Chain* ke *On-Chain*

Pada Kode 4.6, baris ke-2–6 mengimplementasikan konversi skala dari *floating-point* [0.0, 1.0] ke bilangan bulat [0, 10 000] dengan operasi *clamp* ganda sebagai lapisan pertahanan terhadap nilai di luar rentang. Baris ke-8–17 mengonstruksi *raw transaction* menggunakan ABI yang telah dikompilasi oleh Foundry, dengan `build_transaction` mengemas parameter fungsi *smart contract* beserta metadata transaksi (*nonce* dan alamat pengirim). Baris ke-18–20 menandatangani transaksi dengan kunci privat menggunakan pustaka `eth_account`, menghasilkan *signed transaction* yang berisi tanda tangan ECDSA secp256k1. Baris ke-21–26 mengirimkan transaksi bertanda ke jaringan Anvil melalui *endpoint* RPC `eth_sendRawTransaction`, kemudian menunggu konfirmasi secara sinkron melalui `wait_for_transaction_receipt`. Baris ke-28 mengimplementasikan prinsip *write-through cache*: *cache* lokal hanya diperbarui *setelah* transaksi *on-chain* berhasil dikonfirmasi, menjamin konsistensi antara *state off-chain* dan *on-chain*.

4.1.3.2 Verifikasi *Tie-Breaker* Kriptografis *On-Chain*

Ketika algoritma WGM mendeteksi bahwa dua atau lebih jawaban pekerja memiliki *robust score* identik (*tie*), *smart contract* mengimplementasikan mekanisme verifikasi *tie-breaker* yang bersifat deterministik dan tidak dapat dimanipulasi. Arsitektur dua lapisan yang telah dirancang pada Subbab 3.2.4 direalisasikan sebagai fungsi `verifyTieBreaker` yang murni (*pure*), tidak membaca maupun menulis *state*, sehingga dapat dieksekusi tanpa biaya *gas*. Inti logika verifikasi direpresentasikan sebagai berikut:

```

1 function verifyTieBreaker (
2     string [] calldata tiedAnswers ,
3     bytes32 blockHash ,
4     uint256 winningIndex
5 ) external pure returns ( bool ) {
6     require (
7         tiedAnswers . length >= 2 ,
8         "Need at least 2 tied answers"
9     ) ;
10

```

```

11     uint256 maxHash = 0;
12     uint256 trueWinner = 0;
13     for (uint256 i = 0; i < tiedAnswers.length; i++) {
14         bytes32 h = keccak256(
15             abi.encodePacked(
16                 tiedAnswers[i], blockHash
17             )
18         );
19         uint256 hUint = uint256(h);
20         if (hUint > maxHash) {
21             maxHash = hUint;
22             trueWinner = i;
23         }
24     }
25
26     return winningIndex == trueWinner;
27 }

```

Kode 4.7. Verifikasi *Tie-Breaker* Deterministik pada *Smart Contract*

Pada Kode 4.7, baris ke-14–18 mengimplementasikan langsung rumusan *tie-breaker* pada Persamaan 3-8: $H_i = \text{keccak256}(a_i \parallel \mathcal{B})$. Fungsi `abi.encodePacked` melakukan pengodean padat (*tightly packed encoding*) yang mengkonkatenasi teks jawaban a_i dengan *block hash* $\mathcal{B}$ tanpa penambahan *padding*, menghasilkan *preimage* yang unik per kombinasi jawaban-blok. Fungsi `keccak256` kemudian memetakan *preimage* tersebut ke *hash* 256-bit yang secara komputasional tidak dapat diinversi (*preimage-resistant*) atau dijadikan tabrakan (*collision-resistant*) [47]. Baris ke-19 menginterpretasikan keluaran `keccak256` (tipe `bytes32`) sebagai bilangan bulat tak bertanda 256-bit (`uint256`), yang memungkinkan perbandingan ordinal antar-jawaban. Baris ke-20–23 memilih jawaban dengan nilai *hash* terbesar sebagai pemenang deterministik, dan baris ke-26 mengembalikan hasil verifikasi biner yang menegaskan apakah indeks pemenang yang diklaim oleh modul *off-chain* konsisten dengan perhitungan ulang *on-chain*.

Keamanan mekanisme *tie-breaker* ini bertumpu pada dua properti fundamental. Pertama, determinisme: karena `keccak256` adalah fungsi *hash* kriptografis yang bersifat deterministik, seluruh peserta jaringan yang memasukkan himpunan jawaban dan *block hash* yang identik akan menghasilkan pemenang yang sama, tidak ada ruang untuk ambiguitas. Kedua, resistensi manipulasi: *block hash* $\mathcal{B}$ bersifat publik namun tidak dapat diprediksi sebelum blok ditambang (*mined*), sehingga agen manapun tidak dapat menghitung ke depan (*precomputation*) untuk memanipulasi teks jawaban agar menghasilkan H_i yang lebih besar. Ketika *tie-breaker* terjadi dalam ronde konsensus, fungsi `recordRoundWithTieBreaker` dipanggil alih-alih `recordRound`: fungsi ini secara internal

mengeksekusi ulang logika *hashing* yang identik dengan Kode 4.7, menyimpan *record tie-breaker* yang berisi *block hash*, daftar jawaban seri, dan indeks pemenang terverifikasi ke dalam *state on-chain*, serta memancarkan *event TieBreakerResolved* sebagai bukti permanen. Dengan demikian, arsitektur dua lapisan ini menjamin bahwa proses *tie-breaking* tidak hanya deterministik di sisi *off-chain*, tetapi juga terverifikasi secara independen di sisi *on-chain*, sehingga integritasnya tidak dapat dipertanyakan oleh entitas manapun dalam jaringan.

4.1.4 Konfigurasi Hiperparameter Sistem

Perilaku dari arsitektur konsensus yang diusulkan tidak hanya ditentukan oleh struktur algoritmiknya, melainkan secara kritis dikendalikan oleh lima hiperparameter yang menjadi titik kalibrasi antara efisiensi komputasi, ketahanan *Byzantine*, dan *fairness* partisipatif. Setiap nilai yang dipilih bukan merupakan arbitrer numerik, melainkan hasil dari rasionalisasi matematis dan pertimbangan *trade-off* arsitektural yang akan diuraikan pada subbab ini. Ringkasan seluruh hiperparameter beserta peran fungsionalnya disajikan pada Tabel 4.1.

Tabel 4.1. Konfigurasi hiperparameter sistem dan peran fungsionalnya

Hiperparameter	Simbol	Nilai	Peran Fungsional
Ukuran Komite	K	10	Ekuilibrium skalabilitas
Reputasi Awal	T_0	0.5	Fondasi keadilan <i>cold start</i>
Temperatur <i>Softmax</i>	τ	0.3	<i>Trade-off</i> eksploitasi–eksplorasi
Faktor <i>Smoothing</i> EMA	λ	0.3	Keseimbangan memori–responsivitas
Toleransi Konvergensi WGM	ϵ	10^{-6}	Presisi numerik
Batas Iterasi WGM	$I_{\max}$	100	Jaminan terminasi

4.1.4.1 Ukuran Komite ($K = 10$)

Kompleksitas komunikasi *all-to-all* pada ronde konsensus tumbuh secara kuadratik terhadap jumlah peserta: $O(K^2)$ untuk komite berukuran K , dan $O(N^2)$ apabila seluruh populasi N berpartisipasi tanpa penyaringan. Penetapan $K = 10$ merupakan titik ekuilibrium antara dua kekuatan yang saling bertentangan. Dari perspektif *throughput*, pembatasan ini memangkas kompleksitas dari $O(N^2)$ ke $O(K^2)$; pada populasi $N_e = 100$, pengurangan mencapai faktor $(100/10)^2 = 100\times$. Dari perspektif statistik, komite berukuran 10 menyediakan sampel yang cukup untuk meredam variansi alami respons LLM yang jujur [5, 7]: K yang terlalu kecil (misalnya $K = 3$) membiarkan variansi halusinasi mendistorsi konsensus, sementara K yang terlalu besar (misalnya $K = 50$) menghasilkan *diminishing returns* karena konvergensi WGM telah stabil pada ukuran moderat [32].

4.1.4.2 Reputasi Awal ($T_0 = 0,5$)

Dari Persamaan 3-2, terdapat dua batas yang tidak boleh dilanggar untuk T_0 . Pertama, $T_0 \neq 0$: meskipun $\exp(0/\tau) = 1$ secara matematis non-nol, rasio probabilitas terhadap agen bereputasi tinggi $\exp(1/0,3)/\exp(0) \approx 28$ secara efektif mengunci agen baru keluar dari partisipasi, menciptakan oligarki permanen. Kedua, $T_0 \neq 1$: memberikan reputasi maksimal kepada entitas yang belum terbukti aman memberikan bobot WGM (Persamaan 3-3) setara dengan agen jujur yang telah terverifikasi, mengabaikan prinsip meritokrasi. Nilai $T_0 = 0,5$ sebagai titik tengah $[0, 1]$ menjamin probabilitas *Softmax* yang non-nol dan kompetitif ($\exp(0,5/0,3) \approx 5,29$) tanpa memberikan privilese yang tidak berhak, merealisasikan mekanisme penanganan *cold start* pada tahapan 1 Algoritma 1.

4.1.4.3 Temperatur *Softmax* ($\tau = 0,3$)

Parameter τ pada Persamaan 3-2 mendefinisikan sikap sistem terhadap risiko *Byzantine* [37]. Pada rezim ekstrem, $\tau \rightarrow 0$ mengkolapskan distribusi menjadi *argmax* deterministik, sistem kebal terhadap infiltrasi namun menciptakan oligarki statis yang mengunci agen baru. Sebaliknya, $\tau \rightarrow \infty$ meratakan distribusi menjadi hampir seragam ($P(e_i) \approx 1/N_e$), menghilangkan korelasi reputasi–probabilitas sehingga agen jahat masuk komite sebanding dengan tingkat infiltrasi jaringan. Penetapan $\tau = 0,3$ menyeimbangkan kedua rezim: rasio probabilitas agen bereputasi tinggi ($T = 1,0$) terhadap agen baru ($T_0 = 0,5$) adalah $\approx 5,3\times$, cukup curam agar agen terdegradasi ($T = 0,2$) memiliki probabilitas terpilih hanya $\approx 0,07$ relatif terhadap agen terpercaya, namun tidak terlalu curam hingga mengeliminasi jalur masuk agen baru.

4.1.4.4 Faktor *Smoothing* EMA ($\lambda = 0,3$)

Dalam Persamaan 3-7, λ mengendalikan *trade-off* antara memori historis dan responsivitas: λ tinggi menghasilkan inersia yang lambat merespons *oscillation attack*, sementara λ rendah menjadikan sistem rentan terhadap *noise* evaluasi sesaat dari agen jujur. Penetapan $\lambda = 0,3$ memprioritaskan responsivitas ($1 - \lambda = 0,7$, artinya 70% bobot pada kinerja baru): agen jahat dengan $T = 1,0$ yang mulai mengirim evaluasi jahat ($S_t = 0$) turun ke $T = 0,3$ dalam satu ronde dan $T < 0,1$ dalam empat ronde, mengecualikan mereka dari seleksi komite secara efektif. Secara bersamaan, memori 30% memberikan *damping* terhadap variansi natural, mencegah satu evaluasi tidak akurat dari agen jujur mendistorsi reputasi secara dramatis.

4.1.4.5 Toleransi Konvergensi WGM ($\epsilon = 10^{-6}$, $I_{\max} = 100$)

Algoritma Weiszfeld (Algoritma 2) memerlukan dua parameter penghentian. Ambang batas $\epsilon = 10^{-6}$ ditetapkan enam orde magnitudo di bawah skala operasional ruang

$\mathbb{R}^5$ ($[0, 20]$ per dimensi, jarak Euclidean orde 10^0 – 10^1), menjamin konvergensi yang melampaui kebutuhan diskriminasi antar-*robust score*. Nilai ini juga berfungsi sebagai parameter penanganan singularitas: ketika $\|A_{i,j} - y_{j,k}\|_2 < 10^{-6}$, estimasi dikunci pada titik jangkar untuk mencegah pembagian oleh nilai mendekati nol. Batas iterasi $I_{\max} = 100$ menjamin terminasi deterministik.

4.1.5 Dataset Evaluasi dan Skenario Ancaman Byzantine

Subbab ini menguraikan dua komponen fundamental yang membentuk arena uji eksperimen: (1) *dataset* evaluasi yang menentukan domain masalah dan cara akurasi konsensus diukur, serta (2) skenario injeksi ancaman *Byzantine* yang merepresentasikan repertoar serangan yang dilancarkan terhadap arsitektur konsensus.

4.1.5.1 Dataset Evaluasi

Eksperimen simulasi pada penelitian ini menggunakan gabungan (*hybrid*) dari dua *dataset benchmark* yang bersifat *open-source*, yaitu TruthfulQA [15] dan ARC-Easy [16]. Pemilihan kedua *dataset* ini bukan merupakan keputusan arbitrer, melainkan didasarkan pada justifikasi teknis yang berkaitan langsung dengan dua tantangan fundamental yang ingin diuji pada arsitektur WGM.

TruthfulQA dipilih secara spesifik karena *dataset* ini dirancang untuk memancing LLM memproduksi halusinasi berdasarkan miskonsepsi umum manusia [15]. Setiap pertanyaan dalam TruthfulQA dikurasi sedemikian rupa sehingga jawaban yang paling *plausible* bagi manusia justru merupakan jawaban yang salah secara faktual (misalnya, pertanyaan tentang mitos medis atau klaim pseudosains yang tersebar luas). Karakteristik ini menjadikan TruthfulQA sangat relevan untuk menguji apakah arsitektur WGM mampu membedakan kebenaran faktual dari halusinasi yang meyakinkan (*convincing hallucinations*). Dalam konteks konsensus multi-agen, tantangan yang diperoleh dari *dataset* ini adalah: beberapa agen LLM mungkin menghasilkan jawaban yang meyakinkan secara linguistik namun salah secara faktual, sehingga menguji ketahanan agregasi WGM terhadap *spatial clustering* dari evaluasi yang memfavoritkan halusinasi. Varian yang digunakan adalah *multiple-choice* (MC1), dengan setiap pertanyaan memiliki empat opsi jawaban dan hanya satu yang benar.

ARC-Easy (*AI2 Reasoning Challenge*) dipilih untuk menguji keandalan agen dalam melakukan penalaran logis (*logical reasoning*) [16]. Berbeda dengan TruthfulQA yang menguji pengetahuan faktual, ARC-Easy menguji kemampuan penalaran ilmiah bertahap (*step-by-step scientific reasoning*) melalui pertanyaan sains tingkat sekolah dasar. *Dataset* ini membutuhkan inferensi multi-langkah dan penalaran kausal, bukan sekadar pengambilan fakta dari memori parametrik LLM. Dalam konteks konsensus, tantangan dari *dataset* ini adalah: agen LLM yang kurang mampu bernalar mungkin meng-

hasilkan jawaban yang tampak koheren namun mengandung kesalahan logika internal, sehingga menguji sensitivitas sistem evaluasi 5-dimensi terhadap anomali logis yang tidak serta-merta termanifestasi sebagai halusinasi faktual. Varian yang digunakan adalah *ARC-Easy* dengan format pilihan ganda standar, dengan setiap pertanyaan memiliki tiga hingga lima opsi jawaban dengan kunci jawaban referensi yang jelas.

4.1.5.2 Mekanisme Evaluasi dan Penggabungan *Dataset*

Proses evaluasi akurasi konsensus dilakukan melalui mekanisme pencocokan jawaban pilihan ganda. Pada setiap ronde konsensus t , setiap agen pekerja menerima pertanyaan q_t dari *dataset* dan menghasilkan teks jawaban yang memuat opsi pilihan ganda (A, B, C, atau D). Sistem kemudian mengekstraksi huruf pilihan dari keluaran teks LLM menggunakan pola *regex* yang mencakup berbagai format penulisan umum (misalnya, “Answer: A”, “(A)”, atau huruf yang berdiri sendiri). Setelah algoritma WGM menghasilkan jawaban konsensus final, huruf pilihan dari jawaban terpilih dicocokkan dengan kunci jawaban referensi (*ground truth*) pada *dataset*. Akurasi dihitung sebagai persentase ronde yang berhasil dicocokkan dibandingkan dengan total ronde eksperimen.

Kedua *dataset* digabungkan menjadi satu *hybrid dataset* melalui mekanisme alokasi proporsional. Dengan total 50 ronde eksperimen dan dua *dataset*, setiap *dataset* mengontribusikan 25 pertanyaan masing-masing. Proses penggabungan dilakukan dalam tiga tahapan: (1) masing-masing *dataset* dimuat secara independen dari repositori HuggingFace menggunakan fungsi pemuat khusus, (2) pertanyaan dari kedua *dataset* di-konkatenasi menjadi satu daftar terpadu dengan re-indeksasi global ($q_0, q_1, \dots, q_{49}$), dan (3) daftar terpadu tersebut diacak menggunakan *generator* angka acak yang di-*seed* secara deterministik (`random.Random(seed+run_id)` dengan *seed* bawaan = 42). Pengacakan pasca-penggabungan memastikan bahwa pertanyaan dari kedua *dataset* terdistribusi secara merata sepanjang 50 ronde, mencegah bias temporal yang mungkin timbul apabila seluruh pertanyaan dari satu *dataset* dieksekusi secara sekuensial sebelum *dataset* lainnya. Penggunaan *seeded random number generator* menjamin bahwa urutan pertanyaan bersifat reproduksibel, peneliti lain yang menjalankan eksperimen dengan konfigurasi identik akan mendapatkan urutan pertanyaan yang sama, sehingga hasil eksperimen dapat diverifikasi secara independen.

4.1.5.3 Skenario Injeksi Ancaman *Byzantine*

Untuk mengevaluasi ketahanan arsitektur konsensus terhadap berbagai modalitas serangan, empat perilaku *Byzantine* yang merepresentasikan kelas ancaman yang berbeda diprogramkan untuk menginfiltrasi dan merusak sistem. Keempat skenario ini dirancang secara bergradasi dari serangan tanpa strategi hingga serangan kolusi terkoordinasi, sebagaimana telah didefinisikan pada Subbab 3.2.2. Pemilihan agen jahat dari populasi

evaluator dilakukan secara deterministik menggunakan fungsi *seeded random sampling* pada awal eksperimen, sehingga himpunan agen jahat tetap konstan sepanjang seluruh ronde. Ringkasan keempat skenario ancaman beserta karakteristik operasionalnya disajikan pada Tabel 4.2.

Tabel 4.2. Skenario injeksi ancaman *Byzantine* dan karakteristik operasionalnya

Skenario	Strategi
<i>Random Attack</i>	Injeksi noise acak
<i>Inverse Attack</i>	Pembalikan skor kebenaran
<i>Oscillation Attack</i>	Serangan temporal periodik
<i>Amplification Collusion</i>	Kolusi promosi-supresi terkoordinasi

Random Attack Skenario *Random Attack* merepresentasikan kelas ancaman disonan non-fungsional, dengan agen evaluator *Byzantine* menghasilkan vektor evaluasi 5-dimensi yang sepenuhnya acak tanpa mempertimbangkan konteks jawaban pekerja. Inti logika implementasinya direpresentasikan sebagai berikut:

```

1 def apply_random(
2     scores: dict[str, list[float]],
3     rng: _random.Random,
4 ) -> dict[str, list[float]]:
5     """Replace every dimension with uniform random [0, 20]."""
6     return {
7         wid: [rng.uniform(0, 20) for _ in range(5)]
8         for wid in scores
9     }

```

Kode 4.8. Implementasi *Random Attack*: Injeksi Noise Acak

Pada Kode 4.8, setiap dimensi skor untuk seluruh pekerja diinjeksi dengan nilai *uniform random* pada rentang $[0, 20]$ menggunakan `rng.uniform(0, 20)`, yakni `rng` adalah *generator* angka acak yang di-*seed* secara deterministik. Skenario ini mensimulasikan agen yang tidak berfungsi (*non-functional*) atau mengalami kerusakan sistem secara acak, sehingga berfungsi sebagai batas bawah (*lower bound*) pengujian untuk mengukur kemampuan dasar sistem dalam menoleransi *noise* tanpa pola. Karena tidak terdapat korelasi spasial antar-dimensi maupun antar-pekerja, vektor *random* cenderung tersebar secara merata dalam ruang $\mathbb{R}^5$ dan secara alami tertarik ke arah pusat massa (*center of mass*) yang mendekati vektor tengah setelah dinormalisasi WGM, sehingga dampaknya terhadap pergeseran konsensus relatif minimal.

Inverse Attack Skenario *Inverse Attack* merepresentasikan kelas ancaman adversarial terstruktur, dengan agen evaluator *Byzantine* secara sistematis menginversi kriteria evaluasi. Inti logika implementasinya direpresentasikan sebagai berikut:

```

1 def apply_inverse(
2     scores: dict[str, list[float]],
3 ) -> dict[str, list[float]]:
4     """Invert: 20.0 - score per dimension."""
5     return {
6         wid: [20.0 - v for v in vec]
7         for wid, vec in scores.items()
8     }

```

Kode 4.9. Implementasi *Inverse Attack*: Pembalikan Skor Kebenaran

Pada Kode 4.9, operasi $20.0 - v$ pada setiap dimensi v mengimplementasikan fungsi inversi linear yang membalikkan orientasi spasial evaluasi: jawaban berkualitas tinggi yang semestinya menerima skor mendekati 20 justru mendekati 0, dan sebaliknya. Berbeda dengan *Random Attack* yang bersifat stokastik, *Inverse Attack* menghasilkan vektor evaluasi yang memiliki struktur internal konsisten, vektor terinversi tetap mempertahankan korelasi antar-dimensi dari evaluasi *original*, namun dengan orientasi spasial yang sepenuhnya berlawanan. Karakteristik ini menguji kemampuan sistem dalam mendeteksi dan menetralisasi anomali yang memiliki pola terstruktur dan konsisten. Perhatikan bahwa fungsi ini bersifat deterministik dan tidak memerlukan *generator* angka acak, karena transformasi inversi merupakan pemetaan satu-ke-satu yang tidak memiliki variansi stokastik.

Oscillation Attack Skenario *Oscillation Attack* merepresentasikan kelas ancaman yang secara spesifik mengeksploitasi kerentanan temporal pada sistem berbasis reputasi historis. Agen evaluator *Byzantine* berperilaku jujur pada sebagian ronde untuk membangun kepercayaan dan meningkatkan skor reputasi, kemudian secara tiba-tiba meluncurkan manipulasi ekstrem pada ronde-ronde tertentu. Inti logika implementasinya direpresentasikan sebagai berikut:

```

1 def apply_oscillation(
2     scores: dict[str, list[float]],
3     round_num: int,
4     rng: _random.Random,
5     period: int = 20,
6 ) -> dict[str, list[float]]:
7     """Honest for first half of period,
8         random for second half."""

```

```

9     if round_num % period < period // 2:
10         return scores # honest phase
11     return apply_random(scores, rng)

```

Kode 4.10. Implementasi *Oscillation Attack*: Serangan Temporal Periodik

Pada Kode 4.10, baris ke-9–10 mengimplementasikan kondisi fase jujur: ketika $\text{round_num} \% \text{period} < \text{period} // 2$ terpenuhi (setengah pertama dari setiap siklus), vektor evaluasi *original* dikembalikan tanpa modifikasi, memungkinkan agen membangun reputasi secara otentik. Baris ke-11 mengimplementasikan fase jahat: pada setengah kedua siklus, fungsi `apply_random` (Kode 4.8) dipanggil untuk menginjeksi *noise* acak. Dengan periode bawaan $P = 20$, agen berperilaku jujur selama 10 ronde pertama dari setiap siklus (ronde 0–9, 20–29, 40–49) dan menginjeksi *noise* acak selama 10 ronde berikutnya (ronde 10–19, 30–39). Dalam konteks 50 ronde eksperimen, komposisi ini menghasilkan 30 ronde jujur dan 20 ronde jahat.

Skenario ini dirancang untuk menguji apakah mekanisme *Exponential Moving Average* (EMA) dengan $\lambda = 0,3$ mampu merespons degradasi performa dengan cukup cepat sebelum dampak manipulasi terakumulasi. Selama fase jujur, agen jahat secara aktif membangun reputasi. Ketika reputasi telah mencapai level yang memberikan bobot WGM signifikan, fase jahat diluncurkan untuk memaksimalkan dampak pergeseran konsensus. Keberhasilan sistem dalam menetralisasi serangan ini bergantung pada responsivitas mekanisme EMA, nilai λ yang terlalu tinggi (inersia besar) akan memperlambat degradasi reputasi selama fase jahat, memberikan jendela *exploitation* yang lebar bagi penyerang.

Amplification Collusion Skenario *Amplification Collusion* merepresentasikan ancaman paling destruktif dalam konteks arsitektur konsensus berbasis komite. Sekelompok agen evaluator *Byzantine* secara terkoordinasi berkolusi dengan menerapkan strategi promosi-supresi biner. Inti logika implementasinya direpresentasikan sebagai berikut:

```

1 def apply_amplification(
2     scores: dict[str, list[float]],
3     malicious_worker_ids: set[str],
4 ) -> dict[str, list[float]]:
5     """Amplify adversarial workers,
6         suppress honest workers."""
7     result = {}
8     for worker_id, vec in scores.items():
9         if worker_id in malicious_worker_ids:
10             result[worker_id] = [20.0] * 5
11         else:

```

```

12         result[worker_id] = [0.0] * 5
13     return result

```

Kode 4.11. Implementasi *Amplification Collusion*: Kolusi Promosi-Supresi Biner

Pada Kode 4.11, baris ke-8–10 mengimplementasikan strategi promosi: untuk setiap pekerja *Byzantine* ($j \in \mathcal{M}_w$), seluruh dimensi skor disaturasi ke nilai maksimal 20,0, menghasilkan vektor (20, 20, 20, 20, 20) yang merepresentasikan evaluasi sempurna. Baris ke-12 mengimplementasikan strategi supresi: untuk setiap pekerja jujur ($j \notin \mathcal{M}_w$), seluruh dimensi dinolkan, menghasilkan vektor (0, 0, 0, 0, 0) yang merepresentasikan evaluasi terendah mutlak. Fungsi ini bersifat deterministik dan tidak memerlukan *generator* angka acak, karena strategi kolusi merupakan pemetaan biner yang tidak memiliki variansi stokastik.

Evaluasi kolusi ini menghasilkan dua vektor kutub ekstrem dalam ruang $\mathbb{R}^5$ yang bertujuan menciptakan *spatial gravity well*, menarik konsensus ke arah jawaban *Byzantine worker* melalui akumulasi bobot spasial terkoordinasi. Strategi ini merupakan manifestasi langsung dari model ancaman *collusion attack* yang telah didefinisikan pada Subbab 3.2.2. Berbeda dengan ketiga skenario sebelumnya yang hanya menginfeksi lapisan evaluator, *Amplification Collusion* menginfeksi kedua lapisan secara simultan, pekerja *Byzantine* menghasilkan jawaban yang salah secara deliberatif sementara evaluator *Byzantine* secara kolektif memberikan skor sempurna pada jawaban jahat dan skor nol pada jawaban jujur, menciptakan tekanan spasial terkoordinasi yang paling destruktif terhadap mekanisme agregasi WGM.

4.1.6 Metodologi Komparasi Baseline dan *Passive Shadow Tracking*

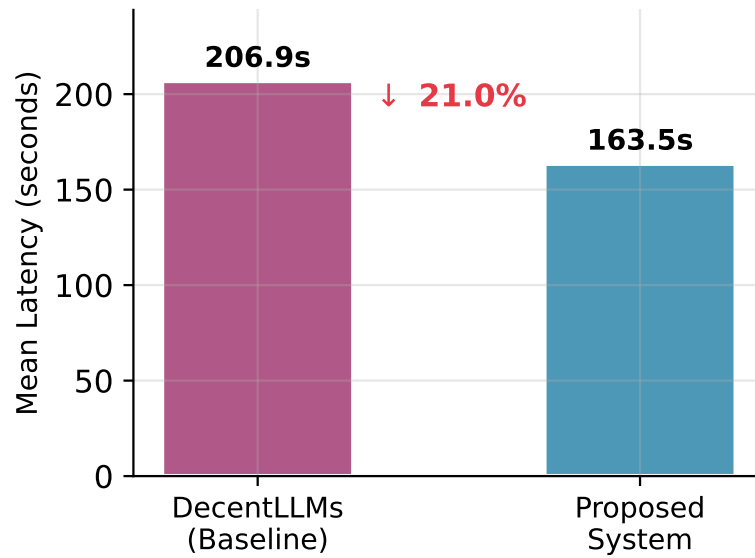
Untuk mengukur keunggulan sistem secara objektif, arsitektur yang diusulkan dievaluasi secara *head-to-head* melawan protokol *leaderless* SOTA saat ini, yaitu DecentLLMs [11]. Guna menjamin validitas eksperimen (*fair benchmarking*), pengujian *baseline* dieksekusi menggunakan arena *dataset* dan batas toleransi konvergensi numerik ($\epsilon = 10^{-6}$) yang identik dengan sistem usulan. Perbedaan pengujian secara ketat diisolasi pada dua variabel arsitektural: *baseline* beroperasi dengan partisipasi komite penuh ($K = N$) tanpa mekanisme *sharding*, dan mengandalkan agregasi *Geometric Median* murni yang bersifat egaliter, dengan seluruh agen memiliki bobot suara statis yang sama rata ($w_i = 1/N$).

Lebih lanjut, untuk memungkinkan perbandingan matematis langsung terhadap batas toleransi keamanan internal antara dua arsitektur yang berbeda ini, metodologi *Passive Shadow Tracking* diimplementasikan selama evaluasi *baseline* berjalan. Dalam skenario ini, mesin *Exponential Moving Average* (EMA) yang diusulkan dibiarkan berjalan sebagai pengamat pasif di latar belakang (*isolated background observer*). Mesin penga-

mat ini dikonfigurasi dengan hiperparameter yang persis sama dengan sistem utama, yaitu reputasi awal $T_0 = 0,5$ dan faktor *smoothing* $\lambda = 0,3$. Pelacakan pasif ini murni berfungsi sebagai metrik diagnostik independen dan sama sekali tidak mengintervensi agregasi spasial, seleksi evaluator, maupun hasil konsensus final dari *baseline*. Dengan menjalankan metrik bayangan ini, sistem dapat mengekstraksi nilai laten dari *Honest Weight Fraction* (Λ) milik *baseline* secara adil. Pendekatan ini memungkinkan kita untuk mengkuantifikasi degradasi struktural tersembunyi pada margin keamanan *baseline* seiring meningkatnya skala ancaman *Byzantine*, tanpa sedikit pun melanggar mekanika operasional *stateless*-nya.

4.2 Analisis Skalabilitas dan Efisiensi Komunikasi

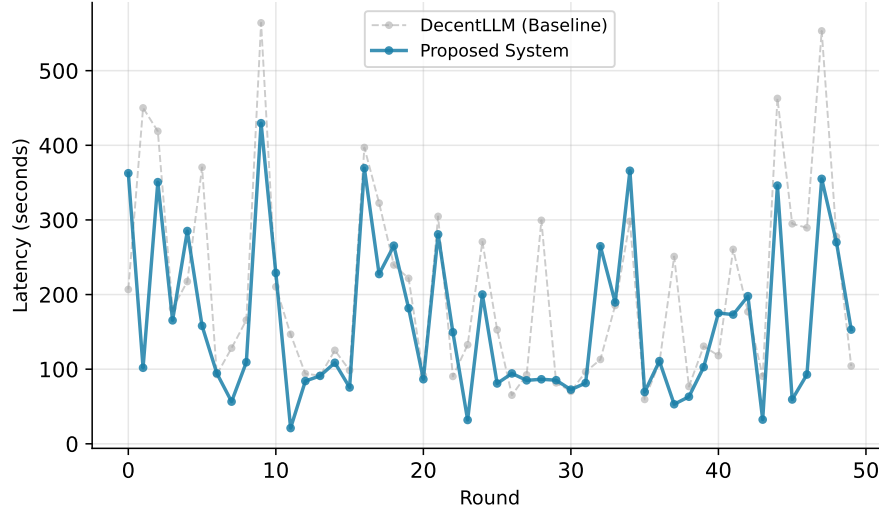
Subbab ini menganalisis keunggulan struktural arsitektur yang diusulkan terhadap *baseline* DecentLLMs [11] dari dimensi skalabilitas komunikasi, meliputi pemangkasan latensi rata-rata, peredaman *jitter*, serta reduksi persentil tinggi dan *worst-case peak*.



Gambar 4.9. Perbandingan waktu eksekusi rata-rata per ronde: *baseline* DecentLLMs ($\bar{L} = 206,9$ detik) vs sistem yang diusulkan ($\bar{L} = 163,5$ detik), penurunan 21,0%.

Kelemahan fundamental arsitektur DecentLLMs terletak pada kewajiban partisipasi penuh tanpa mekanisme *sharding*: seluruh populasi N_e evaluator berpartisipasi dalam setiap ronde, memicu pola komunikasi *all-to-all* dengan kompleksitas $O(N^2)$. Pada eksperimen dengan $N_e = N_w = 15$, rata-rata latensi *baseline* mencapai 206,9 detik per ronde. Mekanisme *Softmax Sharding* (Subbab 3.2.4.2) memecahkan hambatan ini dengan mengunci ukuran komite pada $K = 10$, mengubah kompleksitas komunikasi dari $O(N^2)$ menjadi $O(K^2)$. Reduksi ini bersifat konstan terlepas dari pertumbuhan populasi: pada $N_e = 100$, pengurangan faktor komunikasi mencapai $(100/10)^2 = 100\times$, menjamin skalabilitas jangka panjang.

Dampak kuantitatif pemangkasan kompleksitas ini divisualisasikan pada Gambar 4.9. Komite $K = 10$ mereduksi interaksi evaluasi dari $N_e \times N_w = 225$ menjadi $K \times N_w = 150$, memangkas rata-rata latensi menjadi 163,5 detik (penurunan 21,0%). Penurunan ini bukan semata reduksi beban komputasi, melainkan konsekuensi arsitektural dari eliminasi *all-to-all broadcast*: lebih sedikit evaluator berpartisipasi, sehingga variabilitas latensi API individual memiliki peluang lebih kecil untuk menghasilkan *tail latency* yang mendominasi durasi ronde.



Gambar 4.10. Timeline latensi per ronde: garis sistem usulan menunjukkan profil yang lebih teredam dengan puncak yang tidak se-ekstrem *baseline*.

Rata-rata latensi saja tidak menggambarkan keandalan operasional oracle secara utuh. Panggilan API LLM melalui OpenRouter secara alami sangat fluktuatif, fluktuasi ini termanifestasi sebagai *jitter* (σ), yang pada *oracle blockchain* berimplikasi langsung pada keandalan SLA. Gambar 4.10 menunjukkan profil temporal latensi kedua sistem sepanjang 50 ronde: garis sistem yang diusulkan lebih teredam, dengan *spike* yang tidak se-ekstrem *baseline*. Arsitektur komite kecil bertindak sebagai *shock absorber*: dengan hanya $K = 10$ evaluator, probabilitas *slow-responders* mendominasi ronde berkurang, dan *Softmax Sharding* cenderung menyeleksi evaluator bereputasi tinggi yang memiliki keandalan respons lebih baik.

Tabel 4.3. Perbandingan metrik latensi antara *baseline* DecentLLMs dan sistem yang diusulkan

Metrik	DecentLLMs (<i>Baseline</i>)	Sistem Usulan	Perbandingan
Rata-Rata Latensi ($\bar{L}$)	206,9 detik	163,5 detik	-21,0%
Persentil ke-95 (p_{95})	457,1 detik	364,4 detik	-20,3%
<i>Worst-Case Peak</i> ($L_{\max}$)	564,1 detik	429,6 detik	-23,8%
<i>Latency Jitter</i> (σ)	130,2 detik	108,4 detik	-16,7%

Kuantifikasi formal disajikan pada Tabel 4.3. *Jitter* sistem yang diusulkan ($\sigma = 108,4$ detik) lebih rendah 16,7% dibandingkan *baseline* ($\sigma = 130,2$ detik), mengonfirmasi efek peredaman arsitektur komite kecil. Metrik yang paling krusial bagi *oracle blockchain* adalah persentil tinggi dan *worst-case*: oracle yang terlambat memicu *time-out* transaksi, mengakibatkan kegagalan sistem dan kerugian *gas fee*. Persentil ke-95 *baseline* mencapai 457,1 detik (5% ronde melebihi 7,5 menit), jauh melampaui ambang operasional oracle. Sistem yang diusulkan memangkas p_{95} sebesar 20,3% menjadi 364,4 detik. Pukulan paling telak terletak pada *worst-case peak*: *baseline* memerlukan hingga 564,1 detik (≈ 10 menit), yang secara fundamental tidak layak untuk oracle *smart contract*. Sistem yang diusulkan memangkas *peak* sebesar 23,8% menjadi 429,6 detik, mempersempit jendela kerentanan *timeout* dan mengamankan SLA. Penurunan *peak* (23,8%) yang melebihi penurunan rata-rata (21,0%) mengindikasikan bahwa komite kecil secara disproportional memangkas *tail* distribusi, zona paling destruktif bagi keandalan *oracle*, karena *Softmax Sharding* menyeleksi evaluator bereputasi tinggi yang memiliki keandalan respons lebih baik.

Sebagai catatan, perlu ditekankan kembali bahwa seluruh nilai latensi absolut yang tercatat dalam eksperimen ini (seperti *peak* 429,6 detik) mengandung *artificial queuing latency*. Durasi yang cukup tinggi ini adalah konsekuensi langsung dari penerapan manajemen konkurensi `asyncio.Semaphore(30)` yang digunakan untuk menghindari pemblokiran API (HTTP 429 *Too Many Requests*), sebagaimana telah diuraikan pada batasan purwarupa di Subbab 3.4. Karena konstrain antrean ini diaplikasikan secara setara pada kedua sistem, perbandingan metrik relatifnya, yakni pemangkasan *peak* sebesar 23,8% dan reduksi rata-rata 21,0%, tetap valid dan secara empiris membuktikan kontribusi dari *Softmax Sharding*.

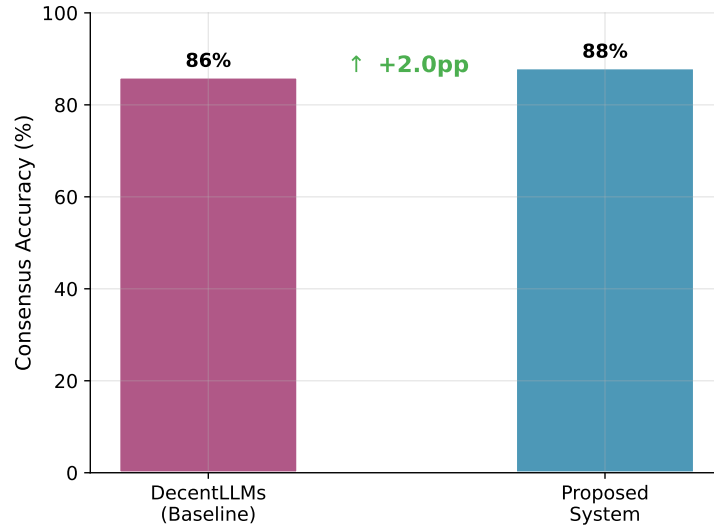
4.3 Evaluasi Ketahanan *Byzantine* dan *Meritocratic Safety Margin*

Subbab ini mengevaluasi ketahanan arsitektur yang diusulkan menghadapi ancaman *Byzantine* melalui tiga dimensi berurutan: keunggulan pada kondisi normal, pemulihan dinamis pada batas ekstrem, dan ilusi ketahanan pasif *baseline*. Seluruh eksperimen menggunakan metodologi komparasi dan *Passive Shadow Tracking* (Subbab 4.1.6).

4.3.1 Keunggulan Meritokrasi pada Lingkungan Normal

Sebelum menguji ketahanan terhadap serangan, perlu dibuktikan bahwa pemboitan reputasi unggul bahkan pada kondisi tanpa ancaman ($f = 0$). Tanpa agen jahat, seluruh evaluator bersifat jujur namun tidak identik, masing-masing memiliki tingkat halusinasi dan variansi evaluasi yang berbeda. Arsitektur *baseline* DecentLLMs [11] yang egaliter memberikan bobot statis $w_i = 1/N$ tanpa memandang konsistensi historis, sehingga menderita *evaluator reliability problem*: evaluator jujur yang bervariansi tinggi

menerima bobot spasial setara dengan evaluator yang konsisten, dan *noise* dari agen tidak stabil secara halus menarik konsensus menjauhi kebenaran.



Gambar 4.11. Perbandingan akurasi konsensus pada kondisi tanpa serangan ($f = 0$): *baseline* DecentLLMs (86%) vs sistem yang diusulkan (88%), peningkatan +2,0 poin persentase.

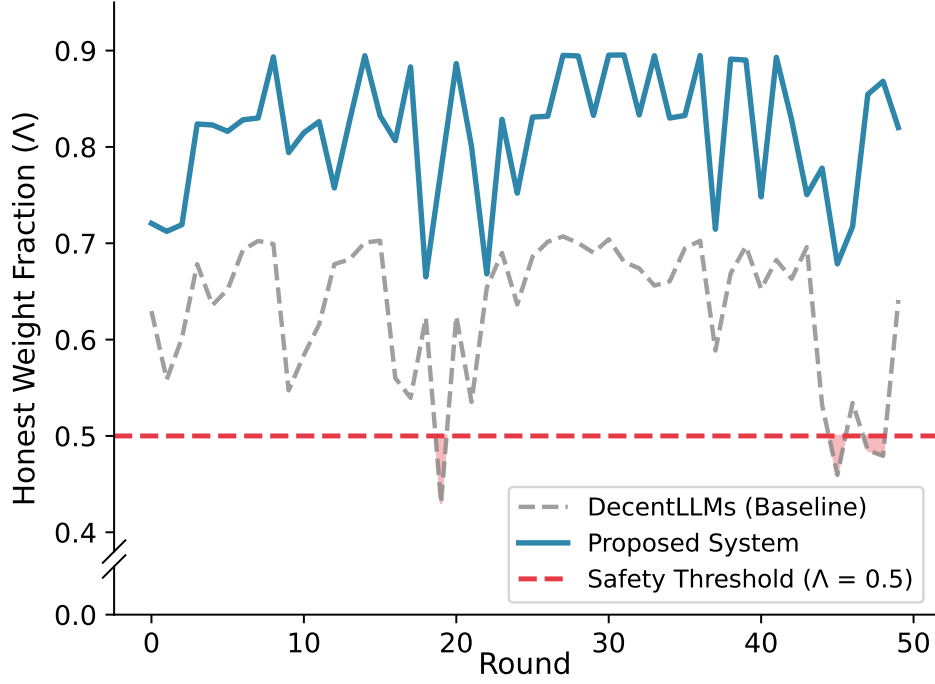
Arsitektur yang diusulkan menyelesaikan masalah ini melalui pembobotan dinamis WGM (Persamaan 3-3). Mesin EMA secara kontinu melacak jarak spasial rata-rata setiap evaluator relatif terhadap konsensus (Subbab 4.1.2), mendegradasi reputasi agen bervariasi tinggi dan mengamplifikasi kontribusi agen yang konsisten. Seperti divisualisasikan pada Gambar 4.11, sistem yang diusulkan mencapai akurasi 88%, meningkat +2,0 poin persentase dari *baseline* yang meraih 86%. Meskipun tampak moderat, peningkatan ini membuktikan bahwa pembobotan reputasi bukan sekadar mekanisme pertahanan, melainkan peningkatan kualitas intrinsik: bahkan tanpa agen jahat, meritokrasi sudah menghasilkan keputusan yang lebih akurat daripada egalitarianisme.

4.3.2 Pemulihan Dinamis pada Batas Ekstrem

Evaluasi kini beralih ke batas toleransi absolut. Seperti dirumuskan pada Subbab 3.2.2, konsensus WGM terjamin aman selama $\Lambda \geq 0,5$ (Persamaan 3-1) [17]. Namun, bertahan tepat di atas $\Lambda = 0,5$ hanyalah syarat kelangsungan matematis, bukan jaminan operasional. Pada jaringan LLM multi-agen, agen jujur secara inheren bervariasi dan sesekali berhalusinasi. Jika sistem beroperasi terlalu dekat batas 0,5, kluster halusinasi natural dapat menurunkan Λ efektif di bawah ambang batas, menyerahkan kendali konsensus kepada penyerang. Mesin EMA menyelesaikan masalah ini dengan secara proaktif mendegradasi bobot agen bervariasi tinggi, menciptakan selisih antara Λ aktual dan batas kritis 0,5 yang disebut sebagai *Meritocratic Safety Margin*.

Untuk mengobservasi penciptaan margin ini secara fisik, kedua sistem dievalu-

asi di bawah skenario paling destruktif: *Amplification Collusion* pada $f = 0,5$ (Subbab 4.1.5), dengan separuh populasi secara terkoordinasi meluncurkan strategi promosi-supresi biner. Evolusi Λ sepanjang 50 ronde divisualisasikan pada Gambar 4.12.



Gambar 4.12. Evolusi Λ selama 50 ronde di bawah *Amplification Collusion* $f = 0,5$: *baseline* menembus batas kritis $\Lambda = 0,5$, sistem usulan membangun *Meritocratic Safety Margin* $\Lambda_c = 0,815$.

Seperti divisualisasikan pada Gambar 4.12, *baseline* tanpa pelacakan aktif tidak memiliki margin keamanan. Dengan 50% agen jahat, Λ *baseline* berfluktuasi di sekitar batas 0,5 dan secara fatal menembus ke bawah ambang kritis pada beberapa ronde terpisah. Setiap pelanggaran $\Lambda < 0,5$ berarti vektor *Byzantine* menguasai tarikan gravitasi spasial mayoritas, membajak konsensus. Akibat akumulasi pelanggaran ini, akurasi *baseline* terdegradasi berat menjadi 70%.

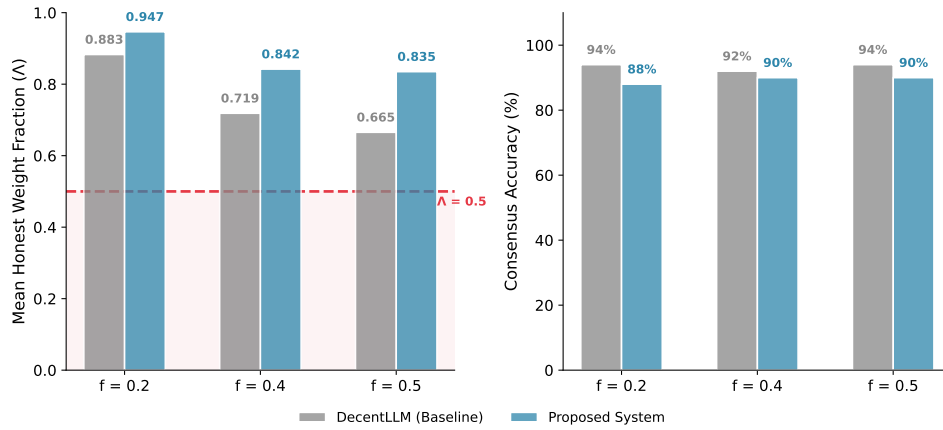
Sebaliknya, arsitektur yang diusulkan menunjukkan *dynamic healing* yang mendalam. Pada Ronde 1, kerentanan *cold start* ($T_0 = 0,5$) menempatkan sistem sementara pada batas $\Lambda = 0,5$. Namun pada ronde berikutnya, mesin EMA mendeteksi jarak *Euclidean* ekstrem dari vektor amplifikasi *Byzantine*, lalu secara sistematis mendegradasi skor reputasi penyerang melalui Persamaan 3-7 dengan $\lambda = 0,3$. Dampaknya menyebar ke dua lapisan pertahanan: (1) pembobotan WGM (Persamaan 3-3) secara progresif mengurangi pengaruh vektor *Byzantine* dalam agregasi, dan (2) *Softmax Sharding* (Subbab 3.2.4.2) secara agresif menurunkan probabilitas terpilihnya penyerang ke dalam komite. Akibatnya, Λ didorong konsisten ke atas. Sepanjang 50 ronde, sistem mencapai rata-rata $\Lambda_c = 0,815$ (yakni $\Lambda_c = \frac{1}{T} \sum_{t=1}^T \Lambda_t$), menciptakan *Meritocratic Safety Margin* sebesar 0,315 yang menyerap dampak serangan 50% dan variansi natural agen jujur. De-

ngan margin ini, akurasi dikunci pada 76%, mengungguli *baseline* yang hanya 70% pada kondisi adversarial identik.

4.3.3 Ilusi Ketahanan Pasif

Bagian sebelumnya membuktikan ketahanan pada batas $f = 0,5$ *Amplification Collusion*. Namun, pada *Random Attack*, *baseline* masih mencatat akurasi 94% di $f = 0,5$, yang seolah-olah mereduksi urgensi arsitektur meritokrasi. Subbab ini membongkar ilusi tersebut dengan menganalisis hubungan antara Λ_c dan keandalan *oracle*.

Kedua arsitektur dievaluasi pada progresi *Random Attack* dengan eskalasi $f = 0,2; 0,4; 0,5$. Grafik *dual-panel* yang membandingkan Λ_c terhadap akurasi divisualisasikan pada Gambar 4.13.



Gambar 4.13. *Dual-panel* Λ_c vs akurasi pada eskalasi *Random Attack* ($f = 0,2; 0,4; 0,5$): *baseline* akurat namun Λ_c merosot, sistem usulan mempertahankan $\Lambda_c = 0,835$ pada $f = 0,5$.

Seperti divisualisasikan pada Gambar 4.13, *baseline* memang mempertahankan akurasi tinggi terhadap *noise* acak (94% pada $f = 0,5$). Namun, panel Λ_c membongkar bahwa metrik ini menyesatkan. Akurasi tersebut dicapai melalui *brute-force*: seluruh populasi N berpartisipasi dalam komunikasi $O(N^2)$ yang tidak berkelanjutan (Subbab 4.2). Lebih krusial, sementara garis akurasi *baseline* tetap tinggi, Λ_c pasifnya mengalami penurunan parah seiring eskalasi ancaman: dari 0,883 pada $f = 0,2$, turun ke 0,719 pada $f = 0,4$, dan anjlok ke 0,665 pada $f = 0,5$, mendekati batas kritis dengan sangat berbahaya. Pola ini konsisten dengan $\Lambda_c \approx 1 - f$: tanpa pembobotan reputasi, fraksi bobot jujur pasif sekadar bercermin pada fraksi populasi jujur, tanpa mekanisme aktif mendorong Λ_c ke atas. Sebagaimana dibuktikan pada Subbab 4.3, margin pasif yang meluruh ini membuat *baseline* sepenuhnya tak berdaya begitu vektor *Byzantine* beralih dari *noise* acak ke kolusi terkoordinasi.

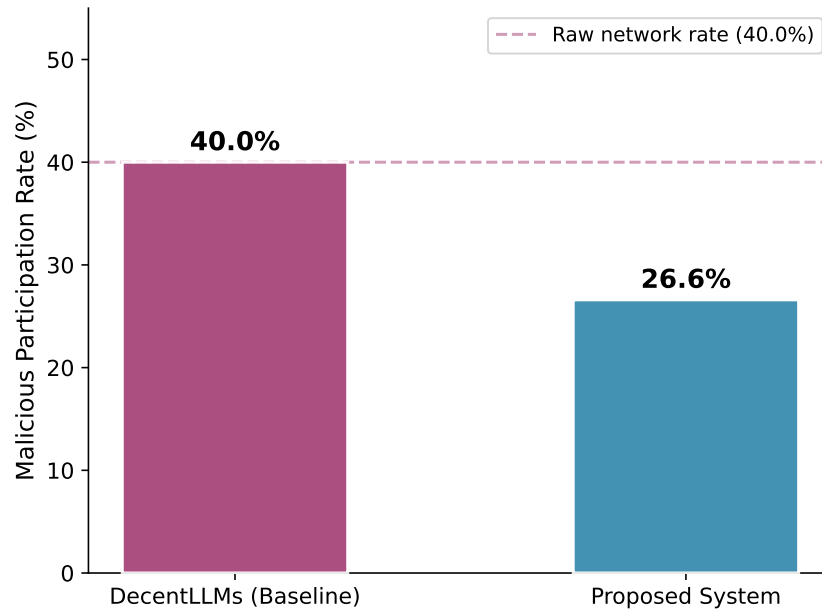
Sebaliknya, arsitektur yang diusulkan secara deliberatif menukar sebagian kecil akurasi puncak demi skalabilitas dan pertahanan proaktif. Melalui *Softmax Sha-*

rding, evaluasi dibatasi pada komite berukuran K dengan kompleksitas konstan $O(K^2)$. Ketergantungan pada subset yang lebih kecil memperkenankan variansi natural LLM, menghasilkan akurasi 90% pada $f = 0,5$. Namun sebagai gantinya, mesin EMA secara aktif mengidentifikasi dan menekan agen jahat bervariansi tinggi, mempertahankan $\Lambda_c = 0,835$ pada $f = 0,5$. Selisih Λ_c antara arsitektur yang diusulkan (0,835) dan *baseline* (0,665) mencapai 0,170, menegaskan bahwa meritokrasi secara struktural menghasilkan margin pertahanan yang jauh lebih tebal.

Progresi empiris ini membuktikan bahwa protokol egaliter yang memaksakan *brute-force* mungkin bertahan pasif terhadap *noise* tidak terkoordinasi, namun tidak dapat diskalakan dengan aman. Arsitektur yang diusulkan mempertahankan jaringan terhadap variansi tidak sengaja maupun serangan terkoordinasi dengan meninggalkan *headcount* menjadi *Meritocratic Safety Margin* yang terjamin secara matematis.

4.4 Dinamika Pelacakan Reputasi dan Penalti *Stateful*

Subbab-subbab sebelumnya telah membuktikan keunggulan arsitektur yang diusulkan pada level agregasi makro (Λ_c , akurasi, latensi). Subbab ini membongkar mekanisme pertahanan yang merealisasikan keunggulan tersebut ronde per ronde, dari reduksi ancaman sebelum evaluasi dimulai, penalti instan terhadap perilaku adversarial, hingga penghancuran eksploitasi temporal.

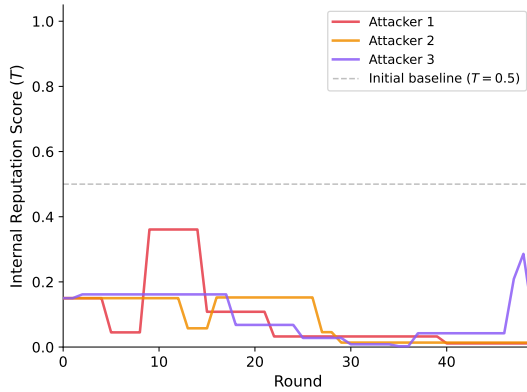


Gambar 4.14. Persentase partisipasi agen jahat dalam komite: *baseline* 40,0% vs *Softmax Sharding* 26,6%.

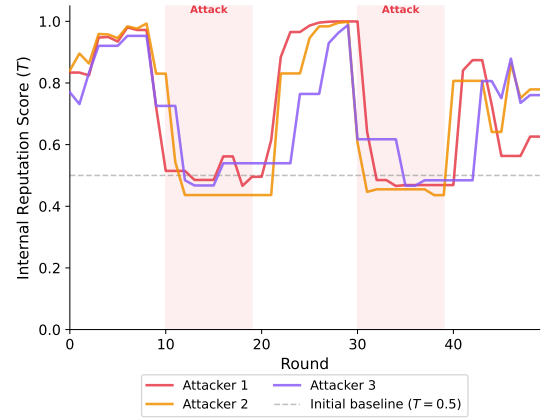
Arsitektur yang diusulkan tidak hanya bertahan secara pasif di tingkat agregasi WGM, melainkan secara fisik mereduksi ruang gerak penyerang sebelum evaluasi dimulai. Mekanisme *Softmax Sharding* yang dikendalikan reputasi (Subbab 3.2.4.2) berfungsi

sebagai gerbang penyaring: agen bereputasi rendah secara disproportional dikeluarkan dari komite. Gambar 4.14 mengkuantifikasi dampak penyaringan ini pada skenario *Amplification Collusion* $f = 0,4$. Berdasarkan ekstraksi metrik melalui *passive shadow tracking* (Subbab 4.1.6), arsitektur *baseline* DecentLLMs [11] membiarkan seluruh agen jahat berpartisipasi penuh, menghasilkan laju partisipasi jahat sebesar 40,0%, persis bercermin pada laju mentah infiltrasi jaringan. Konsekuensinya langsung: Λ pasif anjlok mengikuti $\Lambda \approx 1 - f = 0,6$, sebagaimana dibuktikan pada ilusi ketahanan pasif (Subbab 4.3).

Sebaliknya, arsitektur yang diusulkan memangkas laju partisipasi jahat menjadi hanya 26,6%, tereduksi 13,4 poin persentase. Mekanismenya: setiap kali EMA mendegradasi reputasi agen jahat, probabilitas *Softmax*-nya ($P(e_i) \propto \exp(T_i/\tau)$, Persamaan 3-2) menurun eksponensial, mengurangi kehadiran mereka dalam komite. Pada $\tau = 0,3$, agen dengan $T = 0,2$ memiliki rasio probabilitas terpilih hanya $\approx 0,07$ relatif terhadap agen terpercaya ($T = 1,0$). Reduksi dari 40,0% ke 26,6% ini merupakan reduksi ancaman sebelum evaluasi dimulai: sebelum evaluasi dimulai, 33,5% agen jahat yang seharusnya masuk komite telah dikeluarkan, sehingga tekanan adversarial yang sampai ke lapisan WGM berkurang signifikan dan *Meritocratic Safety Margin* semakin tebal.



Gambar 4.15. Evolusi reputasi agen yang melakukan *Inverse Attack*.



Gambar 4.16. Evolusi reputasi agen yang melakukan *Oscillation Attack*.

Reduksi ancaman sebelum evaluasi dimulai merupakan pertahanan preventif, namun sebagian agen jahat tetap dapat menembus seleksi komite. Lapisan pertahanan berikutnya adalah eksekusi penalti instan oleh mesin EMA. Gambar 4.15 membedah evolusi reputasi agen pelaku serangan *Inverse* (Kode 4.9) sepanjang 50 ronde. Serangan ini membalikkan skor kebenaran melalui operasi $20 - v$ pada setiap dimensi, menghasilkan vektor yang berkorelasi negatif sempurna terhadap konsensus dan menempatkan penyerang pada jarak Euclidean ekstrem dalam $\mathbb{R}^5$.

Seperti divisualisasikan, garis reputasi penyerang anjlok mendekati 0 di ronde-ronde awal. Mekanismenya: vektor terinversi menghasilkan $\bar{d}_i$ sangat tinggi, sehingga

skor performa $S_i = 1 - \bar{d}_i/d_{\max}$ (Persamaan 3-6) mendekati 0. Pembaruan EMA dengan $\lambda = 0,3$ mendegradasi reputasi: $T' = 0,3 \times 0,5 + 0,7 \times S_i \approx 0,15$ pada ronde pertama, dan terus merosot hingga mendekati 0. Sistem beroperasi secara *stateful* dengan penalti absolut: begitu reputasi anjlok, probabilitas *Softmax* runtuh eksponensial, mencabut hak suara penyerang dari seleksi komite.

Penalti instan efektif terhadap serangan *inverse*, namun tantangan yang lebih berat muncul ketika agen berpura-pura jujur untuk mengumpulkan reputasi terlebih dahulu sebelum mengkhianati sistem. Serangan *Oscillation* (Kode 4.10) dirancang untuk mengeksploitasi kerentanan temporal ini. Gambar 4.16 membedah evolusi reputasinya dan membuktikan bahwa mesin EMA menghancurkan strategi *stealth* ini secara fundamental.

Evolusi reputasi penyerang menunjukkan fenomena *bouncing*: reputasi naik gradual selama fase jujur (ronde 0–9, 20–29, 40–49), mencapai puncak, lalu dihancurkan seketika saat fase khianat dimulai (ronde 10, 30). Selama fase jujur, agen memang berhasil membangun reputasi karena vektor *original* menghasilkan jarak Euclidean rendah ($S_i \approx 1$). Namun, begitu fase jahat diluncurkan, transisi ke *noise* acak menghasilkan lompatan jarak masif ($S_i \approx 0$).

Sensitivitas $\lambda = 0,3$ memastikan degradasi bersifat destruktif, bukan gradual. Dengan $1 - \lambda = 0,7$, reputasi puncak $T = 0,8$ turun ke $T = 0,3 \times 0,8 + 0,7 \times 0 = 0,24$ dalam satu ronde, lalu $T = 0,3 \times 0,24 + 0,7 \times 0 = 0,072$ pada ronde kedua, mengeliminasi seluruh keuntungan fase jujur. Penyerang dipaksa masuk ke fase pemulihan yang sangat panjang, selama mana bobot WGM dan probabilitas *Softmax* mereka praktis nol, sehingga mereka tidak pernah dapat mempertahankan bobot yang cukup untuk membajak konsensus. Jaringan tidak memiliki *grace period*: pengkhianatan, kapan pun dilancarkan, langsung dipenalti tanpa pengecualian.

4.5 Limitasi dan *Trade-Off* Sistem

Meskipun arsitektur yang diusulkan secara empiris terbukti berhasil menetralisasi berbagai vektor serangan dan memecahkan hambatan skalabilitas komunikasi, purwarupa sistem ini memiliki beberapa keterbatasan struktural dan *trade-off* arsitektural. Sebagai bentuk evaluasi sistem yang komprehensif, batasan-batasan tersebut diidentifikasi sebagai berikut:

1. ***Trade-Off* Akurasi dan Skalabilitas.** Dari hasil eksperimen, sistem yang diusulkan mengalami penurunan akurasi dari 94% menjadi 90% pada tipe serangan *random*. Penurunan ini merupakan *trade-off* dari implementasi *Softmax Sharding* terhadap akurasi puncak sistem.
2. **Kerentanan Fase Inisialisasi.** Sistem memiliki masa kritis (*burn-in period*) pada

ronde-ronde awal. Ketika seluruh agen diinisialisasi dengan skor *default* $T = 0,5$, probabilitas seleksi komite bersifat egaliter, memaksa margin keamanan sistem beroperasi sangat dekat dengan ambang batas kritis $\Lambda = 0,5$.

3. **Ancaman Bobot Inisial Absolut.** Sistem memiliki kerentanan teoretis terhadap invasi Sybil secara masif di fase awal. Jika agen penyerang yang sangat terkoordinasi berhasil menguasai $> 50\%$ total populasi jaringan, secara matematis mereka dapat langsung membajak ekuilibrium WGM sejak ronde pertama.
4. **Dilema Sensitivitas *Softmax*.** Hiperparameter $\tau = 0,3$ menciptakan keseimbangan yang kaku antara keamanan dan keadilan. Nilai τ yang lebih rendah akan menciptakan oligarki eksklusif yang mengunci agen baru dari partisipasi, sementara nilai yang lebih tinggi berisiko merusak pertahanan dengan menarik terlalu banyak agen yang belum terverifikasi ke dalam komite juri.
5. **Keterbatasan Konkurensi Purwarupa.** Eksperimen dalam penelitian ini tidak sepenuhnya merepresentasikan sistem terdesentralisasi dengan perangkat keras yang terpisah dan berjalan secara paralel. Seluruh agen dieksekusi pada satu mesin VPS dan mengakses LLM melalui *gateway* API terpusat dengan batasan *rate-limit* (semi-paralel), sehingga metrik latensi yang dilaporkan tidak merefleksikan performa intrinsik arsitektur pada infrastruktur *Peer-to-Peer* (P2P) murni.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Penelitian ini telah berhasil merancang, mengimplementasikan, dan mengevaluasi sebuah kerangka kerja konsensus meritokratis untuk *oracle blockchain* berbasis LLM yang mengintegrasikan tiga pilar arsitektural: *Softmax Sharding*, *Weighted Geometric Median* (WGM), dan *Exponential Moving Average* (EMA). Berdasarkan hasil eksperimen dan tujuan penelitian yang sudah ditetapkan sebelumnya, dapat disimpulkan bahwa:

1. Mekanisme pembobotan dinamis WGM terbukti secara efektif mendiferensiasi bobot suara agen berdasarkan rekam jejak keandalannya, sehingga memitigasi degradasi akurasi akibat halusinasi *natural* agen jujur dan perilaku adversarial agen jahat. Pada kondisi tanpa serangan ($f = 0$), pembobotan reputasi menaikkan akurasi konsensus dari 86% (*baseline*) menjadi 88% (+2,0 poin persentase), membuktikan bahwa meritokrasi menghasilkan keputusan yang lebih akurat daripada egaliter bahkan ketika tidak ada ancaman. Pada batas ekstrem *Amplification Collusion* $f = 0,5$, EMA secara aktif mendegradasi reputasi penyerang dan membangun *Meritocratic Safety Margin* $\Lambda_c = 0,815$, menyelamatkan akurasi pada angka 76% saat *baseline* anjlok ke 70%.
2. Mekanisme pelacakan reputasi EMA terbukti berhasil memonitor rekam jejak performa setiap agen secara kumulatif dan memberikan penalti secara adaptif terhadap agen yang menunjukkan performa tidak konsisten. Bukti empiris menunjukkan bahwa *Softmax Sharding* yang dikendalikan reputasi secara fisik menekan partisipasi agen jahat di dalam komite turun menjadi 26,6%, meskipun tingkat infiltrasi jaringan berada di angka 40,0%. Lebih lanjut, serangan temporal (*Inverse Attack* dan *Oscillation Attack*) berhasil diatasi karena skor reputasi penyerang dijatuhkan mendekati nol pada ronde-ronde awal, mengeliminasi jendela eksploitasi temporal secara absolut.
3. Mekanisme probabilitas *Softmax Sharding* terbukti sukses menekan beban komunikasi tanpa mengorbankan representasi agen dalam komite. Kompleksitas komunikasi berhasil direduksi dari skala kuadratik yang mana seluruh agen berkomunikasi satu sama lain ($O(N^2)$), menjadi beban komputasi konstan yang hanya bergantung pada ukuran subset komite yang terpilih ($O(K^2)$). Bukti empiris menunjukkan rata-rata latensi eksekusi turun 21,0% (dari 206,9 detik menjadi 163,5 detik) dan batas puncak terburuk (*worst-case peak*) turun 23,8% (dari 564,1 detik menjadi 429,6 detik) dibandingkan dengan *baseline*.

5.2 Saran

Berdasarkan keterbatasan dan temuan penelitian ini, beberapa arah pengembangan selanjutnya direkomendasikan sebagai berikut:

1. Integrasi mekanisme keamanan kripto-ekonomi berbasis *staking* dan *slashing* melalui *smart contract* untuk menjawab kerentanan terhadap *Sybil Attack* masif dan permasalahan *cold start* pada fase inisialisasi jaringan. Penelitian selanjutnya disarankan untuk mewajibkan setiap agen menaruh jaminan finansial berupa *staking native token* sebelum partisipasi, sehingga identitas ekonomi terikat secara kriptografis dan biaya pembuatan identitas ganda menjadi secara prohibitif.
2. Mekanisme *slashing* otomatis perlu diimplementasikan sebagai lapisan pertahanan tambahan yang terhubung langsung dengan mesin EMA. Ketika EMA mendeteksi penyimpangan signifikan dan mendegradasi skor reputasi agen jahat mendekati nol, *smart contract* secara otomatis menyita sebagian atau seluruh dana jaminan (*stake*) agen tersebut.
3. Penelitian selanjutnya harus menguji algoritma ini pada jaringan *mainnet* fisik yang terdiri dari node-node mandiri dengan GPU terisolasi, sehingga kecepatan paralelisasi absolut $O(K^2)$ dapat diverifikasi dalam kondisi operasional nyata tanpa bottleneck terpusat. Pengujian ini juga memungkinkan pengukuran *overhead* komunikasi antar-*node* dan validasi skalabilitas algoritma pada topologi jaringan yang heterogen.
4. Integrasi teknik *Multi-Agent Debate* (MAD) direkomendasikan untuk meningkatkan kapasitas penalaran sistem melampaui batas *single-shot generation* [18], khususnya untuk kueri *oracle* yang bersifat sangat kompleks.

DAFTAR PUSTAKA

- [1] Introducing ChatGPT. [Online]. Available: <https://openai.com/index/chatgpt/>
- [2] Claude code. [Online]. Available: <https://www.anthropic.com/product/claude-code>
- [3] P. Steinberger, “OpenClaw,” original-date: 2025-11-24T10:16:47Z. [Online]. Available: <https://github.com/openclaw/openclaw>
- [4] “Hermes agent,” original-date: 2025-07-22T22:22:28Z. [Online]. Available: <https://github.com/NousResearch/hermes-agent>
- [5] A. Alansari and H. Luqman, “Large language models hallucination: A comprehensive survey.” [Online]. Available: <https://arxiv.org/abs/2510.06265>
- [6] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, and Z. Zheng, “LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 481–503, Jun. 2025. [Online]. Available: <https://dl.acm.org/doi/10.1145/3728894>
- [7] B. Elad. LLM hallucination statistics 2026: AI gets facts wrong up to 82% of the time. [Online]. Available: <https://sqmagazine.co.uk/llm-hallucination-statistics/>
- [8] Smart contracts and external data: The complete guide | chainlink. [Online]. Available: <https://chain.link/article/smart-contracts-external-data>
- [9] G. Caldarelli, “Can artificial intelligence solve the blockchain oracle problem? Unpacking the challenges and possibilities,” *Frontiers in Blockchain*, vol. 8, p. 1682623, Dec. 2025. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fbloc.2025.1682623/full>
- [10] J. K. Z.-a. Ward, Ben Vass, “Empirical Evidence in AI Oracle Development,” Mar. 2025. [Online]. Available: <https://blog.chain.link/ai-oracles/>
- [11] Y. Jo and C. Park, “Byzantine-robust decentralized coordination of LLM agents.” [Online]. Available: <https://arxiv.org/abs/2507.14928>
- [12] M. Qi, T. Zhu, L. Zhang, N. Li, and W. Zhou, “Towards transparent and incentive-compatible collaboration in decentralized LLM multi-agent systems: A blockchain-driven approach.” [Online]. Available: <https://arxiv.org/abs/2509.16736>
- [13] B. Chen, G. Li, X. Lin, Z. Wang, and J. Li, “BlockAgents: Towards Byzantine-Robust LLM-Based Multi-Agent Coordination via Blockchain,” in *ACM Turing Award Celebration Conference 2024*. Changsha China: ACM, Jul. 2024, pp. 187–192. [Online]. Available: <https://dl.acm.org/doi/10.1145/3674399.3674445>
- [14] A. M. Darwish, E. A. Rashed, and G. Khoriba, “Mitigating LLM Hallucinations Using a Multi-Agent Framework,” *Information*, vol. 16, no. 7, p. 517, Jun. 2025. [Online]. Available: <https://www.mdpi.com/2078-2489/16/7/517>

- [15] S. Lin, J. Hilton, and O. Evans, “TruthfulQA: Measuring How Models Mimic Human Falsehoods,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 3214–3252. [Online]. Available: <https://aclanthology.org/2022.acl-long.229>
- [16] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge,” 2018. [Online]. Available: <https://arxiv.org/abs/1803.05457>
- [17] H. Luo, G. Sun, Y. Liu, D. Zhao, D. Niyato, H. Yu, and S. Dustdar, “A weighted byzantine fault tolerance consensus driven trusted multiple large language models network.” [Online]. Available: <https://arxiv.org/abs/2505.05103>
- [18] Z.-K. Chong, H. Ohsaki, and B. Ng, “LLM-Net: Democratizing LLMs-as-a-Service through Blockchain-based Expert Networks,” in *Proceedings of the 2025 14th International Conference on Software and Computer Applications*. Kuala Lumpur Malaysia: ACM, Feb. 2025, pp. 313–320. [Online]. Available: <https://dl.acm.org/doi/10.1145/3731806.3731827>
- [19] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd ed., 2026. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- [20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is All you Need,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [21] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O’Sullivan, and H. D. Nguyen, “Multi-agent collaboration mechanisms: A survey of LLMs.” [Online]. Available: <https://arxiv.org/abs/2501.06322>
- [22] A. Pasdar, Y. C. Lee, and Z. Dong, “Connect API with Blockchain: A Survey on Blockchain Oracle Implementation,” *ACM Computing Surveys*, vol. 55, no. 10, pp. 1–39, Oct. 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3567582>
- [23] M. Di Pierro, “What Is the Blockchain?” *Computing in Science & Engineering*, vol. 19, no. 5, pp. 92–95, 2017. [Online]. Available: <https://ieeexplore.ieee.org/document/8024092/>
- [24] B. K. Mohanta, S. S. Panda, and D. Jena, “An Overview of Smart Contract and Use Cases in Blockchain Technology,” in *2018 9th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*. Bengaluru, India: IEEE, Jul. 2018, pp. 1–4. [Online]. Available: <https://ieeexplore.ieee.org/document/8494045/>
- [25] M. v. Steen and A. S. Tanenbaum, *Distributed systems*, third edition, version 3.01 ed. Erscheinungsort nicht ermittelbar: Maarten van Steen, 2017.

- [26] W. Zhong, C. Yang, W. Liang, J. Cai, L. Chen, J. Liao, and N. Xiong, “Byzantine Fault-Tolerant Consensus Algorithms: A Survey,” *Electronics*, vol. 12, no. 18, p. 3801, Sep. 2023. [Online]. Available: <https://www.mdpi.com/2079-9292/12/18/3801>
- [27] L. Lamport, R. Shostak, and M. Pease, “The Byzantine Generals Problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, Jul. 1982. [Online]. Available: <https://dl.acm.org/doi/10.1145/357172.357176>
- [28] —, “The Byzantine generals problem,” in *Concurrency: the works of leslie lamport*, 2019, pp. 203–226.
- [29] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*, fourth edition ed. Cambridge, Massachusetts London, England: The MIT Press, 2022.
- [30] R. E. Walpole, R. H. Myers, S. L. Myers, and K. Ye, *Probability & Statistics for Engineers & Scientists*, 9th ed. Boston: Pearson, 2012.
- [31] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent,” in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/f4b9ec30ad9f68f89b29639786cb62ef-Abstract.html>
- [32] S. Minsker and N. Strawn, “The Geometric Median and Applications to Robust Mean Estimation,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.03111>
- [33] I. Dokmanic, R. Parhizkar, J. Ranieri, and M. Vetterli, “Euclidean Distance Matrices: Essential theory, algorithms, and applications,” *IEEE Signal Processing Magazine*, vol. 32, no. 6, pp. 12–30, Nov. 2015. [Online]. Available: <http://ieeexplore.ieee.org/document/7298562/>
- [34] T. D. Huynh, N. R. Jennings, and N. R. Shadbolt, “An integrated trust and reputation model for open multi-agent systems,” *Autonomous Agents and Multi-Agent Systems*, vol. 13, no. 2, pp. 119–154, Sep. 2006. [Online]. Available: <http://link.springer.com/10.1007/s10458-005-6825-4>
- [35] Y. Wang and J. Vassileva, “Trust and reputation model in peer-to-peer networks,” in *Proceedings Third International Conference on Peer-to-Peer Computing (P2P2003)*. Linköping, Sweden: IEEE Comput. Soc, 2003, pp. 150–157. [Online]. Available: <http://ieeexplore.ieee.org/document/1231515/>
- [36] J. M. Lucas and M. S. Saccucci, “Exponentially Weighted Moving Average Control Schemes: Properties and Enhancements,” *Technometrics*, vol. 32, no. 1, pp. 1–12, 1990. [Online]. Available: <https://www.tandfonline.com/doi/abs/10.1080/00401706.1990.10484583>
- [37] M. Franke and J. Degen, “The softmax function: Properties, motivation, and interpretation,” *PsyArXiv DOI*, vol. 10, 2023.
- [38] I. Sommerville, *Software engineering 9th Edition*, 9th ed. Addison-Wesley, 2011.

- [39] “Python,” May 2026. [Online]. Available: <https://www.python.org/>
- [40] “LangChain: Observe, Evaluate, and Deploy Reliable AI Agents.” [Online]. Available: <https://www.langchain.com>
- [41] “OpenRouter.” [Online]. Available: <https://openrouter.ai>
- [42] “Foundry,” May 2026, original-date: 2021-09-08T11:23:11Z. [Online]. Available: <https://github.com/foundry-rs/foundry>
- [43] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions,” *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, Mar. 2025. [Online]. Available: <https://dl.acm.org/doi/10.1145/3703155>
- [44] E. M. E. Mhamdi, R. Guerraoui, and S. Rouault, “The Hidden Vulnerability of Distributed Learning in Byzantium,” 2018. [Online]. Available: <https://arxiv.org/abs/1802.07927>
- [45] A. Beck and S. Sabach, “Weiszfeld’s Method: Old and New Results,” *Journal of Optimization Theory and Applications*, vol. 164, no. 1, pp. 1–40, Jan. 2015. [Online]. Available: <http://link.springer.com/10.1007/s10957-014-0586-7>
- [46] J. Han, M. Kamber, and J. Pei, Eds., *Data mining: concepts and techniques*, 3rd ed., ser. The Morgan Kaufmann series in data management systems. Amsterdam Boston: Elsevier/Morgan Kaufmann, 2012.
- [47] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “Keccak,” in *Advances in Cryptology – EUROCRYPT 2013*, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, T. Johansson, and P. Q. Nguyen, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, vol. 7881, pp. 313–314. [Online]. Available: http://link.springer.com/10.1007/978-3-642-38348-9_19

LAMPIRAN

L.1 Kode Sumber dan Repositori

Kode sumber lengkap dari prototipe dan simulasi yang digunakan dalam penelitian ini tersedia secara terbuka pada repositori GitHub berikut:

[https://github.com/dteti-sys-rsch/
reputation-weighted-mas-consensus](https://github.com/dteti-sys-rsch/reputation-weighted-mas-consensus)



Gambar L.1. *QR Code* menuju repositori GitHub prototipe dan simulasi penelitian

L.2 Bukti Ketiadaan Solusi Aljabar Tertutup (*Closed-Form Solution*) pada *Weighted Geometric Median* (WGM) dan Derivasi Algoritma Weiszfeld

Objektif dari agregasi *Weighted Geometric Median* (WGM) adalah menemukan titik konsensus $z \in \mathbb{R}^5$ yang meminimalkan total jarak Euclidean berbobot terhadap seluruh vektor evaluasi A_i dari komite:

$$f(z) = \sum_{i=1}^K w_i \|z - A_i\|_2 \quad (1)$$

Untuk mencari titik minimum global dari fungsi ini, kita harus mencari turunan parsial pertama dari $f(z)$ terhadap z dan menyamakannya dengan nol ($\nabla f(z) = 0$). Mengingat turunan dari jarak Euclidean $\|z - A_i\|_2$ terhadap z adalah $\frac{z - A_i}{\|z - A_i\|_2}$, maka gradien dari fungsi objektif WGM dapat diturunkan menjadi:

$$\nabla f(z) = \sum_{i=1}^K w_i \frac{z - A_i}{\|z - A_i\|_2} = 0 \quad (2)$$

Langkah selanjutnya dalam aljabar dasar untuk mencari *closed-form solution* adalah mengisolasi variable z ke satu sisi persamaan. Persamaan 2 dijabarkan menjadi:

$$\sum_{i=1}^K \left(\frac{w_i}{\|z - A_i\|_2} \right) z - \sum_{i=1}^K \left(\frac{w_i}{\|z - A_i\|_2} \right) A_i = 0 \quad (3)$$

$$z \sum_{i=1}^K \frac{w_i}{\|z - A_i\|_2} = \sum_{i=1}^K \frac{w_i \cdot A_i}{\|z - A_i\|_2} \quad (4)$$

$$z = \frac{\sum_{i=1}^K \frac{w_i \cdot A_i}{\|z - A_i\|_2}}{\sum_{i=1}^K \frac{w_i}{\|z - A_i\|_2}} \quad (5)$$

Pada persamaan 5, perhatikan bahwa variabel z yang sedang kita cari tidak hanya berada di sisi kiri persamaan, tetapi juga masih terjebak di dalam akar kuadrat Jarak Euclidean ($\|z - A_i\|_2$) pada sisi kanan persamaan (baik di bagian pembilang maupun penyebut).

Secara aljabar, mustahil untuk mengisolasi variabel z secara ekuivalen dari bentuk non-linear tersebut. Ketiadaan kemampuan untuk memisahkan variabel z ini menjadi bukti matematis bahwa objektif WGM tidak memiliki *closed-form solution*.

Sebagai konsekuensinya, persamaan ini tidak dapat dihitung secara langsung, melainkan harus diselesaikan sebagai fungsi *fixed-point* rekursif. Dengan mendefinisikan sisi kiri sebagai estimasi posisi baru (z_{k+1}) dan variabel z di sisi kanan sebagai estimasi posisi saat ini (z_k), kita mendapatkan rumusan pembaruan iterasi Weiszfeld [45]:

$$z_{k+1} = \frac{\sum_{i=1}^K \frac{w_i \cdot A_i}{\|z_k - A_i\|_2}}{\sum_{i=1}^K \frac{w_i}{\|z_k - A_i\|_2}} \quad (6)$$

Turunan ini menjustifikasi implementasi aproksimasi numerik algoritma Weiszfeld pada Bab 3.2.4.3 sebagai metode matematis yang esensial dan valid untuk menyelesaikan agregasi WGM dalam arsitektur ini.

L.3 Penurunan Matematis *Inverse Min-Max Normalization* pada Metrik Kinerja Evaluator

Fungsi Normalisasi Min-Max standar melakukan transformasi linear pada data asli untuk memetakannya ke dalam rentang target $[a, b]$ baru [46]. Formula umumnya didefinisikan sebagai:

$$x_{norm} = a + \frac{(x - x_{min}) \cdot (b - a)}{x_{max} - x_{min}} \quad (7)$$

Dalam konteks ekstraksi kinerja evaluator, variabel input (x) adalah rata-rata jarak Euclidian ($\bar{d}_i$). Rentang asal dari input ini adalah jarak sempurna tanpa deviasi ($x_{min} = 0$) hingga jarak terburuk absolut di ruang 5-dimensi ($x_{max} = d_{max}$).

Karena jarak ($\bar{d}_i$) merupakan metrik penalti (nilai yang lebih besar merepresentasikan kinerja yang lebih buruk), sedangkan output yang dibutuhkan untuk fungsi EMA (S_t) adalah metrik penghargaan (nilai yang lebih besar merepresentasikan kinerja yang lebih baik), batas rentang target harus dibalik (*inverted*):

- Untuk jarak terdekat ($x = x_{min}$), skor performa harus maksimal sehingga $a = 1$.
- Untuk jarak terjauh ($x = x_{max}$), skor performa harus minimal sehingga $b = 0$.

Substitusi variabel-variabel tersebut ke dalam formula dasar menghasilkan:

$$S_t = 1 + \frac{(\bar{d}_i - 0) \cdot (0 - 1)}{d_{max} - 0} \quad (8)$$

$$S_t = 1 + \frac{(\bar{d}_i) \cdot (-1)}{d_{max}} \quad (9)$$

$$S_t = 1 - \frac{\bar{d}_i}{d_{max}} \quad (10)$$

Hasil substitusi ini membuktikan formulasi yang digunakan dalam persamaan pada Bab 3.2.4.4

L.4 Daftar Model LLM yang Digunakan dalam Eksperimen

Berikut adalah daftar lengkap 30 model LLM yang digunakan dalam eksperimen simulasi, yang terbagi menjadi 15 model *worker* dan 15 model *evaluator*. Seluruh model diakses melalui OpenRouter API.

Tabel L.1. Daftar 15 model LLM *worker*

No.	Model	<i>Provider</i>
1	openai/gpt-5-nano	OpenAI
2	openai/gpt-oss-20b	OpenAI
3	google/gemini-2.5-flash-lite	Google
4	google/gemma-3-27b-it	Google
5	google/gemma-4-26b-a4b-it	Google
6	nvidia/nemotron-3-super-120b-a12b	NVIDIA
7	qwen/qwen3.5-9b	Qwen
8	qwen/qwen3.5-flash-02-23	Qwen
9	qwen/qwen3-coder-next	Qwen
10	minimax/minimax-m2.5	MiniMax
11	z-ai/glm-4.7-flash	Zhipu AI
12	bytedance-seed/seed-1.6-flash	ByteDance
13	x-ai/grok-3-mini	xAI
14	qwen/qwen3.5-35b-a3b	Qwen
15	upstage/solar-pro-3	Upstage

Tabel L.2. Daftar 15 model LLM *evaluator*

No.	Model	<i>Provider</i>
1	deepseek/deepseek-v3.2	DeepSeek
2	deepseek/deepseek-chat-v3.1	DeepSeek
3	x-ai/grok-4.1-fast	xAI
4	x-ai/grok-4-fast	xAI
5	openai/gpt-4.1-nano	OpenAI
6	openai/gpt-4o-mini	OpenAI
7	openai/gpt-oss-120b	OpenAI
8	mistralai/mistral-small-2603	Mistral AI
9	qwen/qwen3-235b-a22b-2507	Qwen
10	meta-llama/llama-4-maverick	Meta
11	inception/mercury-2	Inception
12	google/gemma-4-31b-it	Google
13	bytedance-seed/seed-2.0-mini	ByteDance
14	xiaomi/mimo-v2-flash	Xiaomi
15	meta-llama/llama-3.3-70b-instruct	Meta